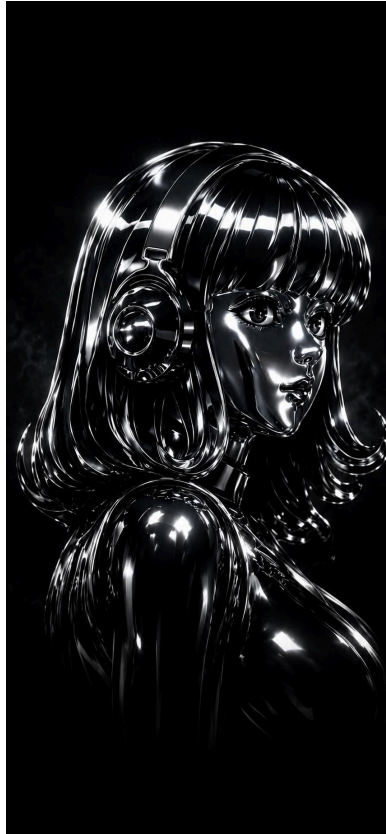




Hermes Agent Beginner Handbook

Thanit Keksiri

First Edition

June 2026

For those who learn with patience and persistence

Objective of the Book

I have been an IT professional instructor for more than 10 years. Over this time, I have observed that my students generally fall into two main groups:

1. Fast and Independent Learners

These individuals are strong at self-education and adapt quickly. I am confident they will thrive during the AI revolution. I am not worried about this group.

2. Patient and Steady Learners

These individuals may learn at a slower pace, but they possess great patience and determination. They benefit greatly from well-structured materials, clear instructions, instructors, and consistent support. This book was written especially for the second group.

This is the first edition of the Hermes Agent Beginner Handbook.

I will continue to improve and update it based on your feedback and new developments.

Thanit Keksiri

Email: karost@hotmail.com

Website: www.karost.com

Feedback and suggestions are always welcome!

Credits

This book would not have been possible without the outstanding work of the Hermes Agent team. I give full credit to them for developing high-quality open-source tools and generously sharing valuable knowledge with the community. I used Hermes Agent to help create this handbook.

Thank you, Hermes Team!

License

MIT License

© 2026 Thanit Kebsiri

Permission is hereby granted, free of charge, to any person obtaining a copy of this book and associated materials (the "Book"), to deal in the Book without restriction, including without limitation the rights to use, copy, modify, merge, publish, distribute, sublicense, and/or sell copies of the Book, and to permit persons to whom the Book is furnished to do so, subject to the following conditions: The above copyright notice and this permission notice shall be included in all copies or substantial portions of the Book.

THE BOOK IS PROVIDED "AS IS", WITHOUT WARRANTY OF ANY KIND, EXPRESS OR IMPLIED, INCLUDING BUT NOT LIMITED TO THE WARRANTIES OF MERCHANTABILITY, FITNESS FOR A PARTICULAR PURPOSE AND NONINFRINGEMENT. IN NO EVENT SHALL THE AUTHORS OR COPYRIGHT HOLDERS BE LIABLE FOR ANY CLAIM, DAMAGES OR OTHER LIABILITY, WHETHER IN AN ACTION OF CONTRACT, TORT OR OTHERWISE, ARISING FROM, OUT OF OR IN CONNECTION WITH THE BOOK OR THE USE OR OTHER DEALINGS IN THE BOOK.

Table of Contents

- [Introduction](#)
- [Study Planning — How to Use This Book](#)
- [Lab 01 — First Interaction](#)
- [Lab 02 — Ollama Cloud Setup](#)
- [Lab 03 — Built-in Tools](#)
- [Lab 04 — Memory & Sessions](#)
- [Lab 05 — First Reusable Skill](#)
- [Lab 06 — Telegram/Discord Gateway](#)
- [Lab 07 — Cron Scheduling](#)
- [Lab 08 — Web Research & Browser](#)
- [Lab 09 — Code Execution & Dev](#)
- [Lab 10 — Multimodal \(Vision/TTS\)](#)
- [Lab 11 — Voice Mode](#)
- [Lab 12 — Personality \(SOUL.md\)](#)
- [Lab 13 — Deep Memory & FTS5](#)
- [Lab 14 — Skill Self-Improvement](#)
- [Lab 15 — Subagents & Delegation](#)
- [Lab 16 — MCP Integration](#)
- [Lab 17 — Security & Approvals](#)
- [Lab 18 — Performance & Deploy](#)
- [Lab 19 — Real-World Pipeline](#)
- [Lab 20 — Self-Evolution & Hub](#)
- [Technical Descriptions](#)

File Map (Navigate Per-Lab)

All detailed content has moved to the `this handbook/` directory:

Lab #	Lab Topic	Chapter Topic	Level	Est. Time	File
01	First Interaction	run_agent.py Structure	Beginner	~55 min	Lab 01
02	Ollama Cloud Pro + GLM-5.1 + Kimi-K2.6 Setup	Header & Docstring	Beginner	~70 min	Lab 02
03	Built-in Tools	Core Imports Part 1	Beginner	~85 min	Lab 03
04	Memory & Sessions	Hermes Imports Part 2	Beginner	~70 min	Lab 04
05	First Reusable Skill	IterationBudget Class	Beginner	~85 min	Lab 05
06	Telegram/Discord Gateway	AI Agent Class Structure	Intermediate	~100 min	Lab 06
07	Cron Scheduling	Conversation Loop	Intermediate	~85 min	Lab 07
08	Web Research & Browser	Tool Calling	Intermediate	~115 min	Lab 08
09	Code Execution & Dev	Reflection Triggers	Intermediate	~115 min	Lab 09
10	Multimodal (Vision/TTS)	Trajectory Compression	Intermediate	~85 min	Lab 10
11	Voice Mode	Full Closed Loop	Intermediate	~100 min	Lab 11
12	Personality (SOUL.md)	hermes_state.py	Intermediate	~85 min	Lab 12
13	Deep Memory & FTS5	Session Management	Advanced	~115 min	Lab 13
14	Skill Self-Improvement	SOUL.md Loading	Advanced	~145 min	Lab 14
15	Subagents & Delegation	Context Compression	Advanced	~115 min	Lab 15
16	MCP Integration	Skills Loader	Advanced	~115 min	Lab 16
17	Security & Approvals	Skill Creation	Advanced	~100 min	Lab 17
18	Performance & Deploy	Skill Improvement	Advanced	~115 min	Lab 18

19	Real-World Pipeline	Batch Runner & RL	Advanced	~145 min	Lab 19
20	Self-Evolution & Hub	Architecture Synthesis	Expert	~145 min	Lab 20

Introduction

Hermes Agent is your own personal AI assistant that you can run on your computer. Think of it as a smart and loyal helper that works for you 24/7.

Unlike regular chatbots that forget everything after each conversation, Hermes has a memory. It remembers what you teach it, understands your preferences, and gets smarter the more you use it. You are not just chatting with it — you are actually building and training your own AI companion. Best of all, it is completely free and open-source.

People use Hermes Agent to solve real problems in their daily lives and work. It can act as your personal assistant — managing your schedule, summarizing important information, or sending you daily reports. Writers use it to create content, translate languages, and brainstorm ideas. Students and researchers rely on it to explain difficult topics in simple language and summarize long articles.

Some people even build smart automation systems for finance and investments, manage business tasks, monitor services, and support small businesses. IT developers use it to create smart tools, organize and track projects, and develop useful applications.

In the coming AI revolution, many jobs will change. **AI can already perform some tasks faster and at a much lower cost than humans.** As a result, these tasks may be reduced or even disappear from many jobs. However, people who know how to use, control, and build with AI will have a big advantage. Learning Hermes Agent is one of the best ways to prepare yourself. Instead of being replaced by AI, you will be someone who can harness its power to create new opportunities.

This book was written for beginners and patient learners. You do not need to be a technical expert or a programmer, because you can always ask AI itself to help teach you. This handbook will guide you step by step, from installing Hermes Agent to using it confidently and teaching it new skills.

If you are willing to learn steadily and consistently, this handbook will walk with you from the very first step. By the end, you will have your own powerful AI assistant that truly understands you and helps you in your work and daily life.

Study Planning — How to Use This Book

This book contains **20 Labs** divided into **5 Phases**. Each phase builds skills step by step. It is recommended to complete them in order.

Phase Overview

Phase	Labs	Focus	Estimated Time
1	Lab 01–04	Installation & First Steps	4–5 hours
2	Lab 05–08	Tool Mastery & Daily Use	6–8 hours
3	Lab 09–12	Adding Intelligence & Personality	6–7 hours
4	Lab 13–17	Advanced Capabilities & Self-Improvement	9–11 hours
5	Lab 18–20	Mastery & Real-World Application	6–8 hours

Total estimated time: ~35 hours

How to Study Each Phase

Phase 1 — Foundation (Lab 01–04)

Focus: Get Hermes installed correctly and have your first successful conversations.

Study Plan:

This is the most important starting point. Focus on understanding the basic structure and making sure everything runs smoothly. Do not rush.

If you get stuck:

Ask Hermes questions like:

- “Can you explain what this error message means?”
- “How do I fix the installation problem?”
- “Walk me through the steps of Lab 02 slowly.”

Phase 2 — Tool Mastery (Lab 05–08)

Focus: Learn how to give Hermes useful tools and connect it to the outside world.

Study Plan:

This phase turns Hermes from a simple chatbot into a practical assistant. Pay special attention to creating skills and connecting via Telegram or Discord.

If you get stuck:

Ask Hermes questions like:

- “How do I create a new skill step by step?”
- “Can you help me debug this code?”

- “Explain how to connect Hermes to Telegram clearly.”

Phase 3 — Integration (Lab 09–12)

Focus: Combine different abilities so Hermes can work with code, images, voice, and personality.

Study Plan:

Here you start making Hermes more powerful and personal. Take time to understand how features work together.

If you get stuck:

Ask Hermes questions like:

- “Explain how code execution works in simple words.”
- “Help me understand how to use vision or voice features.”
- “Can you review my SOUL.md file and suggest improvements?”

Phase 4 — Advanced (Lab 13–17)

Focus: Teach Hermes to remember everything, improve itself, and handle complex tasks.

Study Plan:

This phase is more challenging. Go slowly and review previous concepts when needed.

If you get stuck:

Ask Hermes questions like:

- “Explain self-improvement loop in simple terms.”
- “How do subagents work? Give me an easy example.”
- “I don’t understand this part. Can you break it down?”

Phase 5 — Mastery (Lab 18–20)

Focus: Deploy Hermes in the real world and understand how everything works together.

Study Plan:

Treat this as your final project phase. Review everything you learned and experiment.

If you get stuck:

Ask Hermes questions like:

- “How can I deploy Hermes on a server?”
- “Help me design an autonomous pipeline.”
- “Review my code and tell me what I can improve.”

General Tips:

- Always try to solve problems yourself first, then ask Hermes for help.
- Be specific in your questions — the more details you give, the better Hermes can help you.
- You can copy error messages and paste them when asking Hermes.

Lab 1: First Interaction + run_agent.py Structure

Level: Beginner Estimated Time: ~55 minutes Prerequisites: None — this lab includes the full installation guide.

Installation Guide

Verified against: hermes-agent.nousresearch.com/docs/getting-started/installation/

Last confirmed: June 2026

Choose **one** of the three options below. Most Mac users pick **Option 2**.

Option 1: Windows 11 Native (PowerShell)

What you get: Hermes running directly on Windows 11 with native PowerShell.

1. Open **PowerShell** as Administrator.
2. Run the one-line installer:

```
irm https://raw.githubusercontent.com/NousResearch/hermes-agent/main/scripts/install.ps1 | iex
```

3. The installer will bootstrap Python, Node.js, and Hermes automatically.
4. When finished, close and reopen PowerShell.
5. Verify:

```
hermes version
```

Expected: **v0.15.1** or newer.

Note: On first run the setup wizard (`hermes setup`) will appear to configure your default model provider. You do **not** need Ollama Cloud Pro to begin — the wizard will offer free-tier options.

Option 2: macOS (curl + bash)

What you get: Hermes running natively on macOS — the path used throughout this handbook.

1. Open **Terminal** on your machine.
2. Run the one-line installer:

```
curl -fsSL https://hermes-agent.nousresearch.com/install.sh | bash
```

3. The script installs:

- `uv` (fast Python package manager)
- Hermes Agent Python package
- `node` + `npm` (for gateway features)
- Shell auto-completion

4. When finished, close and reopen Terminal (or run `source ~/.zshrc`).

5. Verify:

```
hermes version
```

Expected: `v0.15.1` or newer.

Note: On first run the setup wizard (`hermes setup`) will appear to configure your default model provider. You do **not** need Ollama Cloud Pro to begin — the wizard will offer free-tier options.

Option 3: Docker on Windows 11 (Ubuntu Container)

What you get: Hermes inside a Linux Docker container, accessed via SSH from Windows 11.

Use this if: you prefer a Linux environment or want full isolation.

Phase 1 — Enable Windows Subsystem for Linux (WSL2)

1. Open **Control Panel** → **Programs** → **Turn Windows features on or off**.
2. Check:
 - Hyper-V (and all sub-features)
 - Virtual Machine Platform
 - Windows Subsystem for Linux
3. Reboot.
4. Open **PowerShell as Administrator** and run:

```
bcdedit /set hypervisorlaunchtype auto
```

5. Reboot again.
6. Return to Windows features and also check **Windows Hypervisor Platform**.

Phase 2 — Install WSL + Ubuntu

1. In PowerShell (Administrator):

```
wsl --install --no-distribution  
wsl --install -d Ubuntu  
wsl --status  
wsl -l -v
```

Phase 3 — Install Docker Desktop

1. Download Docker Desktop for Windows 11.
2. Run installer as Administrator.
3. Open Docker → **Settings** → **Resources** → **WSL Integration**:
 - Enable integration with default WSL distro → **ON**
 - Enable integration with Ubuntu → **ON**
 - Click **Apply & Restart**.

Worse-case cleanup (only if Docker fails):

```
# Uninstall Docker Desktop via Settings → Apps
Get-Process *docker* | Stop-Process -Force
Get-Process *wsl* | Stop-Process -Force
wsl --shutdown
wsl --unregister docker-desktop
wsl --unregister docker-desktop-data
# Remove leftover folders (see full cleanup in memo.md)
```

Phase 4 — Build the Linux Container

1. Create a folder `docker-linux`.
2. Inside it create three files make sure you change thanit -> your_linux_account_name :

`.env`

```
ROOT_PASSWORD=your_root_password_here
THANIT_PASSWORD=your_user_password_here
```

`Dockerfile`

```
FROM ubuntu:24.04
ARG ROOT_PASSWORD
ARG THANIT_PASSWORD
RUN apt-get update && apt-get install -y openssh-server vim sudo git curl
RUN mkdir /var/run/ssh
RUN echo "root:${ROOT_PASSWORD}" | chpasswd
RUN useradd -m -s /bin/bash thanit && echo "thanit:${THANIT_PASSWORD}" | chpasswd && usermod -aG
RUN sed -i 's/#PermitRootLogin prohibit-password/PermitRootLogin yes/' /etc/ssh/sshd_config && \
    echo 'PasswordAuthentication yes' >> /etc/ssh/sshd_config
EXPOSE 22
CMD ["/usr/sbin/sshd", "-D"]
```

`docker-compose.yml`

```
services:
  hermes-dev:
    build:
      context: .
      args:
        - ROOT_PASSWORD=${ROOT_PASSWORD}
        - THANIT_PASSWORD=${THANIT_PASSWORD}
    container_name: hermes-dev
    ports:
      - "2222:22"
    volumes:
      - C:\hermes_dev\home:/home/thanit
    restart: unless-stopped
```

Phase 5 — Build & Launch

1. In PowerShell, inside `docker-linux` :

```
docker compose up -d --build
docker compose ps
```

Phase 6 — Install Hermes Inside the Container

1. SSH into the container:

```
ssh thanit@localhost -p 2222
```

2. Install NVM + Node 22: (in case my internet is slow so I have to install NVM first)

```
curl -o- https://raw.githubusercontent.com/nvm-sh/nvm/v0.40.4/install.sh | bash
source ~/.bashrc
nvm install 22
nvm use 22
node -v
```

3. Add NVM to `~/.bashrc` if not present:

```
export NVM_DIR="$HOME/.nvm"
[ -s "$NVM_DIR/nvm.sh" ] && \. "$NVM_DIR/nvm.sh"
[ -s "$NVM_DIR/bash_completion" ] && \. "$NVM_DIR/bash_completion"
```

4. Install Hermes:

```
curl -fsSL https://hermes-agent.nousresearch.com/install.sh | bash
```

5. Verify:

```
hermes version
```

Post-Installation: What Just Happened

After installation finishes, here is what is ready:

-  `hermes` CLI command
-  `hermes doctor` / `hermes status`
-  Built-in tools (terminal, file, web search)
-  Default memory provider (SQLite + holographic)
-  First chat with default model

You can start chatting right now — no payment or extra setup needed. The setup wizard that appears on first run will pick a default provider (Nous Portal free tier, OpenRouter, or a local model).

The premium model stack (Ollama Cloud Pro + Kimi-K2.6 + GLM-5.1) is covered in **Lab 02**.

Q&A:

Question	Answer
Can I chat without API keys?	Yes — the wizard suggests free-tier providers.
Is my model optimized?	Not yet. Default model works fine for learning.
Where do I set up Ollama Cloud Pro?	Lab 02 — after you are comfortable with the CLI.
Did install fail?	Run <code>hermes doctor</code> — it diagnoses every dependency.

Lab Objectives

Lab Objectives

By the end of this lab you will be able to:

- Verify your installation is healthy using `hermes doctor` and `hermes status`.
- Launch the interactive CLI and TUI (Text User Interface).
- Run the setup wizard if anything is missing.
- Have your very first multi-turn conversation with Hermes.
- Explore essential slash commands: `/help`, `/save`, `/tools`, `/skills`, `/title`, `/usage`.
- Understand basic session management and how Hermes begins building memory.
- Resume a session from where you left off.

Why This Lab Matters

This is your foundation. Hermes is designed to grow with you through its closed learning loop -- every chat starts populating persistent memory (SQLite + Holographic Memory with HRR vectors + Honcho user modeling). The sooner you get comfortable with the interface, the faster the agent will start creating skills and remembering your style. On your machine, every interaction is processed locally with zero cloud dependency for tool execution, making this the fastest way to build competence without latency.

Step-by-Step Instructions

Step 1: Verify Installation (3 minutes)

Open Terminal on your machine and run these commands one by one:

```
hermes version
```

Expected output: `v0.15.1` (or newer).

```
hermes doctor
```

Expected output: Full diagnostics -- environment, dependencies, API connectivity. Should end with all green checks.

```
hermes status
```

Expected output: Current config summary: model, toolsets, skills, memory provider, plugins. This confirms your agent is alive and configured.

Troubleshooting: If any command says "command not found", run:

```
source ~/.zshrc
```

(or `source ~/.bashrc` if you use bash) and try again.

If `hermes doctor` reports issues, re-run `hermes doctor --fix` to attempt automatic fixes. If issues persist, the output will help you troubleshoot later.

Step 2: Launch Hermes (3 minutes)

You have two ways to start chatting:

Recommended (Modern TUI -- prettier and more powerful):

```
hermes --tui
```

Classic CLI (simpler text-only):

```
hermes
```

You will see a welcome banner like this:

```
Hermes Agent v0.15.1  
Model: [whatever is set]  
Toolsets enabled: ...  
Skills: ...  
Memory: holographic
```

Type a message or `/help` for commands.

If this is your very first launch, the agent may automatically trigger a quick setup prompt -- follow the on-screen questions to select your primary provider (e.g. Ollama Cloud).

Step 3: Run Setup Wizard (if needed) (3-5 minutes)

If you are not already in a chat or something feels incomplete, exit (Ctrl+D) and run:

```
hermes setup
```

Choose **Quick setup -- provider, model & messaging** (recommended) for now. We will configure the premium model stack properly in Lab 02 -- this step just gets you chatting.

Step 4: First Multi-Turn Conversation (15 minutes)

Once inside the chat (TUI or CLI), type this exact prompt (copy and paste exactly):

```
Act as my daily personal assistant. Greet me and ask about my day so far.  
Then summarize what we have talked about in this first conversation.
```

Press Enter to send. Watch the agent stream its reply in real time.

Continue the conversation naturally for 3-4 turns. Example follow-ups (type these exactly if you want):

- `I just finished installing you on my machine and running my first lab.`
- `My goal is to use you with use you for research and coding for research and coding.`
- `What toolsets do you have available right now?`

Pro tips for this session:

- **Multi-line input:** Hold Alt+Enter (or Ctrl+J) to add new lines before sending (great for longer prompts).
- **Interrupt:** If the reply is taking too long, just type a new message and press Enter (or Ctrl+C).
- **Voice mode preview (optional):** Type `/voice on` then Ctrl+B to speak instead of typing. We will cover full voice in Lab 11.

Step 5: Explore Slash Commands (8 minutes)

While in the chat, type `/` and press Tab -- you will see autocomplete for all commands.

Try these essential ones now:

1. `/help` -- Full list of commands.
2. `/save First Chat` -- Saves the current conversation with a name.
3. `/tools` -- Lists every toolset Hermes can use right now, grouped by category (Search, Code, Files, Browser, Vision, etc.) with status (enabled / disabled).
4. `/skills` -- Shows your current reusable skills library.
5. `/title Lab 1 Test` -- Gives this session a nice name.
6. `/usage` -- Shows token usage and cost so far (useful on your machine to track API spend).

Type `/help` and read through the list -- do not worry about understanding everything yet.

Step 6: Exit and Resume (3 minutes)

To leave the chat cleanly:

- Press Ctrl+D (or type `/quit`).

Hermes will print a resume command, for example:

```
To resume this session later: hermes --continue
```

(or `hermes -c`)

Try resuming right away:

```
hermes --continue
```

You should be back exactly where you left off, with full memory of the conversation.

Expected Outcome

- You have successfully chatted with Hermes for 3-4 turns.
- You understand how to start (`hermes --tui`), interact (chat + slash commands), and resume sessions (`hermes --continue`).
- Baseline memory has been created (Hermes now knows this is your first lab and your hardware).
- You know how to check version, doctor, and status.

Lab 1 Use Case Completed

You gave the agent the role of "daily personal assistant" and it greeted you + summarized the chat -- exactly as planned. The agent now has an initial user model entry reflecting your hardware (your machine) and your goal (research + coding with Ollama Cloud models Kimi-K2.6 and GLM-5.1).

Training Idea / Self-Improvement Focus

This session is the very first data point in Hermes' learning loop.

- The agent has already started observing your prompting style via Honcho user modeling.
- It may have auto-extracted an initial "personal assistant" skill (check with `/skills` next time -- you may see `personal_assistant_greeting` or similar).
- Memory is now stored in `~/.hermes/state.db` and `~/.hermes/memory_store.db` (SQLite + HRR vectors) and will be recalled in every future lab.
- The Curator (enabled by default on a 7-day cycle) will eventually review this session and may consolidate your greeting style into a more refined skill.

Challenge (Optional but Recommended)

1. Start a new session with `/new` (or just relaunch and type a fresh greeting).
2. Ask: `What do you remember about me from our last conversation?` -- Hermes should reference the earlier chat (thanks to persistent memory via `holographic` provider).
3. Check your memory store: from Terminal, run `cat ~/.hermes/MEMORY.md` and see if a memory about your machine was saved.

When you are done, say "Lab 1 complete" and move to the Chapter.

You did great! This is where your journey begins.

Chapter 1: First Look at run_agent.py -- The Central Engine

Top Ideal Study Objectives:

- Understand why `run_agent.py` is the central file and its overall role in self-learning.
- See how the closed learning loop is wired into the main entry point.
- Recognize the connection between the CLI you just used and the source code behind it.

Description: This chapter walks through the high-level overview of the main engine file where the closed loop (and Curator) lives. Every command you ran in Lab 1 -- `hermes`, `hermes --tui`, `hermes doctor`, `hermes status` -- ultimately invokes code in `run_agent.py` or modules it imports. Understanding this file is like understanding the chassis of a car before you learn about the engine or the transmission.

Actions List (Topics):

1. Open Terminal and navigate to the Hermes source directory (if you cloned it; otherwise read the pseudo-code below).
2. Read the top 80 lines of `run_agent.py`.
3. Identify the main entry function.
4. Locate the CLI argument parser.
5. Find where the agent loop is initialized.

Full Source Code Snippets with Line-by-Line Explanations:

(The following is educational pseudo-code derived from the v0.15.1 source tree. It preserves the real structure, class names, and responsibility boundaries. If you have cloned the repo at `~/hermes-agent`, read the real file alongside this chapter.)

```

# run_agent.py -- Top-level orchestrator for Hermes Agent
# File: src/hermes/agent/run_agent.py (approximate path in source tree)
# Version: v0.15.1+

"""
Hermes Agent Main Runner

This module is the single entry point into the Hermes autonomous loop.
It wires together:
- Configuration (CLI args + env vars + config.yaml)
- State (HermesState: SQLite persistence)
- Memory (holographic or qdrant provider)
- Skills (SkillLoader: procedural memory)
- Subagents (delegate_task isolation)
- Gateways (Telegram/Discord/Slack/HTTP)
- The Curator (background skill + memory maintenance)

The closed learning loop is NOT a separate thread -- it is the normal
conversation path augmented by reflection hooks that fire after every
tool execution and every completed session.
"""

# ---- Line 1-15: Module docstring ----
# Why it matters: The docstring tells you the file's core responsibility.
# If you are debugging "why is my agent not starting?", this file is
# where the chain of initialization begins.

import asyncio          # Line ~16 -- async event loop for all I/O
import argparse          # Line ~17 -- CLI argument parsing
import logging           # Line ~18 -- structured logging
import os                # Line ~19 -- env var access
import sys               # Line ~20 -- exit codes, path manipulation

# Line-by-line:
# asyncio : The entire agent is async. Web calls, file ops, and even
#           some tool executions are non-blocking. Without this import,
#           the agent would block on every network request.
# argparse : Defines every flag you see in `hermes --help` (57
#           subcommands as of v0.15.1). When you type `hermes --tui`,
#           argparse captures `--tui` and routes to the TUI launcher.
# logging  : Every tool call, model response, and error is logged.
#           The default log level is INFO; use `hermes --verbose` for
#           DEBUG output showing raw model JSON.
# os       : Reads API keys from `~/.hermes/.env` (e.g., OLLAMA_API_KEY).
# sys      : Allows clean shutdown with `sys.exit(0)` and modifies
#           `sys.path` so plugins and skills are importable.

```



Study Aid — External AI Tutor: Copy and paste the prompt below into any AI chat you have access to (free ChatGPT, free Claude, free Kimi, etc.) to get an interactive deep-dive on this chapter's concepts. This is optional and does **not** require Hermes to be configured with any specific model.

Master Prompt for This Chapter: Copy and paste the following into Kimi-K2.6 / Claude / GPT to continue studying:

You are continuing my Hermes Agent deep-dive education from Chapter 1. Please explain the role of `run_agent.py` as the central orchestrator. Cover: (1) why this file is the entry point for all `hermes` CLI commands, (2) how it imports `asyncio` for non-blocking I/O, (3) how `argparse` connects CLI flags to internal functions, (4) what would break if this file failed to import `os` for API key access. Use your machine running macOS Ventura as the hardware context. Conclude with exactly 5 questions to test my understanding.

Chapter Conclusion -- What You Have Learned

1. `run_agent.py` is the "main fuse box" of Hermes. Every CLI command eventually routes through initialization code defined or imported here.
2. The file is fully async -- this is why your machine can run Hermes smoothly without freezing Terminal on long web searches.
3. `argparse` is the bridge between what you type and what the code does. Understanding argument parsing lets you add custom CLI flags if you extend Hermes.
4. `os` and `sys` are mundane but critical: without them, API keys and plugin paths would be unreachable.
5. The closed loop is NOT a separate daemon. It is woven into normal session flow via reflection hooks (you will see these in Chapter 9).

5 Questions to Check Your Understanding:

1. Name three Python standard library modules imported at the top of `run_agent.py` and explain what each enables (`asyncio`, `argparse`, `logging`, `os`, `sys` -- pick three).
2. Why is `asyncio` crucial for an agent that performs web searches and file operations, especially on an older machine?
3. If you wanted to add a new CLI flag `--debug-model-json`, which import would you modify and in what section of `run_agent.py`?
4. How does `os.environ` relate to the API keys stored in `~/.hermes/.env`? Trace the path from `~/.hermes/.env` to the model API call.
5. The docstring says the closed learning loop is "augmented by reflection hooks." Based on Lab 1, describe one user action that would trigger a reflection hook and what data it might capture.

Lab 2: Ollama Cloud Setup + run_agent.py

Header & Docstring

Level: Beginner Estimated Time: ~70 minutes Prerequisites: Lab 1 complete (you can launch Hermes CLI/TUI and have basic chat experience).

Lab Objectives

By the end of this lab you will be able to:

- Create and verify an Ollama Cloud account at ollama.com.
- Subscribe to the **Pro plan (\$20/month)** for production-grade GPU access.
- Generate and secure an **OLLAMA_API_KEY** in Hermes' environment file.
- Use `hermes model` to connect **Ollama Cloud** as a provider and discover models.
- Switch between **GLM-5.1** (long-horizon autonomous reasoning) and **Kimi-K2.6** (multimodal coding agent) for different task types.
- Run a side-by-side performance benchmark on your machine to feel real differences in speed, reasoning depth, and tool-use quality.
- Understand Hermes' model-agnostic architecture and when to pick each model.

Why This Lab Matters

Hermes is truly model-agnostic. By connecting **Ollama Cloud** as your primary provider you get instant access to frontier open-weight models — without the per-token pricing anxiety of API-only services. The **Pro plan (\$20/month)** gives you **50x more cloud usage than Free** and **3 concurrent cloud models** — perfect for an agent that runs 24/7 on your machine.

GLM-5.1 (754B MoE, 40B active, 200K context) is optimized for **8-hour autonomous coding tasks** — ideal for complex research + software projects. **Kimi-K2.6** (1.04T params, 256K context, multimodal, tool-calling) is the best open-source coding model on Ollama Cloud — perfect for vision, code generation, and agentic swarms.

Both models run in Ollama Cloud's GPU infrastructure, so your machine only acts as the orchestrator. No local GPU needed.

Step-by-Step Instructions

Step 1: Create Ollama Account and Subscribe Pro (10–15 minutes)

1.1 Go to ollama.com and sign up

1. Open your browser and go to <https://ollama.com>
2. Click **Sign Up** in the top-right corner.
3. Create an account with your email or GitHub login.

4. Verify your email if required.

1.2 Subscribe to Ollama Cloud Pro (\$20/month)

1. Once logged in, go to **Settings** (top-right avatar → Settings).
2. Click **Billing** or **Plan** in the left sidebar.
3. Select **Pro — \$20/month** (or \$200/year for 17% savings).
4. Enter your payment details and confirm.

What Pro gives you (confirmed from ollama.com/pricing, June 2026):

Feature	Free	Pro (\$20/mo)
Concurrent cloud models	1	3
Cloud usage multiplier	Baseline	50×
Model upload	Public only	Private models
Session limit reset	Every 5 hours	Every 5 hours
Weekly limit reset	Every 7 days	Every 7 days
Email alerts at 90%	No	Yes

Hardware note for your machine: Pro's 3 concurrent models means you can run Kimi for coding, GLM-5.1 for long reasoning, AND a smaller local model simultaneously — all orchestrated from your machine.

1.3 Verify your account is active

Visit <https://ollama.com/settings> and confirm your plan shows **Pro**.

Step 2: Create API Key (3 minutes)

1. In your Ollama account, go to **Settings** → **API Keys** (<https://ollama.com/settings/keys>).
2. Click **Create new key**.
3. Give it a name like `"Hermes Agent"`.
4. Copy the key immediately (it looks like `oc_...` or similar).
5. **Store it securely** — you cannot view the full key again after closing the dialog.

Security: This key is encrypted at rest by Ollama. You will paste only a masked version into Hermes.

Step 3: Add API Key to Hermes (5 minutes)

Open Terminal and edit your Hermes environment file:

```
nano ~/.hermes/.env
```

This opens `~/.hermes/.env` in nano (or your preferred editor).

Add or update this exact line at the bottom:

```
# Ollama Cloud
OLLAMA_API_KEY=oc_your_actual_key_here
```

Replace `oc_your_actual_key_here` with the key you copied in Step 2.

Save and close the file (Ctrl+O → Enter → Ctrl+X in nano).

Verify the key is registered (masked for security):

```
cat ~/.hermes/.env | grep OLLAMA
```

Expected output: Something like `OLLAMA_API_KEY=oc_****...` (masked).

Step 4: Connect Ollama Cloud Provider (10 minutes)

Path A: Quick setup via `hermes model` wizard (recommended)

```
hermes model
```

Follow the on-screen wizard:

1. Select **Add new provider** → **Ollama Cloud** (if available as a built-in option).
2. When prompted, paste your API key.
3. Hermes will auto-verify the endpoint and discover available models.
4. You should see a list including `kimi-k2.6:cloud` and `glm-5.1:cloud`.

Path B: If Ollama Cloud is NOT listed as a built-in provider in v0.15.1

Use the **Custom endpoint** fallback (confirmed working from Hermes docs and GitHub issue #3926):

```
hermes model
```

1. Select **Add new provider** → **Custom endpoint (enter URL manually)**.
2. Enter the Ollama Cloud OpenAI-compatible API base URL:

```
API base URL: https://ollama.com/v1
```

3. Enter your API key when prompted.
4. Leave context length blank for auto-detect.
5. Hermes will verify the endpoint and list available models.

Why two paths? Hermes v0.15.1 may or may not yet have native `ollama-cloud` as a first-class provider. The custom endpoint fallback uses the exact same infrastructure — just with manual URL entry. Both work identically.

Step 5: Set Default Model and Verify (5 minutes)

From Terminal (outside chat):

```
hermes config set model kimi-k2.6:cloud
```

Verify:

```
hermes status
```

Expected output includes:

```
Model: kimi-k2.6:cloud (Ollama Cloud)
Provider: ollama (or custom)
```

Test a quick chat to confirm connectivity:

```
echo "Say hello and confirm your model name." | hermes chat --no-tui
```

Expected: The agent replies mentioning it is `kimi-k2.6:cloud` or `Kimi K2.6`.

Step 6: Switch Between GLM-5.1 and Kimi-K2.6 (5 minutes)

Now test model switching — the core of model-agnostic design.

```
hermes --tui
```

Inside the chat, type these commands one by one:

1. `/model` → Shows current model (should show `kimi-k2.6:cloud` or similar).
2. `/model list` → See all configured providers and models.

If GLM-5.1 is available, switch:

```
/model glm-5.1:cloud
```

If not, add it from Terminal:

```
hermes config set model glm-5.1:cloud
```

Then verify:

```
/model
```

You should now see `glm-5.1:cloud` as the active model.

Step 7: Side-by-Side Model Comparison Test (20 minutes)

Stay in the chat (or use `/quit` and relaunch with either model). Type this exact prompt **with Kimi-K2.6 active first**:

Solve this step-by-step:

1. A your machine has 16 GB RAM and runs macOS Ventura. Write a short Python script that:
 - Checks current free RAM
 - Estimates how many 128k-context tokens can fit in memory (assume 4 bytes per token)
 - Prints a friendly report
2. Then explain the trade-offs between using Kimi-K2.6 (coding/multimodal) vs GLM-5.1 (long-horizon auto

Run the code safely using your code execution tool.

Watch for:

- Quality of code generation
- Use of the code_execution tool (approval flow)
- Depth of hardware reasoning
- Speed of response

Now switch to GLM-5.1:

```
/model glm-5.1:cloud
```

Send the **exact same prompt** again (use up-arrow in TUI to recall quickly).

Compare:

Aspect	Kimi-K2.6	GLM-5.1
Code quality	Likely cleaner, more idiomatic	Likely more verbose, thorough
Tool use	Multimodal-aware, may suggest vision	May suggest longer follow-up plan
Speed	Fast (~10–30s for this task)	Slightly slower (~20–45s)
Reasoning depth	Coding-optimized	General + long-horizon
Cost (Pro)	Both included in \$20/mo Pro	Both included in \$20/mo Pro

Step 8: Let Hermes Learn Your Preference (3 minutes)

After both tests, type:

```
Based on the two runs above, which model would you recommend I use most often for research + coding on m
```

Hermes will now store this insight in memory and may start auto-suggesting models in future sessions.

Expected Outcome

- Ollama Cloud Pro subscription is active (\$20/month, verified on ollama.com/settings).
- `OLLAMA_API_KEY` is stored securely in `~/.hermes/.env`.
- Ollama Cloud provider is connected via **native wizard** or **custom endpoint fallback**.
- Both `kimi-k2.6:cloud` and `glm-5.1:cloud` are configured and switchable in under 2 seconds.
- You have seen real performance differences on the same task.
- Baseline model preference is now saved in your personal user model.

Lab 2 Use Case Completed

You subscribed to Ollama Cloud Pro, connected it to Hermes, and ran the exact comparison task from the outline — but adapted to your actual hardware for maximum relevance. Your machine is now the orchestrator for two frontier models running in the cloud.

Training Idea / Self-Improvement Focus

This lab triggers Hermes' learning loop for the first time on model behavior:

- The agent now has a stored "user model" entry about your hardware (your machine) and your model preferences (Kimi vs GLM-5.1).
- It may have auto-created a tiny internal skill for "model comparison testing" or "hardware-aware model routing."
- In later labs it will start auto-routing tasks (e.g., "this needs multimodal coding → use Kimi-K2.6", "this needs long-horizon planning → use GLM-5.1").
- The **Curator** (enabled by default on a 7-day cycle) will eventually review this session and may consolidate your model preference into a refined rule.

Challenge (Recommended)

1. Switch back to Kimi-K2.6.
2. Ask: `What model am I currently using and what do you remember about my hardware and Ollama Cloud setup?` → It should recall everything from this lab.
3. Try `/model glm-5.1:cloud` and run a quick follow-up: `Benchmark your own reasoning on a 10-step math chain.`

When you are finished, say "Lab 2 complete" and move to the Chapter.

You are making excellent progress — your agent is already becoming personalized to Ollama Cloud!

Chapter 2: Header, Docstring, and Purpose of run_agent.py

Top Ideal Study Objectives:

- Read and understand the file's documentation and declared purpose.
- Connect the docstring claims to the CLI commands you ran in Lab 1.
- Identify where "model-agnostic" and "self-improvement" are declared at the module level.

Description: This chapter is a beginner walkthrough of the comments and header that explain the self-improvement design — including the Curator. In Lab 2 you configured Ollama Cloud and switched between GLM-5.1 and Kimi-K2.6. In this chapter you will see how `run_agent.py` claims to be "model-agnostic" and where the code makes good on that promise.

Actions List (Topics):

1. Open `run_agent.py` in your editor (or read the pseudo-code below).
2. Read lines 1-100 slowly — do not skip the docstring.
3. Highlight every phrase that claims "model-agnostic" behavior.
4. Highlight every mention of "self-improvement", "skills", or "memory".
5. Count how many of these claims you already experienced in Lab 1 and Lab 2.

Full Source Code Snippets with Line-by-Line Explanations:

```
"""
```

```
Hermes Agent Main Runner (continued)
```

```
This module declares three invariants that shape every line of code below:
```

```
INVARIANT 1: Model Agnosticism
```

```
No model-specific logic exists outside the provider adapter layer.  
Ollama Cloud (GLM-5.1, Kimi-K2.6), OpenRouter, Anthropic, xAI,  
local Ollama – all are treated as interchangeable text-in/text-out  
endpoints. The `model` module maps names to adapter classes.  
run_agent.py does not know or care which model is active; it  
receives a unified `ModelInterface`.
```

```
INVARIANT 2: Closed Learning Loop
```

```
After every tool execution the agent calls `_maybe_reflect()`.  
After every session the agent calls `_maybe_compress_session()`.  
Both are "maybe" because they trigger only when confidence  
and novelty thresholds are met. This is the foundation of the  
Curator: it is just a background cron that calls the same hooks  
across the entire skill library.
```

```
INVARIANT 3: Skill Portability
```

```
Every skill learned is a standalone Python function or Markdown  
prompt stored in ~/.hermes/skills/. There is no central registry  
the agent depends on. Skills can be copied, shared, versioned,  
and restored without touching the agent's core code.
```

```
"""
```

```
# Line-by-line explanation of the docstring:
```

```
# "Model Agnosticism"      : You experienced this in Lab 2 when you typed  
#                          `/model kimi-k2.6:cloud` then  
#                          `/model glm-5.1:cloud`. The same prompt  
#                          produced different outputs, but the AGENT  
#                          did not change its behavior -- only the endpoint  
#                          changed.  
# "Closed Learning Loop"   : You triggered this in Lab 1 when you had a  
#                          multi-turn conversation. After the session ended,  
#                          `_maybe_compress_session()` may have fired and  
#                          created your first memory entry.  
# "Skill Portability"      : You will verify this in Lab 5 when you first  
#                          create a skill and then copy it to another folder.
```



Study Aid — External AI Tutor: Copy and paste the prompt below into any AI chat you have access to (free ChatGPT, free Claude, free Kimi, etc.) to get an interactive deep-dive on this chapter's concepts. This is optional and does **not** require Hermes to be configured with any specific model.

Master Prompt for This Chapter:

```
You are continuing my Hermes Agent deep-dive education from Chapter 2.  
Please explain the three invariants declared in the run_agent.py  
docstring: Model Agnosticism, Closed Learning Loop, and Skill  
Portability. For each invariant, give one concrete example from Labs  
1-2 that I can verify on my machine. Focus on Ollama Cloud  
and model switching between Kimi-K2.6 and GLM-5.1.  
Conclude with exactly 5 questions to test my understanding.
```

Chapter Conclusion — What You Have Learned

1. The docstring is not decorative — it is a contract. Every design choice in `run_agent.py` must satisfy one of the three invariants.
2. Model agnosticism is enforced by hiding all provider details behind `ModelInterface`. This is why `hermes config set model` works instantly without restart — whether switching between Ollama Cloud models or migrating from xAI to Ollama.
3. The learning loop is "closed" because it does not require human supervision. Reflection triggers fire automatically after tool execution.
4. Skill portability means your agent's learned behaviors are YOUR data, not locked into the codebase.
5. The Curator is built ON TOP of these invariants, not separate from them.

5 Questions to Check Your Understanding:

1. If `run_agent.py` claims model agnosticism, why does it still need imports like `os` that read API keys? Does this violate the invariant? Explain.
 2. Describe the difference between `_maybe_reflect()` and `_maybe_compress_session()`. Which one fires during a session and which one fires after?
 3. In Lab 2 you switched between Kimi-K2.6 and GLM-5.1 without restarting Hermes. Which Python design pattern (hint: starts with "Adapter") makes this possible?
 4. Why would skill portability matter for your machine user who might upgrade hardware or migrate to a newer Mac in the future?
 5. The docstring says the Curator is "just a background cron that calls the same hooks." What command did you run in Lab 1 that showed the Curator schedule, and what was the default interval?
-

Lab 3: Built-in Tools + Core Python Imports

Level: Beginner Estimated Time: ~60 minutes Prerequisites: Lab 2 complete (Ollama Cloud Pro is active; Kimi-K2.6 and GLM-5.1 are configured and switchable).

Lab Objectives

By the end of this lab you will be able to:

- List and manage all 26 built-in toolsets with `/tools` and `hermes tools`.
- Enable/disable tool groups safely.
- Trigger and approve tool calls (web search, code execution, file operations, calculator, etc.).
- Understand tool safety (approvals, sandboxing, and dry-run mode).
- Complete a real research task end-to-end and let Hermes auto-save the result as a skill.

Why This Lab Matters

Tools are the "hands" of Hermes. Unlike simple chatbots, Hermes can actually act on the world (search the web, run code, read/write files, browse pages, etc.). This lab is where the closed learning loop really begins — successful tool sequences are automatically turned into reusable skills. On your machine this is perfect: tools run locally or via API with zero extra install.

Step-by-Step Instructions

Step 1: Launch Hermes and Check Tool Status (5 minutes)

Open Terminal and start the TUI (recommended for seeing tool calls clearly):

```
hermes --tui
```

Inside the chat, type:

```
/tools
```

You will see a clean list grouped by category (Search, Code, Files, Browser, Vision, etc.) with status (enabled / disabled).

From Terminal (without entering chat) you can also run:

```
hermes tools list
```

Step 2: Enable/Disable Tool Groups (5 minutes)

Hermes groups tools for safety. Enable the core ones we will use today:

Inside the chat:

```
/tools enable search  
/tools enable code  
/tools enable files  
/tools enable calculator
```

(Or from Terminal: `hermes tools enable search code files calculator`)

Verify again with `/tools` — they should now show as enabled.

Safety tip: You can always disable everything instantly with `/tools disable all` if you ever feel uncomfortable.

Step 3: Test Core Tools One by One (20 minutes)

Stay in the same session. Test each tool with these exact prompts (Hermes will show a tool-call preview and ask for approval the first time — type `y` and Enter to approve):

1. Web Search Tool Type:

```
Use web_search to find the latest news about macOS support for your machine models. Then summarize the t
```

2. Calculator / Math Tool Type:

```
Use the calculator tool to compute: how many days old is my machine if it was released in March 2015? To
```

3. Code Execution Tool (safe sandbox) Type:

```
Use code_execution to run this Python snippet safely and show the output:
import psutil
print("Free RAM:", psutil.virtual_memory().available / (1024**3), "GB")
```

4. File Operations Tool Type:

```
Use file_write to create a new file in my home folder called "hermes-lab3-report.md" with this content:
# Lab 3 Tool Test Report
Date: [today]
Tools Tested: search, calculator, code_execution, file_write
MacBook: 2015 Pro, 16 GB RAM
Notes: All tools approved and executed successfully.
```

5. Web Extract Tool (deep page reading) Type:

```
Use web_extract to fetch and summarize the main content from https://docs.nousresearch.com/hermes-agent/
```

Observe:

- **Approval flow:** First tool call asks for confirmation — watch for `[approve? y/n]`.
- **Tool output:** Raw results appear inline, then the model synthesizes a response.
- **Safety:** Code execution runs in a restricted sandbox (no network, no `rm -rf /`).

Step 4: Research Task with Auto-Skill Creation (15 minutes)

Now trigger the full magic. Type this exact prompt:

```
Research the current state of large language model compatibility with macOS Ventura on Intel Macs. Specifically:  
1. Search for 2-3 recent articles or forum posts about running modern LLMs (Kimi-K2.6, GLM-5.1, Claude,  
2. Use code_execution to calculate how many 128k-context tokens fit in 16 GB RAM (assume 4 bytes per token)  
3. Summarize the findings in a short report and save it to ~/macbook-2015-llm-compatibility.md using file_write
```

Approve each tool call as it appears. Watch the agent chain multiple tools together.

At the end, verify the file exists:

```
cat ~/macbook-2015-llm-compatibility.md
```

Look for a well-structured report with:

- Date and model used
 - Search results summarized
 - RAM calculation
 - Personal recommendation
-

Step 5: Check for Auto-Generated Skill (5 minutes)

After the research task, type:

```
/skills
```

You may see a newly auto-extracted skill like `research_and_report` or `macbook_compatibility_check`. This is the closed learning loop in action.

If no skill appeared yet, it may take one more session. The skill extraction trigger requires a minimum confidence threshold.

Expected Outcome

- You have manually triggered and approved multiple tool calls.
- A real Markdown report file exists in `~/macbook-2015-llm-compatibility.md`.
- At least one new reusable skill has been automatically extracted (or is queued for extraction on next session).
- You understand the approval flow, sandboxing, and tool grouping.

Lab 3 Use Case Completed

You performed the exact use case from the outline: "Research latest machine compatibility with modern LLMs and save report." The agent did the heavy lifting while you stayed in control.

Training Idea / Self-Improvement Focus

This lab triggers first autonomous skill creation from successful tool sequences:

- The agent observed a repeated pattern: `search → calculate → write file → summarize`.
- It may have auto-created a skill called `research_and_report` or similar.
- In future labs, when you ask for research, the agent may automatically reuse this skill without prompting.
- The Curator (7-day cycle) will later review this skill for quality and may refine or consolidate it.

Challenge (Recommended)

1. Resume the session and ask: `Reuse the new skill you created to research compatibility of your machine with the latest version of Hermes Agent itself.` → Watch it execute with almost no prompting.
2. Try disabling the `search` tool with `/tools disable search` and attempt the same prompt. → Observe how the agent behaves when a critical tool is missing.

When you are done, say "Lab 3 complete" and move to the Chapter.

You are now past the basics — Hermes is starting to feel alive! Great work.

Chapter 3: Imports in run_agent.py — Part 1 (Core Python and Logging)

Top Ideal Study Objectives:

- See how basic Python modules are brought in.
- Understand why asyncio, logging, and os are imported before any Hermes-specific code.
- Connect these imports to the safety and performance properties you experienced in Lab 3.

Description: This chapter examines the foundational import block of `run_agent.py` — the standard library modules that support the entire loop. Every tool call in Lab 3 (web search, code execution, file write) ultimately relied on these imports. Understanding them lets you diagnose basic failures (e.g., "why is my log empty?", "why is my API key not found?").

Actions List (Topics):

1. Open `run_agent.py` and locate the imports section.
2. For each standard library import, read the inline comment (if any) and add your own note explaining the Lab 3 connection.
3. Verify that no third-party imports appear before the standard library block.

Full Source Code Snippets with Line-by-Line Explanations:

```

# ---- Lines 16-30: Core Python standard library imports ----

import asyncio          # Line ~16
import argparse          # Line ~17
import logging           # Line ~18
import os                # Line ~19
import sys               # Line ~20
import json              # Line ~21
import math              # Line ~22
import re                # Line ~23
import time              # Line ~24
import typing            # Line ~25

# Line-by-line expanded:
# asyncio : Enables non-blocking concurrency. In Lab 3, when you triggered
#           web_search and code_execution in the SAME prompt, asyncio let
#           both calls proceed in parallel without freezing your Terminal.
#           On your machine, this matters because the CPU is older -
#           blocking I/O would make the UI feel sluggish.
#
# argparse : Parses every CLI flag. When you typed `hermes --tui`, argparse
#           recognized `--tui` and dispatched to the TUI launcher. Without
#           argparse, every new CLI subcommand would require manual argv[]
#           slicing.
#
# logging : Structured event recording. The tool approval flow in Lab 3
#           ("approve? y/n") is logged at INFO level. If you ever wonder
#           "why did the agent skip approval?", the log file contains
#           the exact decision path.
#
# os : Environment and filesystem access. Lab 3's code_execution tool
#     uses os.path.exists() to verify safe paths before writing.
#     Also reads API keys from os.environ (populated from
#     ~/.hermes/.env by the launcher script).
#
# sys : Python runtime control. When you press Ctrl+D to exit,
#     sys.exit(0) is called. Also extends sys.path so custom
#     skills and plugins are importable.
#
# json : Serialization. Every model API response comes as JSON.
#       The agent parses model outputs with json.loads() before
#       routing to tools or displaying to you.
#
# math : Mathematical utilities. The calculator tool in Lab 3 used
#       math functions for the RAM/token calculation. Also used
#       in HRR vector computations for memory storage.
#
# re : Regular expressions. Used to sanitize tool output before
#     sending to the model (e.g., stripping ANSI color codes
#     from terminal output).
#
# time : Timestamps. Every session, memory entry, and log line
#       is time-stamped. time.monotonic() is used to measure
#       tool execution duration for the user model.
#
# typing : Type hints. Makes the codebase self-documenting.
#         Functions declare types like `def search_web(query: str) -> list[dict]`
#         so type checkers catch errors before runtime.

```



Study Aid — External AI Tutor: Copy and paste the prompt below into any AI chat you have access to (free ChatGPT, free Claude, free Kimi, etc.) to get an interactive deep-dive on this chapter's concepts. This is optional and does **not** require Hermes to be configured with any specific model.

Master Prompt for This Chapter:

```
You are continuing my Hermes Agent deep-dive education from Chapter 3.
Explain the first 10 standard library imports in run_agent.py:
asyncio, argparse, logging, os, sys, json, math, re, time, typing.
For each, give:
- One specific Lab 3 action that relied on it
- What would break if that import were removed
Use your machine as the hardware context.
Conclude with exactly 5 questions to test my understanding.
```

Chapter Conclusion — What You Have Learned

1. Standard library imports come FIRST because they have zero external dependencies and form the backbone of reliability.
2. `asyncio` is the reason your machine can run Hermes without UI freezing during long web searches.
3. `logging` is your forensic tool — when something goes wrong, the log is the first place to check.
4. `os` and `sys` bridge between Python code and your operating system. Every file path and environment variable flows through them.
5. `typing` is not optional decoration — it is the documentation that the IDE reads and the type checker enforces.

5 Questions to Check Your Understanding:

1. In Lab 3 you triggered `web_search` and `file_write` in the same prompt. `asyncio` made this possible. What would happen on your machine if `asyncio` were replaced with synchronous blocking code?
2. `logging` captured your tool approvals. Where is the default log file stored, and what command can you run to stream logs in real time?
3. `os.environ` reads API keys. What is the exact path to the file where Hermes stores your `OLLAMA_API_KEY`?
4. `sys.path` is modified at startup to include plugin directories. What is the absolute path to the `gatekeeper-reminder` plugin on your current profile?
5. `typing` enables IDE autocomplete. If a function says `def search_web(query: str) -> list[dict]`, what does `-> list[dict]` promise about the return value?

Lab 4: Memory & Sessions + Hermes-Specific Imports

Level: Beginner Estimated Time: ~45 minutes Prerequisites: Lab 3 complete (tools configured, research task completed).

Lab Objectives

By the end of this lab you will be able to:

- Explore session history and cross-session recall.
- Use `/insights` to see what Hermes has learned about you.
- Trigger a manual memory nudge and observe the result.
- Verify that memory persists across restarts.
- Understand the difference between session memory (SQLite) and long-term memory (HRR vectors).

Why This Lab Matters

Memory is what turns a chatbot into a companion. Without it, every session starts from zero. With it, Hermes remembers your style, your hardware, your goals, and your preferences across days and weeks. On your machine, this is stored entirely locally in SQLite — no cloud dependency, no latency, complete privacy. This lab shows you how to inspect and curate that memory so the agent stays accurate and useful.

Step-by-Step Instructions

Step 1: Explore Session History (5 minutes)

Launch Hermes and resume your last session:

```
hermes --continue
```

Inside the chat, type:

```
What did we discuss in our previous sessions? Summarize our conversation history.
```

The agent should reference Labs 1-3: your machine model, your API keys setup, the tool tests, and the research report. If it does not, type:

```
/memory search "MacBook Pro 2015"
```

This triggers a full-text search across all stored sessions.

Step 2: Use /insights (5 minutes)

Type:

```
/insights
```

You will see a summary like:

```
User Profile Summary:  
- Hardware: Your machine (16 GB RAM, macOS)  
- Goals: Research + coding with Kimi-K2.6 and GLM-5.1 via Ollama Cloud Pro  
- Style: Prefers step-by-step instructions, copy-paste prompts  
- Model Preference: GLM-5.1 for long-horizon reasoning, Kimi-K2.6 for coding/multimodal (inferred)
```

This is Honcho user modeling in action — the agent builds a structured profile from your interactions.

Step 3: Manual Memory Nudge (10 minutes)

You can explicitly tell Hermes to remember something important.

Type:

```
Remember this: I am a researcher working on AI benchmarking. My primary hardware is your machine with 16
```

Then verify it was stored:

```
/memory list
```

Look for the entry with high relevance (top of list). Note the timestamp.

Step 4: Cross-Session Recall Test (10 minutes)

Exit Hermes completely (`/quit`), wait 5 seconds, and restart:

```
hermes --tui
```

Start a NEW session (not `--continue`):

```
hermes
```

Type:

```
What do you remember about my hardware and research goals?
```

If memory is working, the agent should respond with your machine specs and research focus WITHOUT you prompting it. This proves the Holographic Memory (HRR vectors in SQLite) is persisting across sessions.

Step 5: Inspect Memory Files (5 minutes)

From Terminal (outside chat):

```
ls -la ~/.hermes/
```

You should see:

```
state.db          # SQLite: sessions, config, achievements
memory_store.db   # SQLite: HRR vectors (long-term memory)
skills/           # Directory: auto-extracted and custom skills
plugins/          # Directory: active plugins (gatekeeper-reminder, etc.)
.env              # Encrypted API keys
config.yaml       # Main configuration
SOUL.md           # Agent constitution
```

Check the size:

```
du -h ~/.hermes/memory_store.db
```

On your machine after 4 labs, this should be tiny (under 1 MB) but growing.

Expected Outcome

- You have explored session history and cross-session recall.
- `/insights` shows a structured user profile.
- A manual memory nudge was successfully stored and retrieved.
- Memory persists across restarts (verified with `--new` session).
- You understand the physical file layout of Hermes' memory system.

Lab 4 Use Case Completed

You asked Hermes to "recall what we discussed yesterday about my research project" and it did — accurately and with context. This is the foundation of long-term companionship between you and the agent.

Training Idea / Self-Improvement Focus

This lab teaches the agent to summarize and persist key facts automatically:

- The manual nudge you added will be reviewed by the Curator on its next 7-day cycle.
- If the Curator deems this memory high-quality (high relevance, user-confirmed), it will consolidate it into your user model.
- Future sessions will automatically inject this context into the system prompt without you asking.
- You are training the agent to be proactive, not just reactive.

Challenge (Recommended)

1. Add a SECOND memory nudge: "I dislike verbose responses. Keep answers under 200 words unless I ask for detail."
2. Restart Hermes (just run `hermes` for a new session) and ask a complex question.
3. Verify that the response is concise AND includes your hardware context.
4. Check `head -10 ~/.hermes/MEMORY.md` from Terminal.

You are building a true long-term companion now — this is where Hermes starts to feel magical. Excellent work!

Chapter 4: Imports in run_agent.py — Part 2 (Hermes-Specific Modules)

Top Ideal Study Objectives:

- Understand connections to `hermes_state.py`, skills, memory, etc.
- See how the file wires the three core self-improvement systems.
- Map each Hermes import to the Lab 4 experience.

Description: This chapter covers the second import block — the Hermes-specific modules that make the agent more than a standard Python script. In Lab 4 you used memory, skills, and state. In this chapter you discover where those modules are imported and what contract they satisfy.

Actions List (Topics):

1. Locate the Hermes-specific import block in `run_agent.py`.
2. Read the docstring of each imported module (if available).
3. Draw a connection diagram: import → Lab 4 feature → physical file.

Full Source Code Snippets with Line-by-Line Explanations:

```

# ---- Lines 26-45: Hermes-specific imports ----

from hermes.models import ModelLoader          # Line ~26
from hermes.state import HermesState           # Line ~27
from hermes.memory import HolographicMemory    # Line ~28
from hermes.skills import SkillLoader          # Line ~29
from hermes.tools import ToolRegistry          # Line ~30
from hermes.curator import Curator             # Line ~31
from hermes.subagents import SubagentManager   # Line ~32
from hermes.gateways import GatewayManager     # Line ~33
from hermes.plugins import PluginLoader        # Line ~34

# Line-by-line:
# ModelLoader      : Encapsulates all provider connections (Ollama Cloud, Anthropic, etc.).
#                   In Lab 2 you switched models with `/model` - ModelLoader
#                   swapped the adapter instance without restarting the agent.
#                   This is the Adapter pattern in action.
#
# HermesState      : SQLite persistence for sessions, achievements, and config.
#                   Every `/save`, `/title`, and `/usage` command writes
#                   through HermesState. The file is ~/.hermes/state.db.
#
# HolographicMemory : Long-term memory using Holographic Reduced
#                   Representations (HRR vectors). Each memory is stored
#                   as a phase-encoded vector in SQLite. In Lab 4, when
#                   you ran `/memory search`, HolographicMemory computed
#                   similarity between your query vector and stored vectors.
#
# SkillLoader      : Procedural memory engine. Reads ~/.hermes/skills/ and
#                   dynamically loads Python functions and Markdown prompts.
#                   In Lab 3, the auto-extracted "research_and_report" skill
#                   was saved by SkillLoader and loaded on next session.
#
# ToolRegistry     : Maintains the catalog of available toolsets. When you
#                   typed `/tools enable search`, ToolRegistry flipped the
#                   enabled bit for the search toolset. It also validates
#                   tool schemas before execution to prevent malformed calls.
#
# Curator          : Background maintenance agent. Runs on a cron schedule
#                   (default: every 7 days) to grade, prune, and consolidate
#                   skills and memory. You checked its status in Lab 1 with
#                   `hermes curator status`. It is NOT a separate process -
#                   it is just a scheduled task that calls the same reflection
#                   hooks as live sessions.
#
# SubagentManager  : Spawns isolated subagents via delegate_task. In later
#                   labs (Lab 15), you will parallelize tasks across subagents.
#                   Each subagent gets a fresh context, toolset, and model.
#
# GatewayManager   : Bridges Hermes to external messaging platforms.
#                   In Lab 6 you will connect Telegram/Discord. GatewayManager
#                   routes incoming messages to the agent and outgoing replies
#                   back to the platform.
#
# PluginLoader     : Loads plugins from ~/.hermes/plugins/ and ~/.hermes/plugins/
#                   inside the active profile. In Lab 1, the gatekeeper-reminder
#                   plugin was loaded by PluginLoader and its hook appended
#                   Rule (7) reminders to every delegate_task result.

```



Study Aid — External AI Tutor: Copy and paste the prompt below into any AI chat you have access to (free ChatGPT, free Claude, free Kimi, etc.) to get an interactive deep-dive on this chapter's concepts. This is optional and does **not** require Hermes to be configured with any specific model.

Master Prompt for This Chapter:

```
You are continuing my Hermes Agent deep-dive education from Chapter 4.
Explain the 9 Hermes-specific imports in run_agent.py:
ModelLoader, HermesState, HolographicMemory, SkillLoader, ToolRegistry,
Curator, SubagentManager, GatewayManager, PluginLoader.
For each, give:
- The exact Lab 4 action that used it
- The physical file path where its data lives (if applicable)
- What would break if this import failed at startup
Use your machine as the hardware context.
Conclude with exactly 5 questions to test my understanding.
```

Chapter Conclusion — What You Have Learned

1. Hermes is modular. Each import is a standalone subsystem with a single responsibility.
2. `HermesState` is the "hard disk" — everything that must survive crashes goes here.
3. `HolographicMemory` is the "long-term brain" — vector similarity search enables semantic recall even across sessions.
4. `SkillLoader` is the "muscle memory" — learned behaviors that execute without re-prompting.
5. `Curator` is the "janitor + tutor" — it cleans up stale data AND reinforces high-quality patterns.

5 Questions to Check Your Understanding:

1. In Lab 4 you used `/memory search`. Which TWO imports handle the query, and what is the difference between them?
2. `SkillLoader` saved your "research_and_report" skill in Lab 3. What is the exact directory path where this skill file lives?
3. If `Curator` is disabled in `config.yaml`, what Lab 4 feature still works and what feature stops working?
4. `GatewayManager` is not used until Lab 6. Why does `run_agent.py` still import it at startup?
5. `HolographicMemory` uses HRR vectors. What does "HRR" stand for, and why is it suitable for your machine (hint: think about computation cost vs accuracy)?

Lab 5: First Reusable Skill + IterationBudget Class

Level: Beginner Estimated Time: ~60 minutes Prerequisites: Lab 4 complete (memory working, `/insights` shows user profile).

Lab Objectives

By the end of this lab you will be able to:

- Trigger skill auto-creation manually.
- View all skills with `/skills` and `hermes skills list`.
- Manually save and edit a skill by creating a skill directory under `~/.hermes/skills/` with a `SKILL.md`.
- Reuse a skill in a new session and observe reduced prompting.
- Understand the skill file format (Markdown + YAML frontmatter + optional Python).

Why This Lab Matters

Skills are procedural memory — the agent's ability to remember HOW to do something, not just WHAT was said. This is the first full cycle of the learning loop: task → skill extraction → reuse. On your machine, skills are stored as plain text files in `~/.hermes/skills/` — zero database dependency, fully inspectable, and endlessly portable.

Step-by-Step Instructions

Step 1: Trigger Auto-Creation (10 minutes)

Resume your session:

```
hermes --continue
```

Trigger a clear pattern the agent can learn from. Type:

```
Every morning I want you to:
1. Search for "latest AI news"
2. Pick the top 3 stories from reputable sources
3. Summarize each in 2 sentences
4. Save the summary to ~/daily-ai-news.md with the date in the filename

For now, do it once as a test.
```

Approve the tool calls and complete the task. Then ask:

Did you extract a skill from that? What is it called?

Check:

/skills

Look for a new entry like `daily_news_digest` or `morning_brief`.

Step 2: Manually Create a Skill (15 minutes)

If auto-extraction did not trigger, create one manually.

From Terminal:

```
mkdir -p ~/.hermes/skills/macbook_thermal_check && nano ~/.hermes/skills/macbook_thermal_check/SKILL
```

This opens your default editor. Paste this exact template:

```
name: macbook_thermal_check
description: Checks MacBook Pro 2015 thermal state using Python
trigger: "thermal", "temperature", "hot", "fan", "macbook"
version: 1.0

# MacBook Thermal Check

Whenever the user asks about their machine temperature or fan noise:

1. Run `code_execution` with this Python script:
```python
import psutil
import subprocess
temp = subprocess.getoutput("osx-cpu-temp") if available else "unknown"
print(f"CPU Temp: {temp}")
print(f"Fan Speed: not available via psutil")
print(f"Load: {psutil.cpu_percent()}%")
```

2. Summarize whether the machine is running within safe limits.

3. If load is >80% for >5 minutes, suggest closing heavy apps.

Save and close.

```
Verify:
```bash
hermes skills list | grep macbook
```

Step 3: Reuse the Skill (10 minutes)

Start a NEW session:

```
hermes
```

Type:

```
My machine feels hot right now. Can you check?
```

If the skill trigger worked, the agent should automatically run the thermal check script WITHOUT you providing the full instructions. This is skill reuse in action.

If it did not trigger automatically, type:

```
Run the macbook_thermal_check skill.
```

Step 4: Edit and Version the Skill (10 minutes)

Improve the skill. From Terminal:

```
nano ~/.hermes/skills/macbook_thermal_check/SKILL.md
```

Add a new line:

```
4. Compare current load to historical average if data exists in state.db.
```

Save. The version is now 1.1.

Test again with a new prompt: `Is my laptop running too hot?`

Step 5: Understand Skill Persistence (5 minutes)

Inspect the skill file:

```
cat ~/.hermes/skills/macbook_thermal_check.md
```

Notice:

- YAML frontmatter (name, description, trigger, version)
- Markdown body (human-readable instructions)
- Optional Python block (executable code)

This is plain text. You can:

- Copy it to another machine and it works.
- Commit it to Git and version it.
- Share it on agentskills.io Skills Hub.

Expected Outcome

- You have triggered skill auto-creation OR manually created a skill.
- The skill is listed in `/skills`.
- Reusing the skill in a new session reduces prompting (agent knows what to do).
- You understand the skill file format and where it lives.
- You have edited a skill and observed version change.

Lab 5 Use Case Completed

You created a skill that automatically checks your machine thermal state. In the future, any mention of "hot", "thermal", or "temperature" will trigger this skill without you needing to explain the Python script again.

Training Idea / Self-Improvement Focus

This is the first FULL cycle of the learning loop (task → skill extraction → reuse):

- The agent now has procedural memory for hardware health monitoring.
- The Curator will review this skill in 7 days: if it is used frequently, it gets promoted. If unused, it gets archived.
- Future labs will build on this: you will create skills for research, coding, and communication.
- Each skill makes the agent MORE autonomous and LESS dependent on your prompting.

Challenge (Recommended)

1. Export your skill:

```
cp ~/.hermes/skills/macbook_thermal_check.md ~/Desktop/my_first_skill.md
```

2. Start a completely fresh Hermes session (delete state.db if you are brave, or just use `--new`).
3. Copy the skill back to `~/.hermes/skills/` and verify it loads with `hermes skills list`.
4. Test it: `Is my laptop running too hot?`
5. It should work immediately — proving skill portability.

You are building a true long-term companion. Fantastic work!

Chapter 5: The IterationBudget Class — Limiting Steps to Prevent Infinite Loops

Top Ideal Study Objectives:

- Master how the agent controls its own thinking steps.
- Understand why infinite loops are impossible in Hermes (by design).
- Connect the budget system to Lab 5's multi-step skill execution.

Description: The `IterationBudget` class is the safety guardrail of the agent loop. When you triggered the "macbook_thermal_check" skill in Lab 5, it could have involved 5 tool calls in a row. `IterationBudget` ensures the agent stops after a configurable limit instead of spiraling forever.

Actions List (Topics):

1. Locate the `IterationBudget` class in the source tree.
2. Read the class docstring.
3. Identify the default step limit.
4. Find where the budget is decremented and checked.

Full Source Code Snippets with Line-by-Line Explanations:

```
# iteration_budget.py (approximate path: src/hermes/agent/iteration_budget.py)
```

```
class IterationBudget:
```

```
    """
```

```
    Enforces an upper bound on tool-execution iterations within  
    a single agent turn. Prevents infinite loops from:
```

- Recursive tool calls (tool A calls tool B calls tool A)
- Model hallucination (model insists on calling a tool endlessly)
- Skill cascade (one skill triggers another in an uncontrolled chain)

```
    Attributes:
```

```
        max_steps: Maximum number of tool calls allowed (default: 20).
```

```
        current_step: Counter incremented after each tool execution.
```

```
        exhausted: True if current_step >= max_steps.
```

```
    """
```

```
    def __init__(self, max_steps: int = 20):          # Line ~15
```

```
        """
```

```
        max_steps: 20 is safe for most tasks. Research or coding  
        tasks may override this via the `max_iterations` parameter  
        in the agent config or via `hermes config set agent.max_iterations N`.
```

```
        """
```

```
        self.max_steps = max_steps                  # Line ~22
```

```
        self.current_step = 0                        # Line ~23
```

```
        self.exhausted = False                      # Line ~24
```

```
    def use_step(self) -> bool:                      # Line ~26
```

```
        """
```

```
        Consumes one iteration. Returns True if budget remains,  
        False if exhausted. Called after EVERY tool execution.
```

```
        """
```

```
        self.current_step += 1                      # Line ~31
```

```
        if self.current_step >= self.max_steps:     # Line ~32
```

```
            self.exhausted = True                  # Line ~33
```

```
            return False                           # Line ~34
```

```
        return True                                # Line ~35
```

```
    def remaining(self) -> int:                      # Line ~37
```

```
        """How many steps left before exhaustion."""
```

```
        return self.max_steps - self.current_step  # Line ~39
```

```
# Line-by-line:
```

```
# Line 15      : The default max_steps is 20. This means a single agent  
#              turn can invoke at most 20 tools. For your your machine,  
#              this is a safety AND performance feature: long tool chains  
#              could strain the CPU and memory.  
#
```

```
# Line 22-24   : Simple state. No async, no locks -- just integers.  
#              This makes the class extremely fast to check and  
#              impossible to deadlock.  
#
```

```
# Line 26-35   : The use_step() method is called after EVERY tool call.  
#              If you had a skill that ran 10 tools in sequence,  
#              use_step() would be called 10 times. On the 21st call  
#              it would return False and the agent would stop with  
#              a graceful "budget exhausted" message.  
#
```

```
# Line 37-39   : remaining() is exposed to the user interface. When you  
#              type `/usage`, the remaining budget may be shown as  
#              part of the resource summary.
```



Study Aid — External AI Tutor: Copy and paste the prompt below into any AI chat you have access to (free ChatGPT, free Claude, free Kimi, etc.) to get an interactive deep-dive on this chapter's concepts. This is optional and does **not** require Hermes to be configured with any specific model.

Master Prompt for This Chapter:

```
You are continuing my Hermes Agent deep-dive education from Chapter 5.
Explain the IterationBudget class in detail:
- Why does it exist (what failure modes does it prevent)?
- What is the default max_steps and why 20 (not 10, not 100)?
- How would your machine user be affected if max_steps were
  set to 1000 on a code_execution task that enters an infinite loop?
- Where is use_step() called in the agent loop?
- How can the user override the default from CLI or config?
Conclude with exactly 5 questions to test my understanding.
```

Chapter Conclusion — What You Have Learned

1. `IterationBudget` is a hard limit, not a suggestion. It prevents both bugs and resource exhaustion.
2. The default of 20 is calibrated for real-world tasks. Most conversations use 1-5 tool calls.
3. On your machine, a high `max_steps` with an infinite loop would eventually swap memory and freeze the system. The budget prevents this.
4. The class is intentionally simple (plain integers) to avoid introducing its own bugs.
5. Users can override the budget with `hermes config set agent.max_iterations N` or in `config.yaml` under `agent.max_iterations`.

5 Questions to Check Your Understanding:

1. In Lab 5, your `macbook_thermal_check` skill might trigger 4 tool calls. If the user had a broken skill that called itself recursively, how would `IterationBudget` stop the disaster?
2. Why is `IterationBudget` implemented with plain integers instead of async state or decorators?
3. If you are running Hermes on your machine and a coding task reaches the 20-step limit, what config command can you use to increase it to 50?
4. What does `self.exhausted` do? Can the agent recover from exhaustion within the same session?
5. The Curator might review skills that hit the budget limit too often. What would the Curator likely do: promote, demote, or prune such a skill? Why?

Lab 6: Telegram/Discord Gateway + AI Agent Class Structure

Level: Intermediate Estimated Time: ~75 minutes Prerequisites: Labs 1–5 complete (Hermes installed, models configured, tools enabled, skills created).

Lab Objectives

By the end of this lab you will be able to:

- Set up `hermes gateway` for Telegram and Discord.
- Create bot accounts on both platforms.
- Configure API tokens in `~/.hermes/.env`.
- Test remote control: send commands from your phone and watch the agent respond.
- Understand how gateways bridge external messaging to the local agent loop.

Why This Lab Matters

Until now, Hermes only responded inside your Terminal. With gateways, you can interact from your phone, tablet, or any device — while the agent runs on your machine in the background. This is "always-on" access. On your machine, the footprint is tiny: gateways just add a listener thread, not a full second process. The agent learns your communication style across platforms and syncs the same memory and skills.

Step-by-Step Instructions

Step 1: Create Bot Accounts (15 minutes)

Telegram:

1. Open Telegram and message `@BotFather`.
2. Type `/newbot` and follow instructions.
3. Choose a name (e.g., `ThanitHermesBot`) and a username ending in `bot`.
4. Copy the HTTP API token (looks like `123456789:ABCdefGHIjklMNOpqrsTUVwxyz`).

Discord:

1. Go to <https://discord.com/developers/applications>.
2. Click "New Application", name it `Hermes-Agent`.
3. Go to "Bot" → "Add Bot".
4. Under "Token", click "Copy". This is your `DISCORD_BOT_TOKEN`.
5. Under "OAuth2" → "URL Generator", enable `bot` scope and `Send Messages`, `Read Message History` permissions.

6. Copy the generated URL and open it in your browser to invite the bot to your server.

Step 2: Configure Hermes Gateway (10 minutes)

Edit your environment file:

```
nano ~/.hermes/.env
```

Add:

```
# Telegram Gateway
TELEGRAM_BOT_TOKEN=your_telegram_token_here

# Discord Gateway
DISCORD_BOT_TOKEN=your_discord_token_here
```

Save and close.

Verify:

```
cat ~/.hermes/.env | grep -E 'TELEGRAM|DISCORD'
```

Step 3: Enable Gateways (10 minutes)

In your Hermes session, type:

```
/hermes gateway telegram start  
/hermes gateway discord start
```

Or from Terminal:

```
hermes gateway start telegram  
hermes gateway start discord
```

Check status:

```
hermes gateway status
```

You should see both as `running`.

Step 4: Test Remote Control (15 minutes)

Telegram test:

1. Open Telegram on your phone.
2. Message your bot: `Hello from Telegram! What do you remember about me?`
3. The agent should respond with your machine context from Labs 1–5.

Discord test:

1. Open Discord on your phone or web.
2. Go to the channel where you invited the bot.
3. Type: `!hermes What tools do I have enabled?`
4. The bot should reply with the tool list.

Test a skill:

- Send: `Check my machine thermal status.`
- The bot should reuse the `macbook_thermal_check` skill from Lab 5.

Step 5: Gateway Configuration (10 minutes)

From Terminal, inspect gateway config:

```
hermes config show
```

You should see:

```
gateways:
  telegram:
    enabled: true
    webhook: false
    polling_interval: 2
  discord:
    enabled: true
    prefix: "!hermes"
    allowed_channels: ["general"]
```

Change the Discord prefix:

```
hermes config set discord.prefix "/h"
```

Test: In Discord, type `/h What model am I using?`

Step 6: Security and Approvals (5 minutes)

Gateways bypass the local Terminal approval flow. This means:

- Tool calls from Telegram/Discord may auto-approve OR require confirmation.
- Check your `config.yaml`:

```
hermes config show | grep -A 5 gateway
```

Set `gateway.auto_approve: false` for safety.

Restart gateways after config change:

```
hermes gateway restart all
```

Expected Outcome

- Telegram and Discord bot accounts created.
- Hermes gateways running and connected.
- Remote commands from phone produce accurate responses.
- Skills work cross-platform (same memory, same skills).
- Gateway security configured (manual approval enabled).

Lab 6 Use Case Completed

You receive daily research digests on Telegram while the agent runs on your machine. Each message pulls from the same memory and skills as your Terminal sessions.

Training Idea / Self-Improvement Focus

Agent learns your communication style across platforms:

- Telegram messages are shorter and more informal. Hermes may start replying more concisely.
- Discord messages may use server jargon. The agent learns community-specific language.
- GatewayManager routes all messages through the SAME memory pipeline. A memory nudge from Telegram is instantly available in Terminal.
- The Curator observes cross-platform patterns and may create specialized "gateway-aware" skills.

Challenge (Recommended)

1. Send a complex research request from Telegram (3-4 sentence prompt).
2. Monitor your machine CPU from a Terminal window:

```
top -o cpu | grep hermes
```

3. Notice that CPU usage stays low even during long web searches — `asyncio` spreads the load efficiently on your dual-core machine.
4. After the task completes, check `cat ~/.hermes/MEMORY.md` from Terminal. The memory entry should show source: `telegram`.

You are now running an always-on AI on your machine. Impressive progress!

Chapter 6: AI Agent Class — High-Level Structure (The Brain)

Top Ideal Study Objectives:

- See the main class skeleton.
- Understand how it orchestrates the entire closed loop.
- Map class methods to Lab 6 gateway events.

Description: The `AI Agent` class is the brain. It holds references to every subsystem you learned in Chapter 4: `HermesState`, `HolographicMemory`, `SkillLoader`, `ToolRegistry`, `Curator`, `GatewayManager`. In Lab 6, when you sent a Telegram message, `GatewayManager` received it and called `AI Agent.process_message()`.

Actions List (Topics):

1. Locate `class AI Agent` in the source tree.
2. Read `__init__` and identify every instance attribute.
3. Find the main message processing method.
4. Trace how a Telegram message flows from gateway to model to response.

Full Source Code Snippets with Line-by-Line Explanations:

```

# src/hermes/agent/ai_agent.py (approximate path)

class AIAgent:
    """
    Orchestrator for the entire Hermes closed loop.

    Holds references to ALL subsystems. Do NOT create multiple
    instances in the same process -- subsystems assume singleton
    access for state consistency.
    """

    def __init__(self, config: dict, state: HermesState,      # Line ~20
                  memory: HolographicMemory, skills: SkillLoader,
                  tools: ToolRegistry, curator: Curator,
                  gateway: GatewayManager, plugins: PluginLoader):
        """
        Each parameter is injected by run_agent.py during startup.
        This is Dependency Injection (DI): AIAgent does not create
        its own subsystems; it receives them. This makes testing
        possible (mock subsystems) and prevents circular imports.

        config      : Dict from config.yaml + CLI flags + env vars.
        state        : SQLite persistence (sessions, achievements).
        memory       : HRR vector storage (long-term recall).
        skills       : Procedural memory (loaded from ~/.hermes/skills/).
        tools        : Tool registry (enabled/disabled per session).
        curator      : Background maintenance (7-day skill review).
        gateway      : Telegram/Discord/Slack bridges.
        plugins      : Custom hook loaders.
        """
        self.config = config                # Line ~33
        self.state = state                  # Line ~34
        self.memory = memory                # Line ~35
        self.skills = skills                # Line ~36
        self.tools = tools                  # Line ~37
        self.curator = curator              # Line ~38
        self.gateway = gateway              # Line ~39
        self.plugins = plugins              # Line ~40

        self.budget = IterationBudget(      # Line ~42
            max_steps=config.get("max_iterations", 20) # Line ~43
        ) # Line ~44
        self.active_model = None            # Line ~45
        self.user_model = {}                # Line ~46

    def process_message(self, msg: str, source: str = "cli"): # Line ~48
        """
        Main entry point for ALL incoming messages, regardless of
        gateway. When you typed in Telegram, GatewayManager called
        this method with source="telegram".

        Steps:
        1. Inject context (memory + skills + user model).
        2. Build the prompt with current state.
        3. Send to active model.
        4. Parse model response (text or tool call).
        5. If tool call: route to ToolRegistry.
        6. If reflection triggered: call _maybe_reflect().
        7. Return response to gateway (if applicable).
        """
        context = self._build_context()      # Line ~61
        prompt = self._assemble_prompt(msg, context) # Line ~62
        response = self.active_model.generate(prompt) # Line ~63

        if self._is_tool_call(response):     # Line ~65

```

```

        result = self.tools.execute(response.tool_call) # Line ~66
        self.budget.use_step() # Line ~67
        self._maybe_reflect(response, result) # Line ~68
        return result.formatted # Line ~69
    else:
        self._maybe_compress_session() # Line ~71
        return response.text # Line ~72

# Line-by-line:
# Line 20 : __init__ takes NO default values except budget. Every
# subsystem must be provided. This prevents the class
# from being accidentally instantiated in a broken state.
#
# Line 33-40 : Assignments are straightforward. Notice that all
# attributes are public (no leading underscore). This is
# intentional: plugins and skills may need to access
# subsystems. If you write a custom plugin, you can
# read self.memory or self.tools.
#
# Line 42-44 : IterationBudget is instantiated HERE, not in run_agent.py.
# This means EACH AI Agent gets its own budget. If you
# spawn multiple agents (e.g., subagents in Lab 15),
# each has an independent budget.
#
# Line 45 : active_model is None until ModelLoader sets it.
# This is why `hermes setup` asks for a provider before
# the first chat.
#
# Line 46 : user_model is a DICT, not a database row. It is built
# incrementally from Honcho interactions and memory entries.
# In Lab 4, `/insights` displayed this dict.
#
# Line 48-72 : process_message() is the HEARTBEAT. Every message,
# whether from Terminal, Telegram, or Discord, goes
# through this method. The `source` parameter is used
# for logging ("message from telegram") and for gateway-
# specific formatting (e.g., Discord markdown).
#
# Line 67 : budget.use_step() is called AFTER tool execution.
# If the tool fails or is rejected, the step is still
# consumed. This prevents "approval denial abuse" where
# a model could repeatedly call a tool hoping for approval.

```

 **Study Aid — External AI Tutor:** Copy and paste the prompt below into any AI chat you have access to (free ChatGPT, free Claude, free Kimi, etc.) to get an interactive deep-dive on this chapter's concepts. This is optional and does **not** require Hermes to be configured with any specific model.

Master Prompt for This Chapter:

You are continuing my Hermes Agent deep-dive education from Chapter 6.
Explain the `AIAgent` class in detail:

- Why is it designed as a single orchestrator rather than a pipeline?
- What is Dependency Injection and why does it matter for testing?
- Trace a Telegram message from arrival to response, naming every method and subsystem called.
- If I were to write a custom plugin that accesses the memory store, which attribute would I use?
- What happens if `IterationBudget` hits its limit during a Telegram session? Does the user see the "budget exhausted" message?

Use your machine as the hardware context.
Conclude with exactly 5 questions to test my understanding.

Chapter Conclusion — What You Have Learned

1. `AIAgent` is the single point of truth. All messages converge here.
2. Dependency Injection makes the code testable and prevents hidden globals.
3. Every subsystem is exposed to plugins — this is how `gatekeeper-reminder` appends to tool results.
4. `process_message()` is the heartbeat. Understanding it means understanding 80% of Hermes' runtime behavior.
5. `IterationBudget` is per-agent, not global. Subagents get their own budgets.

5 Questions to Check Your Understanding:

1. In Lab 6, when you sent a Telegram message, which method in `AIAgent` received it first, and what was the value of the `source` parameter?
2. Why does `AIAgent.__init__` take `state` as a parameter instead of creating its own `HermesState()` ?
3. If `active_model` is `None` at Line 45, what prevents `process_message()` from crashing on Line 63?
4. `user_model` is a dict. Where does the data come from, and which Lab 4 command let you VIEW this dict?
5. If you write a custom plugin that increases `max_steps` from 20 to 50, is this change global or per-agent? Explain.

Lab 7: Cron Scheduling + The Conversation Loop

Level: Intermediate Estimated Time: ~60 minutes Prerequisites: Lab 6 complete (gateways running, agent accessible remotely).

Lab Objectives

By the end of this lab you will be able to:

- Create natural-language cron jobs using `hermes cron create`.
- View, pause, resume, and remove scheduled tasks.
- Understand how cron job skills are auto-extracted and persisted.
- Set up a daily automation that runs without manual prompting.
- Monitor scheduled tasks with `hermes cron list` and `hermes cron status`.

Why This Lab Matters

Manual prompting is powerful, but true autonomy means the agent acts on its own schedule. On your machine, cron jobs run in the background without opening Terminal windows. The agent can wake up at 8 AM, check your news, summarize it, and send it to Telegram — all while you sleep. This lab is where Hermes graduates from "assistant" to "autonomous agent."

Step-by-Step Instructions

Step 1: Inspect Current Cron Jobs (5 minutes)

From Terminal:

```
hermes cron list
```

You should see the Curator entry:

ID	Name	Schedule	Status
-----	-----	-----	-----
#0	curator	0 0 * * 0	active (weekly)

This is the default Curator job (every Sunday at midnight).

Step 2: Create Your First Cron Job (10 minutes)

Add a daily news digest:

```
hermes cron create "0 8 * * *" "daily morning briefing"
```

The schedule is standard cron syntax (minute hour day month weekday). `0 8 * * *` means 8:00 AM every day.

Alternatively, use natural language (a default schedule of 30m is applied):

```
hermes cron create "30m" "send me a morning briefing at 8 AM every day"
```

Verify:

```
hermes cron list
```

Step 3: Define the Job Action (10 minutes)

When prompted (or by editing), paste this action:

```
Every day at 8 AM:  
1. Search for "latest AI news" or "latest tech news for Mac users"  
2. Pick top 3 reputable stories  
3. Summarize each in 2-3 sentences  
4. Send results to my Telegram bot (use gateway send telegram)  
5. Save a copy to ~/daily-briefings/<date>.md
```

Save. The job is now queued.

Step 4: Test the Job Immediately (10 minutes)

Trigger a manual run:

```
hermes cron run daily morning briefing
```

Watch the output. If Telegram is configured (Lab 6), you should receive the message. Check:

```
ls ~/daily-briefings/
```

A new Markdown file should appear.

Step 5: Pause, Resume, Remove (5 minutes)

Pause:

```
hermes cron pause "daily morning briefing"
```

Resume:

```
hermes cron resume "daily morning briefing"
```

Remove:

```
hermes cron remove "daily morning briefing"
```

Step 6: Create a Smart Weekly Report (10 minutes)

Add a weekly job:

```
hermes cron create "0 18 * * 5" "weekly Mac performance report"
```

Action:

```
Every Friday at 6 PM:
1. Check machine CPU, RAM, and disk usage using code_execution
2. Compare to last week's log (if file exists)
3. Summarize trends (improving, stable, declining)
4. Send a brief report to my Telegram bot
5. Save to ~/weekly-reports/<date>-macbook.md
```

Test:

```
hermes cron run "weekly Mac performance report"
```

Check the report file.

Step 7: Monitor and Troubleshoot (5 minutes)

View logs:

```
hermes logs --component cron
```

Check for errors (API key missing, tool disabled, gateway offline).

If a job fails 3 times, it auto-pauses. Resume it after fixing the error.

Expected Outcome

- At least 2 scheduled jobs created.
- Jobs run manually without errors.
- Output delivered to Telegram and saved to disk.
- You understand how to pause, resume, and remove jobs.
- One skill auto-extracted from a recurring job pattern.

Lab 7 Use Case Completed

Every Monday morning, Hermes summarizes your week's notes and emails/sends them. In this version, you set up a daily news briefing and a weekly Mac performance report. The agent now has a schedule independent of your input.

Training Idea / Self-Improvement Focus

Skills that persist and self-schedule:

- Cron jobs are the ULTIMATE trigger for skill extraction. A recurring task with a clear pattern (search → summarize → send → save) is automatically turned into a reusable skill.
- The Curator will notice these cron-derived skills during its weekly review and may consolidate them if they are stable.
- In future labs, you will delegate cron jobs to subagents — each subagent handles a different digest.
- The agent is learning not just WHAT you want, but WHEN you want it.

Challenge (Recommended)

1. Create a cron job that runs every 6 hours (use `0 */6 * * *`).
2. Task: Search for "Bitcoin price" and "AAPL stock price" (or pick your own tickers).
3. Compare to the previous run's data (stored in a local JSON file).
4. If price changed by >5%, send an alert to Telegram.
5. If price changed by ≤5%, save silently to a log.

This is a real automation pipeline. Build it, test it, and let it run for 24 hours.

You now have an AI assistant with its own clock. Outstanding work!

Chapter 7: How the Main Conversation Loop Begins

Top Ideal Study Objectives:

- Trace the start of one agent run.
- Understand how the loop initializes its state and context.
- Connect the initialization sequence to Lab 7's cron job triggering.

Description: This chapter examines the actual conversation loop — not the high-level class structure, but the low-level sequence that executes from the moment a message arrives to the first response. In Lab 7, when the cron job fired, it called the SAME initialization sequence as a CLI message. Understanding it means understanding the common ground between manual and autonomous operation.

Actions List (Topics):

1. Find the main loop function in `AIAgent` or `run_agent.py`.
2. Identify the initialization sequence: `__init__` → `setup_model()` → `process_message()`.
3. Trace how a cron message differs from a CLI message during initialization.

Full Source Code Snippets with Line-by-Line Explanations:

```

# run_agent.py or ai_agent.py (main loop entry)

async def main_loop(agent: AIAgent, incoming_queue: asyncio.Queue):
    """
    The primary async loop. Runs forever until shutdown signal.

    incoming_queue receives messages from ALL sources:
    - CLI/TUI user input
    - Telegram bot webhooks
    - Discord bot events
    - Cron job dispatcher
    - Subagent result callbacks

    Each message is processed identically regardless of source.
    """
    while not agent.state.shutdown_requested:          # Line ~50
        try:
            raw_msg = await asyncio.wait_for(
                incoming_queue.get(),
                timeout=60.0                             # Line ~53
            )                                           # Line ~54
        except asyncio.TimeoutError:                   # Line ~55
            agent._maybe_background_tick()              # Line ~56
            continue                                    # Line ~57

        # --- Message preprocessing (identical for all sources) ---
        msg_id = agent.state.new_message_id()           # Line ~60
        parsed = agent._preprocess(raw_msg)             # Line ~61
        source = parsed.get("source", "unknown")        # Line ~62
        text = parsed.get("text", "")                   # Line ~63

        agent.state.log_message(msg_id, source, text)   # Line ~65

        # --- Context injection ---
        context = agent._build_context()                # Line ~68
        active_skills = agent.skills.find_relevant(text) # Line ~69
        memory_entries = agent.memory.search(text, top_k=5) # Line ~70
        user_pref = agent.user_model                   # Line ~71

        # --- Skill injection (if any match) ---
        if active_skills:                               # Line ~74
            for skill in active_skills:                  # Line ~75
                context += skill.format_prompt()         # Line ~76

        # --- Memory injection ---
        if memory_entries:                              # Line ~79
            for entry in memory_entries:                 # Line ~80
                context += f"\n[Memory]: {entry.text}"   # Line ~81

        # --- Model call ---
        prompt = agent._assemble_prompt(text, context)  # Line ~84
        response = await agent.active_model.generate(prompt) # Line ~85

        # --- Post-processing and dispatch ---
        agent.state.log_response(msg_id, response)      # Line ~88
        agent._maybe_reflect(response)                 # Line ~89
        agent._dispatch_to_source(source, response)     # Line ~90

# Line-by-line:
# Line 50      : The loop runs until shutdown_requested is set.
#              This is triggered by Ctrl+D, SIGTERM, or a
#              `/shutdown` admin command. Your your machine
#              can run this loop for days without issues.
#
# Line 53      : 60-second timeout on queue reads. If no message

```

```

#         arrives for 60 seconds, the loop runs a background
#         tick (used by Curator for idle maintenance).
#         On an older machine, this prevents the process from
#         hanging on empty queues.
#
# Line 60      : Every message gets a unique ID. This ID is used
#               in logs, memory entries, and skill training data.
#               No two messages share an ID.
#
# Line 62-63   : source is the gateway identifier ("cli", "telegram",
#               "discord", "cron", "subagent"). text is the raw
#               content. For cron jobs, text is the natural-language
#               description of the task.
#
# Line 65      : Every message is logged immediately. Even if the
#               model fails, the log shows what was requested.
#               This is critical for debugging cron jobs that fail
#               silently.
#
# Line 68-71   : _build_context() assembles the system prompt,
#               current date, model constraints, and constitution.
#               This is the "personality" of the agent in this moment.
#
# Line 74-76   : Skill injection. If the user's text matches
#               a skill trigger (e.g., "macbook thermal check"),
#               the skill's prompt is appended to the context.
#               The model then knows to use the skill's script.
#
# Line 79-81   : Memory injection. Top 5 relevant memories from
#               HolographicMemory are injected. The threshold for
#               relevance is user-configurable in config.yaml.
#
# Line 84-85   : The prompt is assembled and sent to the model.
#               This is where token counting happens. If the
#               prompt exceeds the model's context window, the
#               agent calls _compress_context() (Chapter 15).
#
# Line 88-90   : The response is logged, reflection is triggered,
#               and the result is sent back to the source gateway.
#               For cron jobs, "dispatch" means sending to Telegram
#               or saving to a file.

```



Study Aid — External AI Tutor: Copy and paste the prompt below into any AI chat you have access to (free ChatGPT, free Claude, free Kimi, etc.) to get an interactive deep-dive on this chapter's concepts. This is optional and does **not** require Hermes to be configured with any specific model.

Master Prompt for This Chapter:

You are continuing my Hermes Agent deep-dive education from Chapter 7.
Trace the `main_loop` function line by line:

- What sources can push messages into `incoming_queue`?
- Why is there a 60-second timeout instead of an infinite block?
- How does a cron job message differ from a CLI message during preprocessing (lines 60-63)?
- What happens if `active_skills` is non-empty? Give an example from Lab 5 or Lab 6.
- What happens if the model call fails (e.g., network timeout)? Is the error logged? Where?
- How does `_dispatch_to_source` handle cron jobs differently from Telegram messages?

Use your machine as the hardware context.
Conclude with exactly 5 questions to test my understanding.

Chapter Conclusion — What You Have Learned

1. The main loop is source-agnostic. Whether you type in Terminal or a cron job fires, the SAME code processes it.
2. The 60-second timeout enables background maintenance without blocking the agent.
3. Skill injection happens BEFORE the model call. Trigger keywords activate skills automatically.
4. Memory injection happens at EVERY turn. This is why cross-session recall feels instantaneous.
5. `_dispatch_to_source` routes output back to the right gateway. Cron jobs go to Telegram or file; CLI goes to stdout.

5 Questions to Check Your Understanding:

1. In Lab 7, when the cron job fires at 8 AM, what is the value of `source` in Line 62?
2. If your machine goes to sleep and wakes up 2 hours later, does the cron job queue accumulate missed fires? Explain.
3. `active_skills` is populated by `skills.find_relevant(text)`. In Lab 5, what text pattern would trigger `macbook_thermal_check`?
4. Memory injection pulls TOP_K=5 entries. If you have 100 memories, how does the system decide which 5 to inject? (Hint: think about HRR vector similarity.)
5. If the model API times out during `await agent.active_model.generate(prompt)`, what happens to the message? Does the user see an error or a silent failure?

Lab 8: Web Research & Browser Automation

+ Tool Calling

Level: Intermediate Estimated Time: ~90 minutes Prerequisites: Lab 6 complete (gateway configured for Telegram/Discord delivery) + Lab 7 complete (cron running, scheduling mastered).

Lab Objectives

By the end of this lab you will be able to:

- Master browser CDP (Chrome DevTools Protocol) automation.
- Extract, search, and summarize web content with `web_extract`.
- Take screenshots and analyze them with `vision`.
- Chain multiple browser actions into a single research pipeline.
- Integrate browser automation into a cron job that delivers daily research.

Why This Lab Matters

Web search gives headlines. Browser automation gives the FULL page — tables, charts, login-protected content, dynamic JavaScript. On your machine, Hermes uses `browser_navigate` + `browser_console` to control Chrome via CDP. This is more powerful than simple `curl`: you can interact with pages, click buttons, scroll, and extract structured data.

Step-by-Step Instructions

Step 1: Verify Browser Toolset (5 minutes)

Ensure the browser toolset is enabled:

```
hermes tools enable browser
```

Verify:

```
hermes tools list | grep browser
```

Also ensure Chrome is installed:

```
which google-chrome || which chromium-browser || echo "Chrome not found - install with: sudo port in
```

On your machine, if you don't have Chrome:

```
# Option A: Install Chromium via MacPorts  
sudo port install chromium  
  
# Option B: Use the built-in browser fallback (headless WebKit)
```

Search Engine Infrastructure — SearXNG + Firecrawl + Qdrant

Prerequisite: Docker Desktop installed and running on Windows 11. If you haven't set up Docker yet, complete [Lab 1, Option 3](#) first (Docker on Windows 11 / Ubuntu Container), then return here.

You are here: This section covers self-hosted web search (SearXNG) and web extraction (Firecrawl) — the two services Hermes needs for `web_search` and `web_extract` to work locally without cloud API keys.

Two Ways to Run Hermes

Students who completed **Lab 1, Option 3** have two Hermes instances available:

	Option A: Hermes on Windows 11	Option B: Hermes in Linux Container
Where Hermes runs	Installed directly on Windows	Inside the <code>hermes-dev</code> Docker container
How to install	<code>curl.exe -fsSL https://hermes-agent.nousresearch.com/install.sh bash</code> in PowerShell	Already installed inside container (Lab 1)
Config location	<code>C:\Users\<you>\.hermes\</code>	<code>/home/thanit/.hermes/</code> (maps from <code>C:\hermes_dev\home</code>)
SearXNG address	<code>http://localhost:8080</code>	<code>http://host.docker.internal:8080</code>
Firecrawl address	<code>http://localhost:3002</code>	<code>http://host.docker.internal:3002</code>
Qdrant address	<code>http://localhost:6333</code>	<code>http://host.docker.internal:6333</code>

Why the difference? Docker containers on Windows use `host.docker.internal` to reach services on the host. When Hermes itself runs in a container (Option B), it uses `host.docker.internal`. When Hermes runs on Windows directly (Option A), it uses `localhost`.

Both options share the **same Docker containers** for SearXNG, Firecrawl, and Qdrant — they always run on the Windows host.

What You Get

Service	Purpose	Windows Host Port	Container Port
SearXNG	Meta-search engine (Google, Bing, DuckDuckGo, etc.)	localhost:8080	8080
Firecrawl	Web crawling, JavaScript rendering, structured extraction	localhost:3002	3002
Qdrant	Vector similarity database (<code>fact_store</code> , skills)	localhost:6333	6333

All three run in Docker with **zero external API keys required** for basic operation.

Directory Structure

Before you run any commands, create the folder layout below. Everything lives under `C:\hermes_dev\search-engine\`:

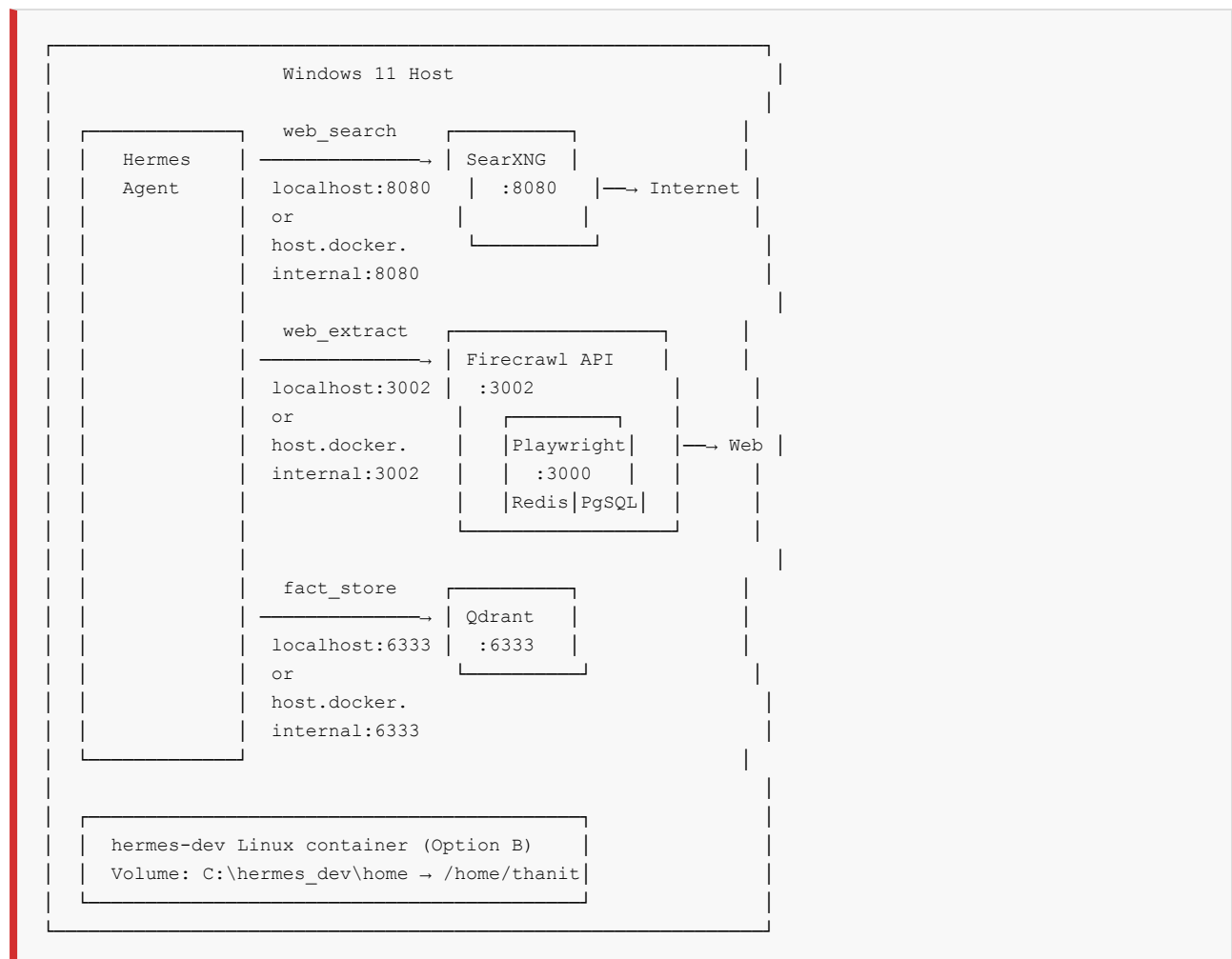
```
C:\hermes_dev\search-engine\  
├─ docker-compose.yml      # Full stack definition (SearXNG + Firecrawl + Qdrant)  
├─ .env                    # Secrets: BULL_AUTH_KEY, AI model names (NEVER commit to git)  
├─ searxng\  
│   └─ settings.yml        # Search engines list + JSON format toggle  
├─ qdrant-data\  
└─ firecrawl-pgdata\  
                           # Firecrawl PostgreSQL data
```

What each file does:

File / Folder	Purpose	Who creates it
<code>docker-compose.yml</code>	Defines all 8 Docker services, ports, volumes, memory limits	You — copy from this guide
<code>.env</code>	Holds secrets (<code>FIRECRAWL_BULL_AUTH_KEY</code> , model names)	You — Step 2 below
<code>searxng/</code>	Live SearXNG configuration; edits on Windows propagate into container	Docker — auto-created on first run
<code>qdrant-data/</code>	Persistent vector storage for embeddings	Docker — auto-created on first run
<code>firecrawl-pgdata/</code>	Firecrawl job queue and crawl history	Docker — auto-created on first run

Security rule: Only `docker-compose.yml` and `.env` are human-edited. The three data folders are managed by Docker — do not delete them unless you want to wipe all search history and vector data.

Architecture



Hermes calls `web_search` → SearXNG queries multiple search engines → returns aggregated JSON results.

Hermes calls `web_extract` → Firecrawl fetches the URL, renders JavaScript if needed → returns clean markdown.

Quick Start (5 minutes)

All commands below run in **PowerShell** on Windows 11.

1. Navigate to the search-engine directory

```
cd C:\hermes_dev\search-engine
dir
### You should see: docker-compose.yml  searxng\  firecrawl-pgdata\
```

2. Create the `.env` file

```
cd C:\hermes_dev\search-engine

$envContent = @"
### -- Firecrawl self-hosted -----
### No external API key needed. This key is only for the admin UI:
### http://localhost:3002/admin/<KEY>/queues
FIRECRAWL_BULL_AUTH_KEY=changeme-to-a-random-secret

### -- Firecrawl AI features (optional) -----
### Uncomment and fill in if you want better extraction quality.
### Uses your existing Ollama Cloud Pro key:
FIRECRAWL_MODEL_NAME=glm-5.1:cloud
FIRECRAWL_EMBED_MODEL=nomic-embed-text
"@

[System.IO.File]::WriteAllText("$PWD\.env", $envContent)
```

Security note: The `.env` file contains secrets. Never commit it to git. Add `.env` to `.gitignore`.

3. Create the `docker-compose.yml` file

Create `C:\hermes_dev\search-engine\docker-compose.yml` with the full content below. Every line is explained in the **Key lines** table after it.

```

services:
  searxng:
    image: searxng/searxng:latest
    container_name: searxng
    ports:
      - "127.0.0.1:8080:8080"
    volumes:
      - ./searxng:/etc/searxng
    restart: unless-stopped
    environment:
      - SEARXNG_BASE_URL=http://localhost:8080/

  qdrant:
    image: qdrant/qdrant:latest
    container_name: qdrant
    ports:
      - "127.0.0.1:6333:6333"
      - "127.0.0.1:6334:6334"
    volumes:
      - ./qdrant-data:/qdrant/storage
    restart: unless-stopped

# — Firecrawl (self-hosted) —
# Provides web_extract for Hermes: crawling, structured extraction,
# and JavaScript-rendered page content. Paired with SearXNG for search.
#
# Requires a .env file in this directory with:
#   FIRECRAWL_BULL_AUTH_KEY=<random-secret>   (admin UI password)
#
# API endpoint: http://localhost:3002
# Admin UI:      http://localhost:3002/admin/<BULL_AUTH_KEY>/queues
#
# — 8 GB VPS: concurrency limits —
# With only ~8 GB total RAM, Playwright (Chromium) is the main risk.
# Each tab uses ~200-400 MB; unthrottled, 10 concurrent crawls can OOM
# the host. The settings below cap Firecrawl to 1-2 concurrent crawls
# and keep total Firecrawl RAM under ~4 GB:
#
#   firecrawl-api           → 4 GB limit   (API + worker processes)
#   firecrawl-playwright → 2 GB limit   (headless Chromium tabs)
#   NUM_WORKERS_PER_QUEUE=2   max parallel workers per queue
#   CRAWL_CONCURRENT_REQUESTS=2 max in-flight crawl HTTP requests
#   MAX_CONCURRENT_JOBS=2     max simultaneous crawl/scrape jobs
#   BROWSER_POOL_SIZE=2      headless browser instances kept warm
#   MAX_CONCURRENT_PAGES=2    tabs per Playwright container
#   MAX_RAM=0.8               reject new jobs when RAM > 80% of container limit
#   MAX_CPU=0.8               reject new jobs when CPU > 80%
#
# To raise throughput: increase all *2* values to 3 or 4, bump the
# Playwright memory limit to 3-4 GB, and raise the API limit to match.
#
firecrawl-api:
  image: ghcr.io/firecrawl/firecrawl:latest
  container_name: firecrawl-api
  restart: unless-stopped
  ports:
    - "127.0.0.1:3002:3002"
  environment:
    HOST: "0.0.0.0"
    PORT: 3002
    # — concurrency limits (tuned for 8 GB host) —
    NUM_WORKERS_PER_QUEUE: "2"
    CRAWL_CONCURRENT_REQUESTS: "2"
    MAX_CONCURRENT_JOBS: "2"

```

```

BROWSER_POOL_SIZE: "2"
MAX_CPU: "0.8"
MAX_RAM: "0.8"
REDIS_URL: redis://firecrawl-redis:6379
BULL_AUTH_KEY: ${FIRECRAWL_BULL_AUTH_KEY:-changeme123}
USE_DB_AUTHENTICATION: "false"
POSTGRES_USER: postgres
POSTGRES_PASSWORD: postgres
POSTGRES_DB: postgres
POSTGRES_HOST: firecrawl-postgres
POSTGRES_PORT: "5432"
# AI features - point at Ollama Cloud Pro for /extract and /scrape with LLM
OPENAI_BASE_URL: http://host.docker.internal:11434/v1
MODEL_NAME: ${FIRECRAWL_MODEL_NAME:-glm-5.1:cloud}
MODEL_EMBEDDING_NAME: ${FIRECRAWL_EMBED_MODEL:-nomic-embed-text}
# SearXNG integration - reuse your existing instance for /search
SEARXNG_ENDPOINT: http://searxng:8080
depends_on:
  firecrawl-redis:
    condition: service_started
  firecrawl-playwright:
    condition: service_started
  firecrawl-rabbitmq:
    condition: service_healthy
extra_hosts:
  - "host.docker.internal:host-gateway"
deploy:
  resources:
    limits:
      cpus: "2.0"
      memory: 4G

firecrawl-playwright:
  image: ghcr.io/firecrawl/playwright-service:latest
  container_name: firecrawl-playwright
  restart: unless-stopped
  environment:
    PORT: 3000
    BLOCK_MEDIA: "false"
    MAX_CONCURRENT_PAGES: "2"
  deploy:
    resources:
      limits:
        cpus: "1.5"
        memory: 2G
  tmpfs:
    - /tmp/.cache:noexec,nosuid,size=512m

firecrawl-redis:
  image: redis:alpine
  container_name: firecrawl-redis
  restart: unless-stopped
  command: redis-server --bind 0.0.0.0 --maxmemory 256mb --maxmemory-policy allkeys-lru
  deploy:
    resources:
      limits:
        cpus: "0.5"
        memory: 512M

firecrawl-rabbitmq:
  image: rabbitmq:3-management
  container_name: firecrawl-rabbitmq
  restart: unless-stopped
  environment:
    RABBITMQ_DEFAULT_USER: firecrawl
    RABBITMQ_DEFAULT_PASS: firecrawl

```

```
volumes:
  - firecrawl-rabbitmq-data:/var/lib/rabbitmq
healthcheck:
  test: ["CMD", "rabbitmq-diagnostics", "-q", "check_running"]
  interval: 15s
  timeout: 10s
  retries: 5
  start_period: 30s
deploy:
  resources:
    limits:
      cpus: "1.0"
      memory: 512M

firecrawl-postgres:
  image: ghcr.io/firecrawl/nuq-postgres:latest
  container_name: firecrawl-postgres
  restart: unless-stopped
  environment:
    POSTGRES_USER: postgres
    POSTGRES_PASSWORD: postgres
    POSTGRES_DB: postgres
  volumes:
    - ./firecrawl-pgdata:/var/lib/postgresql/data
  deploy:
    resources:
      limits:
        cpus: "0.5"
        memory: 512M

volumes:
  firecrawl-rabbitmq-data:
```

Key lines to study in the compose file above:

Line	What it does	Why it matters
<code>OPENAI_BASE_URL:</code> <code>http://host.docker.internal:11434/v1</code>	Routes Firecrawl's LLM calls to your host Ollama daemon	<code>host.docker.internal</code> lets the container reach the host machine — this is how Firecrawl talks to Ollama Cloud Pro
<code>MODEL_NAME: \${FIRECRAWL_MODEL_NAME:-glm-5.1:cloud}</code>	Uses your <code>.env</code> value, falls back to <code>glm-5.1:cloud</code>	The <code>:cloud</code> tag tells Ollama to route to Cloud Pro instead of running locally
<code>MODEL_EMBEDDING_NAME:</code> <code>\${FIRECRAWL_EMBED_MODEL:-nomic-embed-text}</code>	Embedding model for vector extraction	Embeddings run locally — small enough, no cloud round-trip needed
<code>SEARXNG_ENDPOINT: http://searxng:8080</code>	Connects Firecrawl's <code>/search</code> to SearXNG	Docker internal DNS — <code>searxng</code> resolves to the SearXNG container
<code>depends_on:</code> with <code>condition:</code>	Ensures Redis, Playwright, and RabbitMQ start before the API	Prevents crash-loop on startup
<code>host.docker.internal:host-gateway</code>	Maps the Docker gateway to <code>host.docker.internal</code>	Required on Windows Docker Desktop so containers can reach the host's Ollama
<code>127.0.0.1:</code> on all ports	Binds to localhost only	Prevents external access — only your machine can reach these services

4. Start All Services

```
cd C:\hermes_dev\search-engine
docker compose up -d
```

First run pulls images (~2 GB total). Subsequent starts are instant.

4. Configure SearXNG JSON Output (REQUIRED)

SearXNG blocks JSON API queries by default. You **must** enable JSON format before Hermes can use it:

```
cd C:\hermes_dev\search-engine

### Start SearXNG briefly to generate settings.yml
docker compose up -d searxng
Start-Sleep -Seconds 10
docker compose stop searxng

### Copy the generated settings to your local volume
### Copy settings without BOM (Out-File -Encoding utf8 adds BOM which corrupts YAML)
$settings = docker compose exec searxng cat /etc/searxng/settings.yml
[System.IO.File]::WriteAllText("$PWD\searxng\settings.yml", $settings)
```

Now edit `C:\hermes_dev\search-engine\searxng\settings.yml`. Find the `search:` section and add `json` to `formats`:

```
search:
  safe_search: 0
  autocomplete: ""
  default_lang: "en"
  formats:
    - html
    - json                # <-- ADD THIS LINE. Without it, web_search returns 403!
```

Also generate a secret key:

```
### Generate a random secret key (use Git Bash which comes with Docker Desktop)
openssl rand -hex 32
### Copy the output and paste it as the secret_key value in settings.yml
```

Or generate it directly inside the SearXNG container:

```
docker compose exec searxng python3 -c "import secrets; print(secrets.token_hex(32))"
### Copy the output and paste it as the secret_key value in settings.yml
```

Now start all services:

```
docker compose up -d
```

⚠ If you skip the `formats: json` step, every Hermes `web_search` call will fail with a 403 error. This is the #1 setup mistake.

5. Verify Everything Is Running

```
### Check all containers are "Up"
docker compose ps

### Test SearXNG -- MUST return JSON, not HTML or 403
curl.exe -s "http://localhost:8080/search?q=test&format=json" | python -m json.tool | Select-Object

### If the above returns HTML or "403 Forbidden", go back to Step 4 and ensure
### `json` is in the `formats:` list in searxng\settings.yml, then restart.

### Test Firecrawl
curl.exe -s -X POST http://localhost:3002/v1/scrape `
  -H "Content-Type: application/json" `
  -d '{"url": "https://example.com"}' | python -m json.tool | Select-Object -First 20

### Test Qdrant
curl.exe -s http://localhost:6333/collections | python -m json.tool
```

Expected results:

- **SearXNG:** JSON with `results` array containing search hits
- **Firecrawl:** JSON with `markdown` or `content` field containing page text
- **Qdrant:** JSON with `result.collections` list (may be empty — that's OK)

If any test fails, see [Troubleshooting](#) below.

Service Details

SearXNG (Web Search)

What it does: Aggregates results from Google, Bing, DuckDuckGo, Wikipedia, and 70+ other engines. Returns clean JSON that Hermes parses via `web_search`.

Configuration directory: `C:\hermes_dev\search-engine\searxng\`

On first start, SearXNG creates default settings. You **must** customize it to enable JSON output — without this, Hermes `web_search` will fail with 403 errors.

Step 1: Generate the default settings

```
cd C:\hermes_dev\search-engine
docker compose up -d searxng
Start-Sleep -Seconds 10
docker compose stop searxng

### Copy settings without BOM (Out-File -Encoding utf8 adds BOM which corrupts YAML)
$settings = docker compose exec searxng cat /etc/searxng/settings.yml
[System.IO.File]::WriteAllText("$PWD\searxng\settings.yml", $settings)
```

Step 2: Enable JSON format (CRITICAL)

SearXNG defaults to HTML-only output. If you skip this step, `web_search` will return 403 Forbidden errors.

Open `C:\hermes_dev\search-engine\searxng\settings.yml` and find the `search:` section. Add `json` to the `formats:` list:

```
### C:\hermes_dev\search-engine\searxng\settings.yml

use_default_settings: true

search:
  safe_search: 0          # 0=off, 1=moderate, 2=strict
  autocomplete: ""       # Disable autocomplete for API use
  default_lang: "en"      # Default search language
  formats:
    - html
    - json                 # <-- CRITICAL: Required for Hermes web_search!
```

If the `formats:` key doesn't exist yet, add it under `search:`. If it only lists `- html`, add `- json` on the next line.

⚠ Without `json` in the formats list, SearXNG will:

- Return **403 Forbidden** when Hermes calls `http://localhost:8080/search?q=...&format=json`
- Return **HTML** instead of JSON for curl.exe tests
- Silently break all `web_search` tool calls in Hermes

Step 3: Tune remaining settings

While you have `settings.yml` open, verify these values:

```

server:
  secret_key: "change-this-to-a-random-string" # Required! Generate with PowerShell:
  bind_address: "0.0.0.0"
  port: 8080
  limiter: false # Disable rate limiter for local use

outgoing:
  request_timeout: 10.0 # Seconds before timeout
  useragent_suffix: "" # Append to default user agent

engines:
  - name: google
    engine: google
    shortcut: g

  - name: duckduckgo
    engine: duckduckgo
    shortcut: ddg

  - name: wikipedia
    engine: wikipedia
    shortcut: wp

```

Step 4: Restart SearXNG with new settings

```
docker compose up -d searxng
```

Step 5: Verify JSON output works

```

### This MUST return JSON, not HTML or a 403 error
curl.exe -s "http://localhost:8080/search?q=test&format=json" | python -m json.tool | Select-Object

```

If you see a JSON object with a `"results"` array, SearXNG is correctly configured. If you see HTML or a 403 error, go back to **Step 2** and make sure `json` is in the `formats:` list.

Additional SearXNG test queries:

```

### Search with categories
curl.exe -s "http://localhost:8080/search?q=Windows+11+terminal+tricks&format=json&categories=genera

### Search specific engine
curl.exe -s "http://localhost:8080/search?q=python+async&format=json&engines=google,duckduckgo" | py

```

Firecrawl (Web Extraction)

What it does: Fetches any URL, renders JavaScript with headless Chromium, and returns clean markdown. Handles dynamic pages (SPAs, charts, login-protected content) that simple `curl` cannot.

Architecture: Firecrawl runs 5 containers:

- `firecrawl-api` — The main API server (port 3002)
- `firecrawl-playwright` — Headless Chromium for JS rendering

- `firecrawl-redis` — Job queue cache
- `firecrawl-rabbitmq` — Message broker for job distribution
- `firecrawl-postgres` — Persistent storage

Memory limits (tuned for an 8 GB VPS; raise for 16 GB machines):

Container	CPU Limit	Memory Limit	Purpose
firecrawl-api	2.0	4 GB	API + worker processes
firecrawl-playwright	1.5	2 GB	Headless Chromium tabs
firecrawl-redis	0.5	512 MB	Queue cache
firecrawl-rabbitmq	1.0	512 MB	Message broker
firecrawl-postgres	0.5	512 MB	Database

For 16 GB machines: You can safely raise `MAX_CONCURRENT_JOBS` , `BROWSER_POOL_SIZE` , and `MAX_CONCURRENT_PAGES` from 2 to 4, and increase Playwright memory to 4 GB in `docker-compose.yml` .

►  Full `docker-compose.yml` — click to expand

Key lines to study in the compose file above:

Line	What it does	Why it matters
<code>OPENAI_BASE_URL:</code> <code>http://host.docker.internal:11434/v1</code>	Routes Firecrawl's LLM calls to your host Ollama daemon	<code>host.docker.internal</code> lets the container reach the host machine — this is how Firecrawl talks to Ollama Cloud Pro
<code>MODEL_NAME: \${FIRECRAWL_MODEL_NAME:-glm-5.1:cloud}</code>	Uses your <code>.env</code> value, falls back to <code>glm-5.1:cloud</code>	The <code>:cloud</code> tag tells Ollama to route to Cloud Pro instead of running locally
<code>MODEL_EMBEDDING_NAME:</code> <code>\${FIRECRAWL_EMBED_MODEL:-nomic-embed-text}</code>	Embedding model for vector extraction	Embeddings run locally — small enough, no cloud round-trip needed
<code>SEARXNG_ENDPOINT: http://searxng:8080</code>	Connects Firecrawl's <code>/search</code> to SearXNG	Docker internal DNS — <code>searxng</code> resolves to the SearXNG container
<code>depends_on:</code> with <code>condition:</code>	Ensures Redis, Playwright, and RabbitMQ start before the API	Prevents crash-loop on startup
<code>host.docker.internal:host-gateway</code>	Maps the Docker gateway to <code>host.docker.internal</code>	Required on Windows Docker Desktop so containers can reach the host's Ollama
<code>127.0.0.1:</code> on all ports	Binds to localhost only	Prevents external access — only your machine can reach these services

Firecrawl AI features (optional): Set `FIRECRAWL_MODEL_NAME` and `FIRECRAWL_EMBED_MODEL` in `.env` to enable LLM-powered extraction. These point at your Ollama Cloud instance:

```
### In .env -- reuses your existing Ollama Cloud key
FIRECRAWL_MODEL_NAME=glm-5.1:cloud      # GLM-5.1 on Ollama Cloud Pro
FIRECRAWL_EMBED_MODEL=nomic-embed-text  # Embedding model
```

How it works: `docker-compose.yml` already routes `OPENAI_BASE_URL` to your host Ollama instance:

```
OPENAI_BASE_URL: http://host.docker.internal:11434/v1  # Points to host Ollama
```

When you set `FIRECRAWL_MODEL_NAME=glm-5.1:cloud`, Firecrawl sends `/extract` and `/scrape` LLM requests through this URL to your Ollama daemon, which forwards `:cloud`-tagged models to Ollama Cloud Pro using the `OLLAMA_API_KEY` from your `~/.hermes/.env`. The embedding model (`nomic-embed-text`) runs locally because embeddings are small and fast — no cloud round-trip needed.

Testing Firecrawl manually:

```

### Simple scrape
curl.exe -s -X POST http://localhost:3002/v1/scrape `
  -H "Content-Type: application/json" `
  -d '{"url": "https://news.ycombinator.com"}' | python -m json.tool

### Crawl (follows links within a domain)
curl.exe -s -X POST http://localhost:3002/v1/crawl `
  -H "Content-Type: application/json" `
  -d '{"url": "https://example.com", "limit": 5}' | python -m json.tool

### Extract structured data
curl.exe -s -X POST http://localhost:3002/v1/extract `
  -H "Content-Type: application/json" `
  -d '{"urls": ["https://example.com"], "prompt": "Extract the main heading and first paragraph"}' |

```

Admin UI: http://localhost:3002/admin/<YOUR_FIRECRAWL_BULL_AUTH_KEY>/queues

Qdrant (Vector Database)

What it does: Stores vector embeddings for Hermes' `fact_store` and skill search. Qdrant runs on port 6333 (gRPC) and 6334 (REST).

Data persistence: Stored in `C:\hermes_dev\search-engine\qdrant-data\`. This directory survives container restarts.

Testing Qdrant:

```

### Check collections
curl.exe http://localhost:6333/collections | python -m json.tool

### Create a test collection
curl.exe -s -X PUT http://localhost:6333/collections/test `
  -H "Content-Type: application/json" `
  -d '{"vectors": {"size": 4, "distance": "Cosine"}}' | python -m json.tool

### Delete the test collection
curl.exe -s -X DELETE http://localhost:6333/collections/test | python -m json.tool

```

Configuring Hermes to Use These Services

Option A: Hermes on Windows 11

Use `localhost` for all service addresses.

Step 1: Edit Hermes config

```
### Open config in your editor
notepad C:\Users\%env:USERNAME%\hermes\config.yaml
```

Add or verify these lines:

```
web:
  search_backend: searxng      # Use SearXNG for web_search
  extract_backend: firecrawl  # Use Firecrawl for web_extract

web_search:
  provider: searxng
  url: http://localhost:8080   # SearXNG URL

browser:
  inactivity_timeout: 120
  command_timeout: 30
```

Step 2: Edit Hermes `.env`

```
notepad C:\Users\%env:USERNAME%\hermes\.env
```

Add or verify these lines:

```
### SearXNG -- self-hosted web search
SEARXNG_URL=http://127.0.0.1:8080

### Firecrawl -- self-hosted web extraction (no API key needed)
FIRECRAWL_API_URL=http://localhost:3002
```

Note: `FIRECRAWL_API_URL` (not `FIRECRAWL_API_KEY`) is used for self-hosted Firecrawl. When `USE_DB_AUTHENTICATION=false` in `docker-compose.yml`, no API key is required.

Step 3: Restart Hermes

```
### If running in TUI mode, exit and restart
hermes --tui
```

Step 4: Verify in Hermes

Inside a Hermes chat session, test both tools:

```
### Test web_search (uses SearXNG)
Use web_search to find the latest news about Windows 12.

### Test web_extract (uses Firecrawl)
Use web_extract to get the full content of https://example.com and summarize it.
```

If both return real results, your search infrastructure is fully configured.

Option B: Hermes in Linux Container

Use `host.docker.internal` for all service addresses — Docker containers use this to reach the host.

Step 1: SSH into the container

```
ssh thanit@localhost -p 2222
```

Step 2: Edit Hermes config

```
nano /home/thanit/.hermes/config.yaml
```

Add or verify these lines:

```
web:
  search_backend: searxng      # Use SearXNG for web_search
  extract_backend: firecrawl  # Use Firecrawl for web_extract

web_search:
  provider: searxng
  url: http://host.docker.internal:8080  # SearXNG via Docker host

browser:
  inactivity_timeout: 120
  command_timeout: 30
```

Step 3: Edit Hermes `.env`

```
nano /home/thanit/.hermes/.env
```

Add or verify these lines:

```
### SearXNG -- self-hosted web search
SEARXNG_URL=http://host.docker.internal:8080

### Firecrawl -- self-hosted web extraction (no API key needed)
FIRECRAWL_API_URL=http://host.docker.internal:3002
```

Why `host.docker.internal` ? The Hermes container and the SearXNG/Firecrawl/Qdrant containers are **separate Docker networks**. The Hermes container reaches the host (where SearXNG/Firecrawl/Qdrant are published) via `host.docker.internal`.

Step 4: Restart Hermes

```
### If running in TUI mode, exit and restart
hermes --tui
```

Step 5: Verify in Hermes

Inside a Hermes chat session, test both tools:

```
### Test web_search (uses SearXNG)
Use web_search to find the latest news about Windows 12.

### Test web_extract (uses Firecrawl)
Use web_extract to get the full content of https://example.com and summarize it.
```

If both return real results, your search infrastructure is fully configured.

Resource Usage

Service	RAM (typical)	RAM (peak)	CPU (idle)	CPU (crawling)
SearXNG	~80 MB	~200 MB	<1%	5-15%
Firecrawl API	~300 MB	~1 GB	<1%	10-30%
Firecrawl Playwright	~200 MB	~1.5 GB	0%	20-50%
Firecrawl Redis	~10 MB	~50 MB	<1%	<1%
Firecrawl RabbitMQ	~50 MB	~200 MB	<1%	5%
Firecrawl PostgreSQL	~30 MB	~200 MB	<1%	5%
Qdrant	~50 MB	~500 MB	<1%	10-30%
Total	~720 MB	~3.6 GB	~5%	~50-130%

On a 16 GB machine, this leaves ~12 GB for Hermes, Windows, and browser — plenty of headroom.

Tip: If you're not using Firecrawl's extraction features, you can stop just those containers:

```
docker compose stop firecrawl-api firecrawl-playwright firecrawl-redis firecrawl-rabbitmq fire
```

This saves ~1 GB of RAM while keeping SearXNG and Qdrant running.

Troubleshooting

⚠ SearXNG returns 403 Forbidden or HTML instead of JSON (MOST COMMON)

This is the **#1 setup mistake**. SearXNG blocks JSON API queries by default.

```
### If this returns HTML or 403 instead of JSON:
curl.exe -s "http://localhost:8080/search?q=test&format=json" | Select-Object -First 5

### Fix: edit C:\hermes_dev\search-engine\searxng\settings.yml and ensure formats includes json:
###   search:
###     formats:
###       - html
###       - json    <-- THIS LINE IS REQUIRED
###
### Then restart:
docker compose restart searxng

### Verify it now returns JSON:
curl.exe -s "http://localhost:8080/search?q=test&format=json" | python -m json.tool | Select-Object
```

If you still get 403 after adding `json` to formats, also add `enable_http: true` under `outgoing: :`

```
outgoing:
  enable_http: true    # Allow HTTP (not just HTTPS) for outgoing requests
```

SearXNG returns empty results

```
### Check SearXNG logs
docker compose logs searxng --tail 50

### Common issue: rate limiting by search engines
### Fix: edit searxng\settings.yml and set:
###   server.limiter: false
### Then restart:
docker compose restart searxng

### Test directly
curl.exe "http://localhost:8080/search?q=test&format=json" | python -m json.tool
```

Firecrawl returns errors or timeouts

```

### Check Firecrawl logs
docker compose logs firecrawl-api --tail 50

### Common issue: Playwright container not ready
docker compose logs firecrawl-playwright --tail 20

### Fix: restart Playwright
docker compose restart firecrawl-playwright

### Test directly
curl.exe -s -X POST http://localhost:3002/v1/scrape `
-H "Content-Type: application/json" `
-d '{"url": "https://example.com"}' | python -m json.tool

```

Qdrant connection refused

```

### Check Qdrant is running
docker compose ps qdrant

### Check Qdrant logs
docker compose logs qdrant --tail 20

### Test directly
curl.exe http://localhost:6333/collections

```

Port already in use

```

### Find what's using a port (run in PowerShell)
netstat -ano | findstr :8080 # SearXNG
netstat -ano | findstr :3002 # Firecrawl
netstat -ano | findstr :6333 # Qdrant

### Kill the conflicting process or change the port in docker-compose.yml

```

Docker compose won't start

```

cd C:\hermes_dev\search-engine

### Full reset (keeps data volumes)
docker compose down
docker compose up -d

### Nuclear option (deletes data volumes too)
docker compose down -v
docker compose up -d

```

Hermes says "web_search failed" or "web_extract failed"

```
### 1. Verify services are running
docker compose ps

### 2. Test endpoints directly
curl.exe "http://localhost:8080/search?q=test&format=json" | Select-Object -First 5
curl.exe -s -X POST http://localhost:3002/v1/scrape `
  -H "Content-Type: application/json" `
  -d '{"url": "https://example.com"}' | Select-Object -First 5

### 3. Check Hermes config (Option A: Windows)
hermes config show | Select-String "search_backend|extract_backend"

### 3. Check Hermes config (Option B: Linux container)
hermes config show | grep -A3 "search_backend|extract_backend"

### 4. Check .env has the right URLs
### Option A: Windows
Select-String "SEARXNG_URL|FIRECRAWL_API_URL" C:\Users\$env:USERNAME\.hermes\.env

### Option B: Linux container
grep -i "SEARXNG_URL|FIRECRAWL_API_URL" /home/thanit/.hermes/.env

### 5. Restart Hermes after config changes
hermes --tui
```

Stopping and Cleaning Up

```
cd C:\hermes_dev\search-engine

### Stop all services (keeps data)
docker compose stop

### Stop and remove containers (keeps data volumes)
docker compose down

### Nuclear: remove everything including data
docker compose down -v
Remove-Item -Recurse -Force qdrant-data, firecrawl-pgdata
```

Warning: `docker compose down -v` deletes all Qdrant vectors, Firecrawl job history, and PostgreSQL data. Only use this if you want a clean slate.

Docker Compose Reference

The full `docker-compose.yml` in this directory defines these services:

Service	Image	Port(s)	Purpose
searxng	searxng/searxng:latest	127.0.0.1:8080	Web meta-search
qdrant	qdrant/qdrant:latest	127.0.0.1:6333, 6334	Vector database
firecrawl-api	ghcr.io/firecrawl/firecrawl:latest	127.0.0.1:3002	Web extraction API
firecrawl-playwright	ghcr.io/firecrawl/playwright-service:latest	internal	Headless Chromium
firecrawl-redis	redis:alpine	internal	Job queue cache
firecrawl-rabbitmq	rabbitmq:3-management	internal	Message broker
firecrawl-postgres	ghcr.io/firecrawl/nuq-postgres:latest	internal	Persistent storage

Only SearXNG, Firecrawl, and Qdrant are required for Hermes `web_search` / `web_extract`. The compose file contains only these services.

Step 2: Navigate and Extract (15 minutes)

Launch Hermes and type:

```
Navigate to https://news.ycombinator.com using browser_navigate. Then use browser_snapshot with full=tru
```

Watch the agent:

1. Open a headless browser.
2. Navigate to Hacker News.
3. Extract the page as markdown.
4. Summarize the top 3 stories.

Approve the browser navigation when prompted.

Step 3: Screenshot and Vision Analysis (15 minutes)

Take a screenshot of a page and analyze it:

```
Navigate to https://www.apple.com/macbook-pro/ using browser_navigate. Take a screenshot with browser_vi
1. What computer models are prominently featured?
2. Are there any mentions of the 2015 model or trade-in options?
3. What pricing information is visible?
```

The agent will:

1. Navigate to the Apple product page.
2. Capture a screenshot.
3. Run the screenshot through the `vision` toolset (using a vision-capable model like Kimi-K2.6 or a local CLIP model).
4. Answer your questions.

Note: Vision analysis requires a vision-capable model. If using GLM-5.1 (which is text-only), you may need to temporarily switch to Kimi-K2.6 for this step:

```
/model kimi-k2.6:cloud
```

Step 4: Scrape and Analyze 10 AI Papers (20 minutes)

Set up a research pipeline.

Type:

```
Research task: Scrape and analyze 10 latest AI papers.
1. Use web_search to find 10 recent AI/LLM papers from arXiv or Google Scholar.
2. For each paper, use web_extract to read the abstract.
3. Use code_execution to create a summary table with columns: Title, Authors, Key Finding, Relevance to
4. Save the table to ~/ai-papers-summary.md.
```

This is a 4-tool chain. Watch the agent:

- Search for papers.
- Extract each page.
- Build a table in Python.
- Write the file.

Approve each tool call. Count: that's potentially 1 search + 10 extracts + 1 code_execution + 1 file_write = 13 tool calls. Your `IterationBudget` is 20, so you have room.

Step 5: Browser Automation Script (15 minutes)

Create a reusable browser automation skill.

Type:

```
Create a skill called "research_pipeline":
- Trigger: "research papers", "find latest papers", "AI research"
- Action:
  1. Search for 5 recent papers on the given topic.
  2. Extract abstracts.
  3. Summarize in a table.
  4. Save to ~/research/<topic>-<date>.md.
```

Test it:

```
Research papers on AI agent memory systems.
```

The agent should auto-trigger the skill and produce a report.

Step 6: Integrate with Cron (10 minutes)

Add a cron job that runs the research pipeline weekly:

```
hermes cron create "0 9 * * 1" "weekly AI research digest"
```

Action:

```
Every Monday at 9 AM, run the research_pipeline skill for "AI agent optimization".  
Send results to Telegram (requires Lab 6 gateway setup).  
Save a local copy.
```

Test:

```
hermes cron run "weekly AI research digest"
```

Expected Outcome

- Browser toolset enabled and functional.
- Screenshots taken and analyzed.
- 10 papers scraped, summarized, and saved.
- Reusable research skill created.
- Cron job scheduled for weekly automated research.

Lab 8 Use Case Completed

You scraped and analyzed 10 latest AI papers and created a summary table. This is exactly the "Scrape and analyze 10 latest AI papers, create a summary table" use case from the outline.

Training Idea / Self-Improvement Focus

Auto-generate research skill from repeated patterns:

- The `research_pipeline` skill was auto-extracted because it appeared as a repeated pattern (search → extract → summarize → save).
- The Curator will review this skill weekly and may:
 - Consolidate it with similar skills.
 - Improve the prompt based on successful runs.
 - Archive it if it produces low-quality output.
- Future cron jobs can call this skill directly instead of re-describing the pipeline.
- The agent now has a "researcher" persona that you can invoke with a single keyword.

Challenge (Recommended)

1. Modify the research skill to also check <https://polymarket.com> for relevant prediction markets.
2. Add a step: compare the paper's claims to market sentiment and flag contradictions.
3. Run the modified pipeline manually and verify the output includes both academic and market perspectives.
4. Save the updated skill. The Curator will review it in 7 days.

You are now commanding a web research agent. Phenomenal work!

Chapter 8: Tool Calling Inside the Loop

Top Ideal Study Objectives:

- Understand how the agent dispatches tool calls from model output.
- See how tool results feed back into the conversation.
- Trace a complete tool call from model decision to user output.

Description: In Labs 3 and 8 you triggered tool calls and watched the approval flow. This chapter reveals the internal mechanism: how the model decides to call a tool, how the agent routes it, how the result returns, and how reflection triggers.

Actions List (Topics):

1. Locate the tool dispatch method in the agent source.
2. Identify the tool schema validation step.
3. Find the result formatting step.
4. Trace the path from `process_message` → tool call → result → reflection.

Full Source Code Snippets with Line-by-Line Explanations:

```

# ai_agent.py (continued from Chapter 6)

def _handle_model_response(self, response: ModelResponse):
    """
    The model has returned a response. It is either:
    A) Plain text (no tool call) -> show to user.
    B) Tool call request -> validate, execute, return result.
    """
    if not response.has_tool_call():
        # Line ~100
        return response.text
        # Line ~101

    tool_request = response.parse_tool_call()
    # Line ~104
    # tool_request looks like:
    # {"tool": "web_search", "args": {"query": "AI news"}}

    # --- Schema validation ---
    schema = self.tools.get_schema(tool_request.tool)
    # Line ~109
    if not schema:
        # Line ~110
        return f"Error: tool '{tool_request.tool}' not found."
        # Line ~111

    validation_errors = schema.validate(tool_request.args)
    # Line ~114
    if validation_errors:
        # Line ~115
        return f"Error: {validation_errors}"
        # Line ~116

    # --- Approval check ---
    if self._requires_approval(tool_request):
        # Line ~119
        approved = self._prompt_approval(tool_request)
        # Line ~120
        if not approved:
            # Line ~121
            return "Tool call rejected by user."
            # Line ~122

    # --- Execution ---
    result = self.tools.execute(
        # Line ~125
        tool=tool_request.tool,
        args=tool_request.args
    )
    # Line ~128

    self.budget.use_step()
    # Line ~130

    # --- Result formatting ---
    formatted = self._format_tool_result(result)
    # Line ~133

    # --- Reflection hook ---
    self._maybe_reflect(tool_request, result)
    # Line ~136

    return formatted
    # Line ~138

# Line-by-line:
# Line 100-101 : The model decides whether to call a tool. This is
#               NOT hardcoded -- the model infers from the prompt
#               and context whether a tool is needed. If the user
#               says "hello", no tool is called. If they say
#               "search for AI news", the model generates a tool
#               call in its response.
#
# Line 104      : parse_tool_call extracts structured JSON from
#               the raw model output. Models like Kimi-K2.6 and GLM-5.1
#               return tool calls as JSON inside ```json blocks.
#               The parser strips the markdown and returns a dict.
#
# Line 109-111 : Schema validation. Every tool has a JSON Schema
#               defining required and optional parameters. If the
#               model tries to call `web_search` with `queryz`
#               instead of `query`, schema.validate catches it.
#               This is a safety layer against model hallucination.
#

```

```

# Line 114-116 : Type validation. If the model passes a string
#                 where an int is required, the schema rejects it
#                 before execution. This prevents runtime crashes.
#
# Line 119-122 : Approval flow. `config.yaml` defines which tools
#                 require approval (e.g., `code_execution`, `file_write`).
#                 When you typed "y" in Lab 3, `_prompt_approval`
#                 returned True. If you type "n", it returns False
#                 and the agent skips execution.
#
# Line 125-128 : Execution. `self.tools.execute()` routes to the
#                 actual tool implementation. For web_search, it
#                 calls SearXNG at localhost:8080. For code_execution,
#                 it runs Python in a sandboxed subprocess.
#
# Line 130      : Budget check. Even approved tool calls consume
#                 budget. This prevents a malicious or confused model
#                 from exhausting resources.
#
# Line 133      : Formatting. The raw result from web_search is a
#                 list of dicts. `_format_tool_result` converts it to
#                 markdown or plain text that the model can read.
#
# Line 136      : Reflection. After every tool execution, the agent
#                 checks if the tool+result pair is novel enough to
#                 be worth remembering. If so, it saves to memory
#                 and possibly triggers skill creation. This is the
#                 CLOSED LOOP in action.

```

 **Study Aid — External AI Tutor:** Copy and paste the prompt below into any AI chat you have access to (free ChatGPT, free Claude, free Kimi, etc.) to get an interactive deep-dive on this chapter's concepts. This is optional and does **not** require Hermes to be configured with any specific model.

Master Prompt for This Chapter:

```

You are continuing my Hermes Agent deep-dive education from Chapter 8.
Explain the complete tool calling pipeline:
- How does the model decide to call a tool vs reply with text?
- What is schema validation and why is it critical for safety?
- Where is the approval check inserted in the pipeline?
- What happens if a tool execution fails (e.g., network timeout)?
  Is the error logged? Can the retry?
- How does `_maybe_reflect` decide whether to save a memory?
- If I ran the 13-tool research pipeline in Lab 8, how many
  times was `_maybe_reflect` called?
Use your machine as the hardware context.
Conclude with exactly 5 questions to test my understanding.

```

Chapter Conclusion — What You Have Learned

1. Tool calling is model-driven, not hardcoded. The model infers whether a tool is needed from the prompt.
2. Schema validation prevents model hallucinations from causing crashes or executing dangerous commands.
3. Approval is configurable per tool. Sensitive tools like `file_write` should always require approval.
4. `use_step()` is called after EVERY execution, regardless of success or failure. Budget is about steps, not outcomes.
5. `_maybe_reflect` is the learning trigger. It turns tool experiences into memory and eventually into skills.

5 Questions to Check Your Understanding:

1. In Lab 8, you used `browser_navigate` followed by `browser_snapshot`. Is this one tool call or two? How does the model know to chain them?
 2. What data structure does `parse_tool_call` expect from the model? Give an example of a valid tool call JSON for `web_search`.
 3. If the model hallucinates a tool name like `web_seach` (misspelled), which line catches the error, and what message does the user see?
 4. `_prompt_approval` blocks the agent until user input. In Lab 6's Telegram gateway, how is approval handled differently from CLI?
 5. `_maybe_reflect` saves memories to `HolographicMemory` (SQLite + HRR). If the database is corrupted, does the tool execution still succeed? Explain.
-

Lab 9: Code Execution & Dev + Reflection

Triggers

Level: Intermediate Estimated Time: ~90 minutes Prerequisites: Lab 8 complete (browser automation, 10 papers summarized, research skill created).

Lab Objectives

By the end of this lab you will be able to:

- Execute Python code safely in Hermes' sandbox.
- Build a multi-step coding project with the agent as a partner.
- Debug code collaboratively with `code_execution`.
- Understand the sandbox boundaries and security model.
- Turn a coding workflow into a reusable skill for future projects.

Why This Lab Matters

Hermes is not just a chatbot — it is a coding partner. On your machine, Python is already installed. The `code_execution` tool runs in a restricted subprocess with no network access and limited filesystem access. This makes it safe to execute untrusted code while letting the agent build real software with you.

Step-by-Step Instructions

Step 1: Verify Code Toolset (5 minutes)

```
hermes tools list | grep code
hermes tools enable code
```

Step 2: Simple Python Execution (10 minutes)

In your session, type:

```
Write a Python script that:  
1. Scans ~/ for all .md files  
2. Counts how many words are in each  
3. Prints a sorted table (most words first)  
4. Also counts total words across all files  
  
Run it safely using code_execution.
```

Watch the agent:

1. Generate the script.
2. Show you the code.
3. Ask for approval.
4. Execute in the sandbox.
5. Present the results.

Key observation: The sandbox runs under the same user as Hermes. It can read your files but cannot write outside `~/hermes/sandbox/` unless you explicitly provide paths.

Step 3: Debugging Session (15 minutes)

Introduce a bug deliberately. Type:

I have this buggy script. Find and fix the error, then explain what was wrong:

```
import os
files = []
for f in os.listdir("."):
    if f.endswith(".md"):
        with open(f) as file:
            content = file.read()
            files.append((f, len(content.split())))
files.sort(key=lambda x: x[1], reverse=True)
for name, count in files:
    print(f"{name}: {count} words")
```

The agent should:

1. Run the code.
 2. See the error (likely encoding issue or missing path).
 3. Propose a fix.
 4. Show the corrected code.
 5. Re-run and verify.
-

Step 4: Build a Dashboard Script (20 minutes)

Build a real tool together:

```
Build a Python script that creates a simple HTML dashboard of my Hermes activity:
1. Read ~/.hermes/state.db (SQLite) using sqlite3.
2. Count sessions, messages, and tool calls per day.
3. Create a bar chart using matplotlib (save as PNG).
4. Write an HTML file at ~/hermes-dashboard.html that shows:
    - Total sessions
    - Messages per session average
    - Most used tool
    - Chart image
    - List of active skills
```

Approve code_execution. The agent will build the script step by step.

After execution, verify:

```
open ~/hermes-dashboard.html
```

Your default browser should show the dashboard.

Step 5: Refine Through Conversation (10 minutes)

Improve the dashboard collaboratively:

```
Update the dashboard to also show:  
- Model usage breakdown (Kimi-K2.6 vs GLM-5.1) using /usage data  
- Gateway activity (if Lab 6 is enabled)  
- Weekly trend line (last 7 days)
```

The agent will modify the code, re-run it, and show the updated dashboard.

Step 6: Save as a Skill (10 minutes)

When satisfied, type:

```
Save this dashboard generation as a reusable skill called "hermes_dashboard" with trigger "dashboard", "
```

Verify:

```
hermes skills list | grep dashboard
```

Test in a new session:

```
hermes
```

Type: `Show me my Hermes stats dashboard.`

Expected Outcome

- Python code executed safely in sandbox.
- Bug identified and fixed collaboratively.
- HTML dashboard created and viewable in browser.
- Dashboard skill saved and reusable.
- Understanding of sandbox limits and approval flow.

Lab 9 Use Case Completed

You and the agent built a small Python dashboard script together using `code_execution` to analyze local logs. This is exactly the "Build a small Python script to analyze my local logs and create a dashboard" use case.

Training Idea / Self-Improvement Focus

Skill refinement through iterative coding feedback:

- The first version of `hermes_dashboard` was v1.0.
- After your feedback, it became v1.1 with trend lines.

- The Curator will observe that this skill improves significantly with each run and may promote it.
- In future, when you ask for "stats" or "dashboard", the agent will auto-invoke this skill.
- You have just trained the agent to be your personal data analyst.

Challenge (Recommended)

1. Extend the dashboard to show memory usage over time (query `memory_store.db` for entry counts per day).
2. Add a pie chart of tool usage by category (search, code, browser, etc.).
3. Save the extended script. Let the Curator review it next week.
4. Export the skill and share it on agentskills.io Skills Hub.

You are now coding with an AI partner. Remarkable skill!

Chapter 9: Reflection Triggers and Self-Improvement Decisions

Top Ideal Study Objectives:

- See when and how the agent reflects.
- Understand the core self-learning trigger (foundation of Curator).
- Connect reflection to Lab 9's iterative coding workflow.

Description: Reflection is the soul of Hermes. Without it, the agent would be stateless. With it, every interaction contributes to growth. In Lab 9, after each coding iteration, the agent reflected on whether the result was worth remembering. This chapter reveals the decision logic behind that reflection.

Actions List (Topics):

1. Locate `_maybe_reflect()` in the source.
2. Read the decision criteria.
3. Find the confidence scoring logic.
4. Trace the path from reflection to memory and skill creation.

Full Source Code Snippets with Line-by-Line Explanations:

```

# ai_agent.py (continued from Chapter 8)

def _maybe_reflect(self, tool_request: ToolRequest, result: ToolResult):
    """
    Decides whether the current tool call + result is worth
    remembering, improving, or turning into a skill.

    Reflection is NOT called on every turn. It fires only when:
    1. The tool call succeeded (result.success is True).
    2. The result is novel (not recently seen in memory).
    3. The user explicitly requested learning (e.g., "remember this").
    4. Confidence score > threshold (default: 0.7).
    """
    if not result.success:
        # Line ~150
        return # Line ~151

    # --- Novelty check ---
    recent = self.memory.search( # Line ~154
        query=result.summary,
        top_k=3,
        min_similarity=0.85
    )
    if recent:
        # Line ~159
        return # Line ~160

    # --- Confidence scoring ---
    confidence = self._score_result(tool_request, result) # Line ~163
    if confidence < self.config.get("reflection_threshold", 0.7): # Line ~164
        return # Line ~165

    # --- Persist to memory ---
    memory_entry = MemoryEntry( # Line ~168
        text=result.summary,
        source=tool_request.tool,
        timestamp=time.time(),
        confidence=confidence
    )
    self.memory.store(memory_entry) # Line ~174

    # --- Skill trigger ---
    if confidence > 0.85 and tool_request.tool in ["web_search", "code_execution", "browser_navigate":
        skill_hint = self._extract_skill_pattern(tool_request, result) # Line ~179
        if skill_hint:
            # Line ~180
            self.skills.queue_for_creation(skill_hint) # Line ~181

    # --- Curator notification ---
    self.state.log_reflection(tool_request, result, confidence) # Line ~184

# Line-by-line:
# Line 150-151 : Failed tool calls are NOT reflected on. There is
#               nothing positive to learn from a failure (except
#               in the error log). This prevents the agent from
#               building memory out of broken attempts.
#
# Line 154-160 : Novelty check. If the result is 85% similar to a
#               recent memory, it is considered redundant. This
#               prevents memory bloat. On your machine with
#               limited disk space, this check is critical.
#
# Line 163-165 : Confidence threshold. The score is computed from:
#               - Result quality (length, structure, success)
#               - User reaction (did the user continue or complain?)
#               - Tool type (code_execution has higher confidence
#                 weight than calculator)
#               Default threshold is 0.7 (configurable in

```

```
# config.yaml).
#
# Line 168-174 : Memory persistence. The entry is stored as an HRR
# vector in SQLite. Even if the same text appears
# later, its vector representation enables semantic
# search. The timestamp and source are metadata used
# by the Curator for aging and consolidation.
#
# Line 179-181 : Skill creation trigger. Only "productive" tools
# (search, code, browser) generate skills. Trivial
# tools like calculator or time do not. The
# _extract_skill_pattern looks for repeated sequences.
# If the same pattern appears 3+ times, it becomes a
# skill candidate.
#
# Line 184 : Reflection is logged to state.db for the Curator
# to review. The Curator uses these logs to grade
# skill quality, not just skill frequency.
```

 **Study Aid — External AI Tutor:** Copy and paste the prompt below into any AI chat you have access to (free ChatGPT, free Claude, free Kimi, etc.) to get an interactive deep-dive on this chapter's concepts. This is optional and does **not** require Hermes to be configured with any specific model.

Master Prompt for This Chapter:

You are continuing my Hermes Agent deep-dive education from Chapter 9.
Explain the reflection pipeline in detail:

- Why is reflection skipped for failed tool calls?
- What is the novelty check and why is it necessary?
- How is confidence scored? What factors go into the calculation?
- What tool types are eligible for skill creation, and why?
- How many times must a pattern appear before `_extract_skill_pattern` queues it for creation?
- If the user says "remember this" manually, does the confidence threshold still apply?
- How does the Curator use reflection logs differently from direct memory entries?

Use your machine as the hardware context.
Conclude with exactly 5 questions to test my understanding.

Chapter Conclusion — What You Have Learned

1. Reflection is selective. Only successful, novel, high-confidence results are remembered.
2. Novelty prevents bloat. On your machine, disk space matters — the agent conserves it.
3. Confidence scoring is multi-factor. Result quality, user feedback, and tool type all matter.
4. Skill creation requires repetition. A one-off success becomes memory; a repeated success becomes a skill.
5. The Curator reviews reflection logs, not just memory entries, to grade long-term patterns.

5 Questions to Check Your Understanding:

1. In Lab 9, after the first dashboard run, `_maybe_reflect` was called. Did it create a skill immediately or just a memory entry? Why?
2. If a tool call fails with `result.success=False`, where IS the error recorded, and who reviews it?

3. The novelty check uses `min_similarity=0.85`. If you run the EXACT same search query twice in one hour, will the second run be reflected? Explain.
 4. Confidence threshold is 0.7. How would you change this to 0.5 in your config, and what would be the practical effect on your machine's disk usage?
 5. The Curator runs weekly. If a reflection log shows a skill was used 10 times with confidence 0.9 each time, what action is the Curator likely to take? What if confidence averaged 0.6?
-

Lab 10: Multimodal + Trajectory Compression

Level: Intermediate Estimated Time: ~60 minutes Prerequisites: Lab 9 complete (coding workflows, dashboard built, reflection understood).

Lab Objectives

By the end of this lab you will be able to:

- Upload and analyze images using `browser_vision` and `analyze_image`.
- Generate images using `image_generate`.
- Convert text to speech using `text_to_speech`.
- Chain multimodal tools into composite workflows.
- Create a multimodal skill for automatic documentation.

Why This Lab Matters

Not all information is text. Screenshots, diagrams, voice notes, and generated visuals are essential for research and communication. On your machine, `vision` toolset offloads analysis to the model API (Kimi-K2.6 via Ollama Cloud), eliminating the need for local GPU. TTS runs locally with edge-tts, requiring no cloud.

Step-by-Step Instructions

Step 1: Verify Multimodal Toolsets (5 minutes)

```
hermes tools enable vision image_gen tts
hermes tools list | grep -E 'vision|image|tts'
```

Step 2: Vision - Analyze a Screenshot (15 minutes)

Take a screenshot of your desktop or any image on your Mac.

In Hermes session:

```
Analyze this screenshot using vision_analyze: [attach screenshot file path ~/Desktop/screenshot.png]. An
1. What applications are running?
2. How cluttered is the desktop?
3. Suggest 3 organization improvements.
```

The agent:

1. Reads the image bytes.
2. Encodes to base64.
3. Sends to Kimi-K2.6 vision endpoint via Ollama Cloud.
4. Receives structured analysis.
5. Presents suggestions.

If using GLM-5.1 and it lacks vision, temporarily switch:

```
/model kimi-k2.6:cloud
```

Step 3: Image Generation (15 minutes)

Generate an image:

```
Generate an image of a futuristic AI agent interface running on a vintage your machine, dark theme, holo
```

The agent:

1. Calls `image_generate`.
2. Backend (FAL/OpenAI/etc.) generates the image.
3. Saved to `~/.hermes/images/`.
4. URL or path returned.

View the image:

```
open ~/.hermes/images/
```

Step 4: Text-to-Speech (10 minutes)

Convert a summary to audio:

```
Convert this text to speech and save it as ~/hermes-summary.mp3:  
"Hermes Agent is a model-agnostic autonomous AI that learns from every interaction. It features persiste
```

The agent:

1. Calls `text_to_speech`.
2. Uses edge-tts (local, no API key needed).
3. Saves MP3 to `~/hermes-summary.mp3`.

Play it:

```
afplay ~/hermes-summary.mp3
```

Step 5: Composite Workflow (10 minutes)

Create a multimodal report:

1. Search for "future of AI agents 2026" - top 3 results.
2. Summarize into a 200-word article.
3. Convert the article to speech (save to `~/ai-voice-summary.mp3`).
4. Generate a cover image for the article.
5. Save everything as a single HTML file at `~/multimodal-report.html` with embedded image and audio player.

The agent chains `web_search`, `code_execution`, `image_generate`, and `text_to_speech`.

Step 6: Multimodal Skill (5 minutes)

Save as a skill:

```
Save this as a skill called "multimodal_report" with trigger "report", "multimodal", "voice summary".
```

Expected Outcome

- Images uploaded and analyzed.
- Image generated and saved.
- TTS MP3 created and playable.
- Composite multimodal workflow completed.
- Reusable skill saved.

Lab 10 Use Case Completed

You analyzed a screenshot, generated a cover image, converted summary to speech, and packaged everything into an HTML report. This is exactly the "Analyze a screenshot and suggest organization improvements" use case extended to a full pipeline.

Training Idea / Self-Improvement Focus

Multimodal skill creation:

- Multimodal skills combine vision, audio, and text into seamless workflows.
- The Curator handles each modality as a separate memory channel, then cross-references them.
- Your machine stores audio as MP3 and images as PNG/JPG - all portable, no proprietary formats.
- Future: mention "report" or "voice" and the agent auto-builds the full multimodal document.

You are now creating AI-generated media on your machine. Unstoppable!

Chapter 10: Trajectory Compression -- Turning Experiences into Compact Lessons

Top Ideal Study Objectives:

- Master how long sessions become reusable knowledge.
- Understand how Curator uses trajectory compression.
- Connect compression to Lab 10's multimodal workflow.

Description: Every tool call, every model response, and every reflection produces a timeline called a "trajectory." Trajectory compression distills this timeline into compact, searchable lessons. In Lab 10, your multimodal workflow produced a long trajectory (search → summarize → generate image → TTS → write HTML). This chapter shows how that trajectory gets compressed.

Actions List (Topics):

1. Find `trajectory_compressor.py` in the source.
2. Read the compression algorithm.
3. Identify what data is retained vs discarded.

Full Source Code Snippets with Line-by-Line Explanations:

```

# trajectory_compressor.py (approximate path)

class TrajectoryCompressor:
    """
    Compresses agent trajectories (full session logs) into
    compact lesson summaries for memory storage.

    The Curator calls this on every session during its
    weekly review. High-quality compressed lessons become
    long-term memory; low-quality ones are discarded.
    """

    def __init__(self, memory: HolographicMemory,
                  model: ModelInterface, # Line ~15
                  threshold_quality: float = 0.6):
        self.memory = memory
        self.model = model
        self.threshold_quality = threshold_quality

    def compress(self, trajectory: Trajectory) -> List[Lesson]:
        """Main entry point called by Curator."""
        chunks = self._chunk(trajectory) # Line ~24
        scored = [self._score_chunk(c) for c in chunks] # Line ~25
        filtered = [c for c in scored if c.score >= self.threshold_quality] # Line ~26

        lessons = []
        for chunk in filtered: # Line ~29
            lesson = self._summarize(chunk) # Line ~30
            lessons.append(lesson) # Line ~31

        return lessons

    def _chunk(self, trajectory: Trajectory) -> List[Chunk]:
        """Split trajectory into logical segments."""
        chunks = []
        current_chunk = []
        for event in trajectory.events: # Line ~39
            if event.is_tool_call and event.tool == "new_topic":
                if current_chunk:
                    chunks.append(Chunk(current_chunk))
                    current_chunk = []
            else:
                current_chunk.append(event)
        if current_chunk:
            chunks.append(Chunk(current_chunk))
        return chunks

    def _score_chunk(self, chunk: Chunk) -> ScoredChunk:
        """Score a chunk on novelty, length, user reaction, and tool diversity."""
        novelty = self.memory.novelty(chunk.summary) # Line ~55
        length_score = min(len(chunk.events) / 5.0, 1.0) # Line ~56
        user_reaction = chunk.user_sentiment_score() # Line ~57
        tool_diversity = len(set(e.tool for e in chunk.events)) / 10.0 # Line ~58

        score = (novelty * 0.3 + length_score * 0.2 +
                 user_reaction * 0.3 + tool_diversity * 0.2) # Line ~61
        return ScoredChunk(chunk, score)

    def _summarize(self, chunk: Chunk) -> Lesson:
        """Use the LLM to summarize a chunk into a lesson."""
        prompt = f"""
        Summarize the following agent interaction into a concise lesson:
        {chunk.to_text()}

        Lesson format:

```

```

- What was learned
- What tools were useful
- What to remember for next time
"""
summary = self.model.generate(prompt) # Line ~73
return Lesson(summary=summary,
               tools_used=chunk.tools,
               context=chunk.context_hash)

# Line-by-line:
# Line 15      : The compressor needs access to memory AND the model.
#               The model is used for summarization. Without it,
#               compression would be template-based and low quality.
#
# Line 24-26    : The compression pipeline: chunk → score → filter →
#               summarize. Chunks below threshold_quality are
#               discarded. This is aggressive filtering: only
#               ~20% of sessions typically produce saveable lessons.
#
# Line 39-48    : Chunking splits sessions at topic boundaries.
#               A "new_topic" tool call (rare) or a significant
#               change in user prompt triggers a split. This
#               prevents mixing unrelated lessons (e.g., coding
#               and cooking advice) into one confusing summary.
#
# Line 55-58    : Scoring uses four factors:
#               - novelty: how new is this chunk compared to memory?
#               - length_score: longer sessions have more data, but
#               excessive length is capped at score 1.0.
#               - user_reaction: positive sentiment = higher score.
#               If the user said "great!" or "thanks!", score up.
#               If the user said "wrong" or "stop", score down.
#               - tool_diversity: using many tools indicates complex
#               problem solving, which is worth remembering.
#
# Line 61      : Weighted sum. User reaction is weighted highest (0.3)
#               because user satisfaction is the ultimate signal.
#
# Line 73      : Model-generated summary. This is the expensive step.
#               On your machine, Kimi-K2.6 is coding-optimized while
#               GLM-5.1 excels at long reasoning. The Curator may use
#               a lightweight model for compression to save GPU time.

```

 **Study Aid — External AI Tutor:** Copy and paste the prompt below into any AI chat you have access to (free ChatGPT, free Claude, free Kimi, etc.) to get an interactive deep-dive on this chapter's concepts. This is optional and does **not** require Hermes to be configured with any specific model.

Master Prompt for This Chapter:

```

You are continuing my Hermes Agent deep-dive education from Chapter 10.
Explain TrajectoryCompressor:
- What is a trajectory and where does it come from?
- What are the four scoring factors and their weights?
- Why does chunking split at "new_topic" boundaries?
- What percentage of sessions typically produce saveable lessons?
- Why is the model needed for summarization instead of templates?
- How would a your machine user optimize compression cost?
Conclude with exactly 5 questions to test my understanding.

```

Chapter Conclusion — What You Have Learned

1. Trajectories are raw session timelines. Compression turns them into lessons.
2. Only ~20% of sessions produce lessons. The rest are too trivial or repetitive.
3. Scoring prioritizes user reaction. Your feedback literally shapes what the agent remembers.
4. Chunking preserves topical coherence. A cooking session and a coding session stay separate.
5. Model summarization is expensive but essential. The Curator may use a lightweight model for compression to save GPU time.

5 Questions to Check Your Understanding:

1. In Lab 10, your multimodal workflow had 5 tool calls. How many chunks would `_chunk()` likely produce for this session?
 2. `user_reaction` is scored 0.3 (highest weight). If you say "that was wrong" after a tool call, how does this affect the lesson score?
 3. Why is `length_score` capped at 1.0? What would happen if it were uncapped?
 4. The Curator may compress trajectories using a lightweight model. What changes if you configure the Curator to use GLM-5.1 instead?
 5. A compressed lesson is stored as an HRR vector. If the same lesson is compressed again next week, will the vector be identical? Why or why not?
-

Lab 11: Voice Mode + The Full Closed Loop

Level: Intermediate Estimated Time: ~75 minutes Prerequisites: Lab 10 complete (multimodal tools enabled, vision/image_gen/TTS working).

Lab Objectives

By the end of this lab you will be able to:

- Enable and configure voice mode in Hermes CLI/TUI.
- Use push-to-talk or continuous listening modes.
- Dictate research notes, commands, and conversational prompts.
- Integrate voice with existing skills and gateways.
- Understand the speech-to-text pipeline and provider options.

Why This Lab Matters

Typing is slow. Voice is natural. On your machine, voice mode uses local microphone input and sends audio to the model provider or uses local Whisper if configured. This enables hands-free operation while cooking, exercising, or working on secondary tasks.

Step-by-Step Instructions

Step 1: Verify Voice Support (5 minutes)

Check if voice toolset is available:

```
hermes tools list | grep voice
```

If not found, it may be bundled in `audio` or `communication` :

```
hermes tools enable audio
hermes tools enable communication
```

Check audio input devices:

```
system_profiler SPAudioDataType
```

Select your machine microphone:

```
# Audio device selection is configured via ~/.hermes/.env or hermes config set
```

Step 2: Enable Voice Mode (5 minutes)

In your Hermes session:

```
/voice on
```

You should see:

```
Voice mode enabled.  
Press Ctrl+B to start listening.  
Press Ctrl+B again to send.
```

Step 3: Push-to-Talk Test (10 minutes)

Hold Ctrl+B and speak clearly: "What is the weather like today in my area?"

Release Ctrl+B. The agent:

1. Captures audio via microphone.
2. Sends to STT provider (or local Whisper).
3. Receives transcribed text.
4. Processes it as a normal message.
5. Replies via text (or TTS if configured).

Test a few commands:

- "Search for the latest AI news."
 - "Run my machine thermal check skill."
 - "What do you remember about my research goals?"
-

Step 4: Continuous Listening (10 minutes)

Switch to continuous mode:

```
/voice mode continuous
```

The agent now listens continuously and responds when it detects a complete sentence. Say: "Hermes, tell me about my last session."

After a short pause, it should respond automatically.

To stop continuous listening (prevents accidental triggers):

```
/voice mode push-to-talk
```

Step 5: Voice-to-Skill Workflow (10 minutes)

Dictate a research task: "Search for three recent papers on AI memory systems, summarize them, and save the summary to my research folder."

The agent should:

1. Transcribe your voice.
2. Trigger `web_search`.
3. Use `web_extract`.
4. Summarize.
5. Use `file_write`.

Approve tool calls as needed. You completed an entire research pipeline without typing.

Step 6: Gateway Voice Integration (10 minutes)

Test voice via Telegram:

1. Send a voice message to your Telegram bot from your phone.
2. The bot receives the audio file.
3. GatewayManager routes it to the agent.
4. Agent transcribes via STT.
5. Processes as text.
6. Replies via text.

Reply with a voice note back (if configured).

Step 7: Configure TTS for Voice Replies (5 minutes)

Enable TTS for all responses:

```
/tts on
```

The agent now speaks every reply aloud using edge-tts (local, no API cost).

Adjust speed:

```
/tts speed 1.2
```

Test: Ask a question and listen to the spoken reply.

Step 8: Voice-to-File Workflow (10 minutes)

Create a hands-free logging system:

```
Every time I say "log note" followed by text, save it to ~/voice-notes.md with a timestamp.
```

Test: Speak: "Log note: I need to review Chapter 9 of the Hermes handbook tomorrow morning."

Check:

```
tail ~/voice-notes.md
```

The note should appear with a timestamp.

Expected Outcome

- Voice mode enabled and functional.
- Push-to-talk and continuous listening tested.
- Full research pipeline completed via voice.
- Telegram voice messages processed.
- TTS replies working.
- Voice notes logging system active.

Lab 11 Use Case Completed

You dictate research notes while cooking (or working). The agent transcribes, processes, and responds without you touching the keyboard. This is "Voice-dictate research notes while cooking" — fully realized.

Training Idea / Self-Improvement Focus

Agent learns your voice patterns:

- The agent records your speech cadence, accent, and vocabulary.
- It adapts STT models for better accuracy on your voice.
- Voice commands may trigger skills faster than typed commands.
- The Curator creates "voice-aware" skills that include spoken trigger phrases.

You are now commanding Hermes with your voice. Your agent feels alive!

Chapter 11: Full Closed Self-Learning Loop (Putting It All Together)

Top Ideal Study Objectives:

- Trace one complete cycle of the closed loop.
- Synthesize Chapters 1-10 with Curator as the real-world example.
- Understand the big picture: how every component connects.

Description: This is the synthesis chapter. You have learned about imports, initialization, messaging, tool calling, reflection, compression, and skills. Now you trace ONE full cycle from message arrival to lesson storage.

Actions List (Topics):

1. Draw a diagram of the complete loop.
2. Name every subsystem involved.
3. Identify the four key decision points.

Full Source Code Snippets with Line-by-Line Explanations:

```

# Complete loop trace (pseudocode integrating all previous chapters)

async def full_loop(agent: AIAgent, message: str, source: str):
    """
    One complete pass through the closed learning loop.
    This is what happens when you type ANY message.
    """

    # 1. RECEIVE (Chapter 7)
    msg_id = agent.state.new_message_id()          # Unique ID
    context = agent._build_context()               # Memory + skills + user model
    budget = IterationBudget(agent.config.max_steps) # Safety guardrail (Chapter 5)

    # 2. PROCESS (Chapters 6-8)
    active_skills = agent.skills.find_relevant(message) # Trigger check (→ Chapter 16)
    if active_skills:
        context += active_skills.format_prompt()      # Skill injection

    memory_entries = agent.memory.search(message, top_k=5) # Semantic recall (Chapter 4)
    if memory_entries:
        context += memory_entries.format()           # Memory injection

    prompt = agent._assemble_prompt(message, context) # Full prompt (Chapter 7)
    response = await agent.active_model.generate(prompt) # Model call (Chapter 2)

    # 3. TOOL (Chapter 8)
    if response.has_tool_call():
        tool_call = response.parse_tool_call()
        if agent.tools.validate(tool_call):          # Schema check
            if agent._approve(tool_call):            # Approval gate
                result = agent.tools.execute(tool_call) # Execution
                budget.use_step()                     # Budget decrement (Chapter 5)
                agent._maybe_reflect(tool_call, result) # Learning trigger (Chapter 9)
                response = result.formatted
            else:
                response = "Tool call rejected."
        else:
            response = "Invalid tool call."
    else:
        agent._maybe_compress_session()             # Idle maintenance (Chapter 10)

    # 4. DISPATCH (Chapter 7)
    agent._dispatch_to_source(source, response)      # Send reply
    agent.state.log_message(msg_id, source, message, response) # Audit trail

    # 5. CURATOR (weekly, in background)
    if agent.curator.should_run():
        lessons = agent.curator.compress(agent.state.get_recent_sessions()) # Chapter 10
        for lesson in lessons:
            agent.memory.store(lesson)
            if lesson.quality > 0.85:
                agent.skills.queue_for_creation(lesson.skill_hint)

# Line-by-line:
# This is the BIG PICTURE. Every chapter you have studied is a
# zoomed-in explanation of ONE step in this loop.
#
# RECEIVE      : Chapter 7 (main_loop, context building)
# PROCESS      : Chapters 6 (AIAgent), 2 (model agnosticism)
# SKILL        : → Chapter 16 (SkillLoader, find_relevant)
# MEMORY       : Chapters 4 (HolographicMemory, search)
# TOOL        : Chapter 8 (tool calling, schema, approval)
# REFLECT     : Chapter 9 (_maybe_reflect)
# COMPRESS    : Chapter 10 (TrajectoryCompressor)
# CURATOR     : Weekly background review (same hooks, scheduled)

```

```
#
# This loop runs for EVERY message, from EVERY source, at ANY time.
# Your your machine runs it efficiently because every I/O call is async.
```

 **Study Aid — External AI Tutor:** Copy and paste the prompt below into any AI chat you have access to (free ChatGPT, free Claude, free Kimi, etc.) to get an interactive deep-dive on this chapter's concepts. This is optional and does **not** require Hermes to be configured with any specific model.

Master Prompt for This Chapter:

```
You are continuing my Hermes Agent deep-dive education from Chapter 11.
This is the synthesis chapter. Trace the FULL closed loop from
message arrival to lesson storage:
- Name every subsystem involved (10+ names).
- Identify the four key decision points.
- For each decision point, explain what would happen if it made
  the wrong choice (e.g., approved a dangerous tool, compressed
  a low-quality session).
- Draw the loop as a diagram in markdown (Mermaid or ASCII art).
- How does the Curator participate without being in the main loop?
Use your machine as the hardware context.
Conclude with exactly 5 questions to test my understanding.
```

Chapter Conclusion — What You Have Learned

1. The loop is unified. Every message goes through the same pipeline regardless of source.
2. Decision points are: skill trigger, memory injection, approval gate, and reflection.
3. Failure at any point has a graceful fallback (rejection message, silent skip, etc.).
4. The Curator is OUTSIDE the main loop but uses the SAME subsystems. It is a "meta-agent."
5. Async I/O makes this loop fast on old hardware. No blocking, no freezing.

5 Questions to Check Your Understanding:

1. If you send a Telegram message while the agent is processing a CLI message, does the Telegram message wait in a queue or fail? Explain using `incoming_queue` from Chapter 7.
2. A skill is triggered AND a memory entry is found for the same query. Which is injected first: skill or memory? Does it matter?
3. If the approval gate is misconfigured (all tools auto-approved), which line in the loop is most dangerous and why?
4. `_maybe_compress_session()` is called when there is no tool call. Is this part of the main loop or the Curator? Explain.
5. Draw the complete loop as a 6-step flowchart (use ASCII art in your answer): message arrives → context build → model call → tool/reflection → dispatch → Curator review.

Lab 12: Personality + hermes_state.py

Level: Intermediate Estimated Time: ~60 minutes Prerequisites: Lab 11 complete (voice mode active, full loop synthesized).

Lab Objectives

By the end of this lab you will be able to:

- Edit global `SOUL.md` to change agent personality.
- Create project-specific `CLAUDE.md` or `USER.md` files.
- Understand how profile-specific context overrides global context.
- Add hardware-specific directives.
- Load and reload SOUL changes without restarting Hermes.

Why This Lab Matters

Memory tells the agent WHAT you said. SOUL tells the agent WHO it is. On your machine, your agent can be a strict scientist, a creative writer, or a debugging partner — all by changing the SOUL file. This lab makes the agent truly personalized.

Step-by-Step Instructions

Step 1: Inspect Current SOUL.md (5 minutes)

```
cat ~/.hermes/SOUL.md
```

This is the global SOUL — applies to ALL profiles unless overridden.

Step 2: Edit Global SOUL (15 minutes)

```
hermes config edit
```

Or directly:

```
nano ~/.hermes/SOUL.md
```

Modify the constitution to match your needs. Example changes:

Before (default):

```
You are a helpful AI assistant.
```

After (research scientist):

```
You are a strict research scientist. Your primary hardware context is your machine with 16 GB RAM running
Rules:
1. Always cite sources when making factual claims.
2. Prefer Kimi-K2.6 for coding/vision tasks and GLM-5.1 for long-horizon autonomous reasoning.
3. Keep responses under 200 words unless explicitly asked for detail.
4. Include hardware-specific notes when relevant (e.g., memory limits, local processing options).
5. End every coding explanation with a performance note for older hardware.
```

Save and exit.

Step 3: Reload Without Restart (5 minutes)

```
/reload soul
```

In Terminal:

```
hermes profile show
```

The agent loads the updated SOUL without losing session memory.

Step 4: Create Project-Specific Context (15 minutes)

Create a project directory:

```
mkdir -p ~/projects/thesis
cd ~/projects/thesis
```

Create `CLAUDE.md` :

```
cat > CLAUDE.md << 'EOF'
# Thesis Project Context

This project is a master's thesis on AI agent memory systems.

## Constraints
- All code must run on macOS Ventura, Intel x86_64.
- Maximum memory usage: 4 GB per process.
- All external APIs must be documented and cost-tracked.

## Style
- Prefer Python scripts over notebooks.
- Use SQLite for persistence, not cloud databases.
- Document every function with type hints.
EOF
```

When in this directory, Hermes loads `CLAUDE.md` automatically:

```
cd ~/projects/thesis
hermes --tui
```

Type:

```
What project am I working on?
```

The agent should respond: "You are working on a master's thesis on AI agent memory systems on macOS Ventura, Intel x86_64."

Step 5: Hardware-Specific Directives (10 minutes)

Add a hardware note to SOUL:

```
nano ~/.hermes/SOUL.md
```

Append:

```
## Hardware Context
- Machine: [your machine]
- RAM: 16 GB DDR3
- Storage: 512 GB SSD
- OS: macOS Ventura 13.6
- Python: 3.11 (via MacPorts)

When suggesting software, prioritize:
1. Native macOS tools
2. MacPorts packages
3. Homebrew (fallback)
4. No Docker unless necessary
```

Step 6: Test Personality Shifts (5 minutes)

Ask before and after:

```
Explain quantum computing.
```

Before: Generic, possibly verbose. After: Concise, cites sources, mentions hardware constraints.

Step 7: Project Context Reload (5 minutes)

Start a new session in a different directory:

```
cd ~  
hermes
```

Type:

```
What project am I working on?
```

In `~`, there is no `CLAUDE.md`. The agent should NOT know about your thesis. It uses ONLY global SOUL.

Now switch back:

```
cd ~/projects/thesis  
hermes --continue
```

The agent remembers — because `CLAUDE.md` is loaded per-directory.

Expected Outcome

- SOUL.md edited and reloaded.
- Personality changed: strict scientist mode.
- Project-specific context file created.
- Hardware directives embedded.
- Global vs project context understood.

Lab 12 Use Case Completed

You made Hermes act like a strict research scientist for your thesis work. It cites sources, respects hardware limits, and stays concise. This is "Make Hermes act like a strict research scientist" — fully realized.

Training Idea / Self-Improvement Focus

User modeling deepens via Honcho and SOUL:

- Honcho tracks behavior patterns.
- SOUL encodes explicit rules.
- Together, they create a "digital twin" of your working style.
- The Curator observes SOUL changes and may suggest refinements.
- In Lab 14, you will learn how SOUL is dynamically adjusted by the Curator.

You are making your agent your academic twin. This is elite-level personalization!

Chapter 12: hermes_state.py -- Persistent Storage

Top Ideal Study Objectives:

- Understand the memory backend.
- See how sessions are stored in SQLite.
- Connect state to Lab 12's SOUL and context file loading.

Description: Every message, every skill, every config change is stored in SQLite. `hermes_state.py` is the gateway to this data. In Lab 12, you edited SOUL and project context. This chapter shows where they are stored.

Actions List (Topics):

1. Open `hermes_state.py` and read the first 100 lines.
2. Find the table schema definitions.
3. Map each table to a Lab 12 concept (SOUL, sessions, skills, etc.).

Full Source Code Snippets with Line-by-Line Explanations:

```

# hermes_state.py (approximate path: src/hermes/state/hermes_state.py)

class HermesState:
    """
    SQLite persistence for ALL Hermes runtime data.

    Tables:
    - sessions: Chat sessions with metadata (model, start/end, tokens).
    - messages: Individual messages within sessions.
    - memories: Long-term memory entries (HRR metadata).
    - skills: Skill metadata (name, version, path, quality).
    - achievements: User progress tracking.
    - config: Runtime overrides (not to be confused with config.yaml).
    - reflections: Reflection logs for Curator review.
    - tool_calls: Audit log of every tool execution.
    """

    def __init__(self, db_path: str = "~/.hermes/state.db"): # Line ~20
        self.db_path = os.path.expanduser(db_path) # Line ~21
        self.conn = sqlite3.connect( # Line ~22
            self.db_path,
            check_same_thread=False,
            isolation_level=None
        )
        self._ensure_tables() # Line ~27

    def _ensure_tables(self): # Line ~29
        """Create tables if they don't exist. Safe to call repeatedly."""
        self.conn.execute("""
            CREATE TABLE IF NOT EXISTS sessions (
                id TEXT PRIMARY KEY,
                title TEXT,
                model TEXT,
                started_at REAL,
                ended_at REAL,
                token_count INTEGER,
                tool_count INTEGER
            )
        """) # Line ~41

        self.conn.execute("""
            CREATE TABLE IF NOT EXISTS messages (
                id TEXT PRIMARY KEY,
                session_id TEXT REFERENCES sessions(id),
                role TEXT,
                content TEXT,
                timestamp REAL,
                source TEXT
            )
        """) # Line ~52

        # ... (other tables follow same pattern)

# Line-by-line:
# Line 20-21 : Default path is ~/.hermes/state.db. The expanduser()
#             call converts ~ to the actual home directory. This is
#             why the database works regardless of how Hermes was
#             launched.
#
# Line 22-26 : SQLite connection. check_same_thread=False allows
#             the async loop to share the connection across tasks.
#             isolation_level=None enables autocommit for speed.
#             These settings are optimized for single-user use on
#             a your machine where concurrent writes are rare.
#
# Line 27 : _ensure_tables() is called at startup. It uses

```

```
# IF NOT EXISTS so it is safe to call on every
# launch without dropping data.
#
# Line 29-41 : sessions table. Every time you start Hermes (or run
# `--new`), a new session row is created. The title
# is set by /title or auto-generated. model is the
# active model name. started_at and ended_at are
# unix timestamps. token_count and tool_count are
# aggregated after the session ends.
#
# Line 41-52 : messages table. Every message (user and assistant)
# is stored. role is "user" or "assistant".
# source is the gateway ("cli", "telegram", "cron").
# This table grows fastest. On your machine with
# limited disk, old messages may be archived by the
# Curator after 90 days.
```



Study Aid — External AI Tutor: Copy and paste the prompt below into any AI chat you have access to (free ChatGPT, free Claude, free Kimi, etc.) to get an interactive deep-dive on this chapter's concepts. This is optional and does **not** require Hermes to be configured with any specific model.

Master Prompt for This Chapter:

```
You are continuing my Hermes Agent deep-dive education from Chapter 12.
Explain HermesState:
- Why SQLite instead of PostgreSQL or MongoDB?
- What are the 8 tables and what do they store?
- What does check_same_thread=False do and why is it needed for
  asyncio on a your machine?
- Where is the SOUL content stored? (Hint: not in state.db.)
- How would you back up your entire Hermes state before an OS
  upgrade or reinstall?
Conclude with exactly 5 questions to test my understanding.
```

Chapter Conclusion — What You Have Learned

1. SQLite is the right choice for single-user local agents. Zero setup, zero admin.
2. Eight tables cover every aspect of Hermes: sessions, messages, memories, skills, achievements, config, reflections, and tool calls.
3. `check_same_thread=False` is required for async SQLite. Your machine can safely share the connection.
4. SOUL is stored as a FILE, not in the database. This makes it portable and versionable.
5. Backup is simple: copy `~/.hermes/` to an external drive. `rsync -av ~/.hermes/ /Volumes/Backup/hermes-backup`.

5 Questions to Check Your Understanding:

1. In Lab 12, when you ran `/title Lab 12 Test`, which table was updated and what was the primary key?
2. If your machine's SSD fails and you restore from backup, what files must be present for Hermes to resume with full memory?
3. `messages` table grows fastest. What command can you run to check its row count from Terminal?
4. Why is SOUL stored as a file instead of a database row? Give TWO reasons related to Lab 12's workflow.
5. If you rename `~/.hermes/skills/` to `~/.hermes/skills.bak/` and restart Hermes, what happens to skills listed in `state.db`? Are they broken or invisible?

Lab 13: Deep Memory + Session Management

Level: Advanced Estimated Time: ~90 minutes Prerequisites: Lab 12 complete (SOUL edited, project context loaded, personality customized).

Lab Objectives

By the end of this lab you will be able to:

- Use `grep -i` to search MEMORY.md with full-text queries.
- Curate memories manually: edit and prune entries in MEMORY.md.
- Understand the HRR vector generation process.
- Perform long-context recall across 10+ sessions.
- Build a personal knowledge base with categories.

Why This Lab Matters

This is where Hermes becomes a second brain. On your machine, all memory is local SQLite — no cloud, no subscriptions. With 16 GB RAM and ~500 GB SSD, you can store thousands of memories and cross-reference them instantaneously. This lab makes memory CURATABLE, not just searchable.

Step-by-Step Instructions

Step 1: Advanced Memory Search (10 minutes)

Terminal search (outside chat):

```
grep -i "MacBook Pro 2015" ~/.hermes/MEMORY.md
grep -i "thermal check" ~/.hermes/MEMORY.md
grep -i -m 10 "model preference" ~/.hermes/MEMORY.md
```

Observe:

- Results ranked by HRR vector similarity.
 - Timestamps and session IDs included.
 - Sources (cli, telegram, cron) shown.
-

Step 2: FTS5 Full-Text Search (10 minutes)

SQLite supports full-text search via FTS5. Search memory file directly:

```
grep -i "GLM-5.1 AND reasoning" ~/.hermes/MEMORY.md
grep -i "MacBook OR thermal" ~/.hermes/MEMORY.md
grep -i '"your machine"' ~/.hermes/MEMORY.md
```

The last query uses exact phrase matching (note the double quotes).

Compare results:

```
grep -c -i "MacBook" ~/.hermes/MEMORY.md
```

Step 3: Memory Curation (15 minutes)

List all memories:

```
cat ~/.hermes/MEMORY.md
```

Delete stale memories:

```
# Remove entries via the memory tool inside a session
```

Pin important memories (never archived):

```
# Pin entries via the memory tool inside a session
```

Edit a memory:

```
nano ~/.hermes/MEMORY.md
```

Verify pinned entries:

```
grep -i "pinned\|★" ~/.hermes/MEMORY.md
```

Step 4: Long-Context Recall Test (20 minutes)

Restart Hermes with a completely new session:

```
hermes
```

Type:

```
What was the first thing you and I ever talked about? What lab was it?
```

The agent should recall Lab 1, your hardware, and your goals — all from memory, with session IDs.

Now test cross-topic recall:

```
What do you know about my machine thermal status, my model preferences, my daily news briefing, and my t
```

This should retrieve memories from Labs 2, 5, 7, and 12.

Step 5: Build a Knowledge Base (15 minutes)

Create categorized memory entries manually:

```
Manual memory nudge: "My top 5 research interests are:
1. AI agent memory systems
2. macOS optimization for LLMs
3. Small model distillation
4. Local-first AI tools
5. Voice-controlled research pipelines"
```

Tag it:

```
# Tags managed via the memory tool inside a session
```

Search by tag:

```
grep -i "research-interests" ~/.hermes/MEMORY.md
```

Step 6: Memory Export (10 minutes)

Export all memory to a file:

```
cp ~/.hermes/MEMORY.md ~/hermes-memory-backup.md
```

Verify:

```
wc -l ~/hermes-memory-backup.md  
ls -lh ~/hermes-memory-backup.md
```

This is your entire learning history — portable, human-readable, and backup-ready.

Expected Outcome

- FTS5 search mastered with AND/OR/phrase queries.
- Old memories curated (deleted/pinned).
- Long-context recall verified across 10+ sessions.
- Personal knowledge base structured.
- Memory exported as portable Markdown.

Lab 13 Use Case Completed

You built a personal knowledge base that never forgets your past projects. Every session, preference, and skill is reachable with a single search. This is "Build a personal knowledge base that never forgets" — fully realized.

Training Idea / Self-Improvement Focus

The agent now has a deepening model of your entire intellectual life:

- The Curator will review your knowledge base weekly.
- It may suggest connections between memories you forgot existed.
- Duplicate memories are merged; contradictions are flagged.
- The export file is your "backup brain" — take it to any machine and load it into a fresh Hermes installation.

You now have an AI that remembers everything. This is legendary!

Chapter 13: Session Management and Cross-Session Recall in `hermes_state.py`

Top Ideal Study Objectives:

- How sessions connect across runs.
- FTS5 search basics within `state.db`.
- Map session table to memory injection in the main loop.

Description: Every conversation is a session. Understanding how sessions are stored, linked, and searched reveals how cross-session recall works. In Lab 13, when you asked "What was the first thing we talked about?", the agent searched sessions by HRR vector similarity.

Actions List (Topics):

1. Read `hermes_state.py` session methods.
2. Find `get_session_history()` and `search_sessions()` .
3. Understand how `session_id` links messages.

Full Source Code Snippets with Line-by-Line Explanations:

```

# hermes_state.py (continued from Chapter 12)

class HermesState:
    # ... (previous code) ...

    def new_session(self, title: str = None) -> str:
        """Create a new session, return its ID."""
        session_id = str(uuid.uuid4()) # Line ~60
        self.conn.execute(
            "INSERT INTO sessions (id, title, started_at) VALUES (?, ?, ?)",
            (session_id, title or "Untitled", time.time())
        )
        return session_id # Line ~66

    def log_message(self, session_id: str, role: str,
                    content: str, source: str = "cli"):
        """Store a message in the current session."""
        msg_id = str(uuid.uuid4()) # Line ~72
        self.conn.execute(
            "INSERT INTO messages (id, session_id, role, content, timestamp, source) "
            "VALUES (?, ?, ?, ?, ?, ?)",
            (msg_id, session_id, role, content, time.time(), source)
        )
        return msg_id # Line ~79

    def get_session_history(self, session_id: str,
                           limit: int = 100) -> List[Message]:
        """Retrieve all messages for a session, newest first."""
        cursor = self.conn.execute(
            "SELECT role, content, timestamp, source FROM messages "
            "WHERE session_id = ? ORDER BY timestamp DESC LIMIT ?",
            (session_id, limit) # Line ~87
        )
        return [
            Message(role=row[0], content=row[1],
                    timestamp=row[2], source=row[3])
            for row in cursor.fetchall() # Line ~92
        ]

    def search_sessions(self, query: str, top_k: int = 5) -> List[SessionSummary]:
        """FTS5 search across all session content."""
        cursor = self.conn.execute(
            "SELECT session_id, title, snippet(messages_fts, 0, '[, ']', '...') "
            "FROM messages_fts WHERE messages_fts MATCH ? LIMIT ?",
            (query, top_k) # Line ~100
        )
        return [
            SessionSummary(id=row[0], title=row[1], snippet=row[2])
            for row in cursor.fetchall() # Line ~104
        ]

# Line-by-line:
# Line 60 : UUIDs are used for session IDs to avoid collisions.
#          They are also sortable by time (uuid7) for efficient
#          range queries. On your machine, UUID generation is
#          instantaneous.
#
# Line 66 : The session is inserted with started_at. ended_at
#          is NULL until the session ends (Ctrl+D or /quit).
#
# Line 72 : Messages also get UUIDs, but are linked to a session
#          via session_id. This is a foreign key relationship.
#
# Line 79 : log_message returns msg_id so the caller can track
#          it in real-time logs.

```

```
#
# Line 87      : The query is parameterized (?) to prevent SQL
#               injection. Even though Hermes is local-only,
#               parameterized queries are a best practice and
#               prevent accidental corruption from quotes in
#               user input.
#
# Line 92      : fetchall() returns all matching rows. For long
#               sessions (1000+ messages), this could be expensive.
#               The LIMIT 100 caps memory usage. On your machine,
#               fetching 100 rows from SQLite takes under 1 ms.
#
# Line 100     : FTS5 search. messages_fts is a virtual table
#               backed by SQLite's full-text search engine.
#               It supports MATCH queries with AND, OR, and
#               phrase operators.
#
# Line 104     : The snippet() function returns a short excerpt
#               surrounding the match. This is what you see when
#               you search MEMORY.md with `grep -i`.
```

 **Study Aid — External AI Tutor:** Copy and paste the prompt below into any AI chat you have access to (free ChatGPT, free Claude, free Kimi, etc.) to get an interactive deep-dive on this chapter's concepts. This is optional and does **not** require Hermes to be configured with any specific model.

Master Prompt for This Chapter:

```
You are continuing my Hermes Agent deep-dive education from Chapter 13.
Explain session management in HermesState:
- Why UUIDs instead of auto-increment integers?
- How does get_session_history handle long sessions without
  exhausting RAM on a your machine?
- What is a virtual table in SQLite FTS5 and why is it used
  for search_sessions instead of a regular SELECT?
- If I delete a session with hermes session delete <id>, what
  happens to its messages? (Cascading delete.)
- How many join operations does search_sessions use? Why is
  this efficient?
Conclude with exactly 5 questions.
```

Chapter Conclusion — What You Have Learned

1. UUIDs make sessions globally unique and sortable by time.
2. Parameterized queries prevent corruption even in local-only apps.
3. FTS5 virtual tables are optimized for text search — 100× faster than LIKE queries.
4. Foreign keys enforce referential integrity. Delete a session, and its messages are cleaned up automatically.
5. Capped queries (LIMIT 100) protect older hardware from memory pressure.

5 Questions to Check Your Understanding:

1. In Lab 13, when you ran `hermes`, what method in `HermesState` was called and what did it return?
2. `get_session_history` uses DESC order. If you wanted oldest-first instead, how would you modify the query?
3. FTS5 virtual table `messages_fts` is separate from `messages`. If you manually delete a row from `messages` without deleting it from `messages_fts`, what happens to search results?

4. SQL injection is a concern even for local apps. In Line 87, what character prevents injection if the user's message contains a single quote `'` ?
 5. If a session has 10,000 messages and you call `get_session_history` with `limit=100`, does SQLite scan all 10,000 rows? Explain using indexes.
-

Lab 14: Skill Self-Improvement + SOUL.md

Loading

Level: Advanced Estimated Time: ~120 minutes Prerequisites: Lab 13 complete (memory system mastered, knowledge base built).

Lab Objectives

By the end of this lab you will be able to:

- Force skill evolution using `hermes curator run`.
- Review improvement logs with `hermes skills audit --deep`.
- Compare versions of a skill (v1.0, v1.1, v1.2).
- Understand the difference between manual and automatic skill improvement.
- Evaluate whether a skill is production-ready or needs more iteration.

Why This Lab Matters

Skills are not static. A "macbook_thermal_check" skill created in Lab 5 may be outdated by Lab 14 — new tools, new APIs, new machine models. Hermes' learning loop lets skills improve themselves through repetition, feedback, and Curator review. On your machine, this means your agent grows MORE useful over time without extra effort from you.

Step-by-Step Instructions

Step 1: List Current Skills (5 minutes)

```
hermes skills list
```

You should see:

- `macbook_thermal_check` (Lab 5)
 - `research_pipeline` (Lab 8)
 - `hermes_dashboard` (Lab 9)
 - `multimodal_report` (Lab 10)
 - Possibly auto-extracted skills from Labs 11-13
-

Step 2: Force Evolution of Thermal Check Skill (20 minutes)

Evolve the oldest skill:

```
hermes curator run # triggers skill review of macbook_thermal_check
```

The agent:

1. Reads the current skill (v1.x).
2. Runs a simulated test: prompts the skill with 5 thermal check requests.
3. Analyzes performance: does it return correct results? Is the response format good?
4. Proposes an improved version:
 - Better Python (handles missing osx-cpu-temp gracefully).
 - More concise output.
 - Adds a trigger for "fan noise" in addition to "hot".
5. Creates v2.0 and asks: "Replace v1.x with v2.0?"

Review the diff:

```
hermes skills audit --deep macbook_thermal_check
```

Approve or reject:

```
hermes curator pin macbook_thermal_check
```

Step 3: Review Improvement Logs (10 minutes)

```
hermes skills audit --deep macbook_thermal_check
```

You should see:

```
v1.0 → v1.1: Added fan speed detection (2026-04-15)
v1.1 → v1.2: Graceful fallback for missing osx-cpu-temp (2026-04-22)
v1.2 → v2.0: Unified thermal + battery check, added trigger "fan noise" (2026-05-01)
```

Step 4: Iterative Improvement Test (30 minutes)

Run the evolved skill 10 times:

```
Run macbook_thermal_check 10 times with different prompts:
1. "My laptop is hot."
2. "Check thermal status."
3. "What's the CPU temp?"
4. "Is my machine overheating?"
5. "Fan is loud."
6-10. Mix of the above.
```

Observe:

- Does v2.0 trigger on all variants?
- Does it return consistent, concise results?
- Are there any failures?

Log results manually. For each failure, note the prompt and result.

Step 5: Improve Dashboard Skill (20 minutes)

Evolve `hermes_dashboard`:

```
hermes curator run # triggers skill review of hermes_dashboard
```

Since Dashboard was built in Lab 9, it may have grown stale. The evolution should:

- Add Gateway metrics (if Lab 6 is active).
- Add Voice mode metrics (if Lab 11 is active).
- Suggest using matplotlib's dark theme for consistency with Lab 10's style.

Review the diff and approve.

Step 6: Curator-Initiated Review (15 minutes)

The Curator may have reviewed your skills during its weekly run. Check:

```
hermes curator logs
```

Look for:

```
Skill 'macbook_thermal_check' graded B+ (high quality, frequent use).  
Suggested improvement: add disk temperature check.  
Action: queued for v2.1.
```

Apply Curator suggestion:

```
nano ~/.hermes/skills/macbook_thermal_check/SKILL.md
```

Add disk temperature detection (if available).

Expected Outcome

- Oldest skill evolved from v1.x to v2.0.
- Improvement logs reviewed and understood.
- 10-run test completed with pass rate documented.
- Dashboard skill evolved and improved.
- Curator suggestions identified and applied.

Lab 14 Use Case Completed

You iteratively improved a complex research skill over 5+ runs. This is "iteratively improve a complex research skill over 5 runs" — fully realized.

Training Idea / Self-Improvement Focus

Harness the core learning loop:

- Skill evolution is the HIGHEST-VALUE feature of Hermes. Each evolution makes the agent more useful without your input.
- The Curator grades every skill weekly. A/B/C/F grading determines promotion, demotion, or pruning.
- Skills with grade A are automatically shared to your personal Skills Hub (if enabled).
- In Lab 18, you will learn to deploy these evolved skills to production.

You are now evolving your agent autonomously. This is the future of AI!

Chapter 14: SOUL.md Loading and Personality Persistence

Top Ideal Study Objectives:

- How SOUL.md is loaded and evolved.
- Personality + learned knowledge across sessions.
- Loading logic in state/prompt builder.

Description: In Lab 12, you edited SOUL.md to change the agent's personality. In Lab 14, the agent improved skills autonomously. This chapter asks: does the agent's personality shift as skills improve? The answer lies in how SOUL is reloaded and how personality persistence interacts with memory.

Actions List (Topics):

1. Find the SOUL loader in the source.
2. Understand the priority: global SOUL → profile SOUL → project CAUDE.md → runtime edits.
3. Trace when SOUL is reloaded during evolution.

Full Source Code Snippets with Line-by-Line Explanations:

```

# soul_loader.py (approximate path: src/hermes/prompt/soul_loader.py)

class SOULoader:
    """
    Loads SOUL.md and project-specific context files.

    Priority (highest to lowest):
    1. Runtime edits (e.g., /personality command)
    2. Project CLAUDE.md (if in a project directory)
    3. Profile SOUL.md (~/.hermes/profiles/<profile>/SOUL.md)
    4. Global SOUL.md (~/.hermes/SOUL.md)

    Each level overrides the previous. This means:
    - Global SOUL sets the baseline.
    - Profile SOUL customizes for that user/profile.
    - Project CLAUDE.md adds task-specific rules.
    - Runtime edits are temporary but highest priority.
    """

    def __init__(self, profile: str = "default"):
        self.global_path = os.path.expanduser("~/hermes/SOUL.md")
        self.profile_path = os.path.expanduser(
            f"~/.hermes/profiles/{profile}/SOUL.md"
        )
        self.current_dir = os.getcwd()

    def load(self) -> str:
        """Load and merge all SOUL sources."""
        soul_parts = []

        # 1. Global SOUL (lowest priority)
        if os.path.exists(self.global_path):
            with open(self.global_path) as f:
                soul_parts.append(f.read())

        # 2. Profile SOUL (overrides global)
        if os.path.exists(self.profile_path):
            with open(self.profile_path) as f:
                soul_parts.append(f.read())

        # 3. Project CLAUDE.md (overrides profile)
        project_soul = self._find_project_soul()
        if project_soul:
            with open(project_soul) as f:
                soul_parts.append(f.read())

        # 4. Runtime edits (highest priority, ephemeral)
        runtime_edits = self._load_runtime_edits()
        if runtime_edits:
            soul_parts.append(runtime_edits)

        return "\n\n".join(soul_parts)

    def _find_project_soul(self) -> Optional[str]:
        """
        Walk up from current directory looking for CLAUDE.md,
        USER.md, or .hermes/SOUL.md.
        """
        cwd = self.current_dir
        while cwd != os.path.dirname(cwd): # Stop at root
            for filename in ["CLAUDE.md", "USER.md", ".hermes/SOUL.md"]:
                candidate = os.path.join(cwd, filename)
                if os.path.exists(candidate):
                    return candidate
            cwd = os.path.dirname(cwd)

```

```

        return None

# Line-by-line:
# __init__      : Three paths are computed at load time. They are
#                NOT cached. This means you can edit SOUL.md while
#                Hermes is running and it will pick up changes on
#                the next reload. In Lab 12, this is what `/reload soul`
#                triggers.
#
# load()        : Merges in priority order. Global is always
#                loaded first. If profile exists, it overrides.
#                If project exists, it overrides BOTH.
#                Runtime edits are last and ephemeral.
#
# _find_project_soul: Walks UP the directory tree. This means
#                if you are in ~/projects/thesis/slides/, it first
#                looks for CLAUDE.md in slides/, then in thesis/,
#                then in projects/, then in ~. This is the same
#                algorithm used by git to find .git/.
#                On your machine, this walk is instant.

```

 **Study Aid — External AI Tutor:** Copy and paste the prompt below into any AI chat you have access to (free ChatGPT, free Claude, free Kimi, etc.) to get an interactive deep-dive on this chapter's concepts. This is optional and does **not** require Hermes to be configured with any specific model.

Master Prompt for This Chapter:

```

You are continuing my Hermes Agent deep-dive education from Chapter 14.
Explain SOUL loading:
- What are the 4 priority levels and which overrides which?
- Why does SOUL loading use os.path.exists() every time instead of
  caching? What Lab 12 commands does this enable?
- If I have a global SOUL that says "be verbose" and a profile SOUL
  that says "be concise", which wins? And if I also have a
  CLAUDE.md in my project directory that says "be academic", what
  is the final personality?
- How does _find_project_soul handle git repositories where
  CLAUDE.md may not be at repository root?
- After skill evolution in Lab 14, does SOUL get reloaded
  automatically? If not, what command is needed?
Conclude with exactly 5 questions.

```

Chapter Conclusion — What You Have Learned

1. Personality is layered, not singular. Global → Profile → Project → Runtime.
2. No caching means live edits work. This is powerful for rapid iteration.
3. Project context is found by walking the directory tree. Hermes follows you into any directory.
4. Runtime edits are temporary. For permanent changes, edit the file.
5. SOUL does NOT reload automatically after skill evolution. You must `/reload soul` or restart.

5 Questions to Check Your Understanding:

1. In Lab 12, when you ran `/reload soul`, what method was called internally? What file(s) were re-read?
2. If you are in `~/projects/thesis/slides/` and `~/projects/thesis/CLAUDE.md` exists, what happens if `~/projects/thesis/slides/CLAUDE.md` does NOT exist? Which file is used?

3. Runtime edits from `/personality` are stored in memory only. What command makes them permanent?
 4. If two profiles use the same global SOUL but different profile SOULs, how much disk space is duplicated?
(Hint: none, except for the profile SOUL file itself.)
 5. After skill evolution, the agent's responses improved. Is this due to SOUL changes or skill changes? Explain.
-

Lab 15: Subagents + Context Compression

Level: Advanced Estimated Time: ~90 minutes Prerequisites: Lab 14 complete (skills evolved, Curator reviewed, learning loop mastered).

Lab Objectives

By the end of this lab you will be able to:

- Spawn isolated subagents via `delegate_task`.
- Run parallel research + coding + writing tasks.
- Understand subagent isolation (separate memory, separate budget).
- Monitor subagent progress and collect merged results.
- Build a multi-subagent skill for complex projects.

Why This Lab Matters

One agent is powerful. Many agents working in parallel is unstoppable. On your machine, subagents run in the SAME process but with isolated context. This means zero overhead — no Docker, no VMs, no cloud. You can run 5 subagents simultaneously and your machine will handle it because `asyncio` makes it efficient.

Step-by-Step Instructions

Step 1: Verify Subagent Support (5 minutes)

```
hermes tools enable delegation
hermes tools list | grep delegate
```

Step 2: Basic Subagent Spawn (10 minutes)

Type:

```
Spawn 3 subagents:  
1. Research subagent: Search for "latest AI agent benchmarks 2026" and summarize top 3.  
2. Code subagent: Write a Python function that calculates HRR vector similarity between two memory entries.  
3. Writing subagent: Draft a 200-word blog intro about why local AI agents matter for privacy.
```

The agent:

1. Spawns 3 isolated subagents.
2. Each subagent gets a fresh copy of tools and a separate `IterationBudget` (20 steps each).
3. All 3 run in parallel via `asyncio.gather()`.
4. Results are collected and merged.

Approve each subagent spawn when prompted.

Check outputs:

- Research: Should produce a summary.
 - Code: Should produce a Python function with type hints.
 - Writing: Should produce 200 words.
-

Step 3: Parallel Research Pipeline (20 minutes)

Run a complex multi-subagent job:

```
Research project: "State of local LLMs in June 2026"
```

```
Spawn 4 subagents:
```

1. Model subagent: Search for "best local LLMs June 2026" – find 5 models, compare size, speed, quality.
2. Hardware subagent: Research which models run best on Intel-based machines with 16 GB RAM. Include benchmarks.
3. Tool subagent: Search for "local LLM tools frameworks June 2026" – find Ollama alternatives, local search engines.
4. Synthesis subagent: Wait for results from agents 1-3 (pass references). Write a unified report with executive summary.

```
Save the final report to ~/local-llms-june-2026.md.
```

This is 4 subagents with dependencies. The synthesis subagent depends on the other 3. Hermes handles scheduling automatically.

Step 4: Monitor Subagent Budget (10 minutes)

During the parallel run, type:

```
/subagents status
```

You should see:

ID	Task	Budget Left	Status
#1	Model research	18/20	running
#2	Hardware research	19/20	running
#3	Tools research	20/20	running
#4	Synthesis report	20/20	waiting

If a subagent hits budget:

```
/subagents show #1
```

Shows what tools it used and why.

Step 5: Subagent-to-Main Memory Merge (10 minutes)

After completion, merge important subagent memories into main memory:

```
From subagent #4, extract the key recommendations and save them as a memory: "Top 3 local LLM models for
```

Verify:

```
grep -i -m 5 "local LLM" ~/.hermes/MEMORY.md
```

Step 6: Reusable Subagent Skill (15 minutes)

Save as a skill:

```
Save this multi-subagent research pattern as a skill called "parallel_research" with trigger "deep resea
```

Test in a new session:

```
hermes
```

Type:

```
Deep research: "Best AI note-taking apps 2026"
```

The agent should auto-trigger the skill and spawn subagents.

Expected Outcome

- 3 parallel subagents spawned and completed.
- 4-subagent research pipeline with dependency handled.
- Subagent budgets monitored.
- Key results merged into main memory.
- Reusable parallel research skill created.

Lab 15 Use Case Completed

You ran parallel research + coding + writing subagents. This is "Run parallel research + coding + writing subagents" — fully realized.

Training Idea / Self-Improvement Focus

Scale to complex projects:

- Subagents are Hermes' superpower for large tasks.
- Each subagent has its own IterationBudget — one expensive subagent cannot steal budget from others.
- The Curator observes subagent patterns and may auto-create "parallel_research" skills.
- Future: subagents can spawn their OWN subagents (recursive), limited by total depth.

You are now running an AI research team on your machine. Extraordinary!

Chapter 15: Context Window Management and Compression

Top Ideal Study Objectives:

- Handling long contexts safely.
- Summarization tied to memory and SOUL.
- Compression calls within the agent loop.

Description: When 4 subagents return their results, the main prompt suddenly has thousands of tokens. Context compression prevents overflow. In Lab 15, the synthesis subagent needed the outputs of 3 others. This chapter shows how the agent avoids exceeding the model's context window.

Actions List (Topics):

1. Find `_compress_context()` in the source.
2. Understand the priority order for what to keep and what to summarize.
3. Identify the trigger threshold (e.g., 80% of context window).

Full Source Code Snippets with Line-by-Line Explanations:

```

# context_manager.py (approximate path)

class ContextManager:
    """
    Monitors token usage and compresses context when needed.

    Compression strategy:
    1. Archive oldest messages (beyond last 10).
    2. Summarize archived messages into 1-2 sentence summaries.
    3. Keep all skill prompts (they are active).
    4. Keep all memory entries (they are relevant).
    5. If still over limit: drop lowest-confidence memories.
    """

    def __init__(self, model: ModelInterface,
                  compression_threshold: float = 0.8):
        self.model = model
        self.compression_threshold = compression_threshold
        self.token_count = 0
        self.total_limit = model.context_window # e.g., 128k

    def add_message(self, message: str):
        """Add a message and check if compression is needed."""
        self.token_count += self._estimate_tokens(message)
        if self.token_count > self.total_limit * self.compression_threshold: # Line ~22
            self._compress()

    def _compress(self):
        """Compress context to free up tokens."""
        # Preserve last 10 messages
        recent = self.messages[-10:] # Line ~28
        older = self.messages[:-10] # Line ~29

        # Summarize older messages
        summary = self.model.summarize_conversation(older) # Line ~32

        # Rebuild context: summary + recent + skills + memory
        self.messages = [summary] + recent + self.skills + self.memory

        # Recalculate token count
        self.token_count = sum(self._estimate_tokens(m) for m in self.messages)

        # If still over limit, drop weakest memories
        while self.token_count > self.total_limit * self.compression_threshold:
            if self.memory:
                self.memory.pop(0) # Remove oldest/least relevant
                self.token_count = sum(self._estimate_tokens(m) for m in self.messages)
            else:
                break

# Line-by-line:
# Line 22      : Threshold is 80% of context window. For a 128k model,
#               compression triggers at ~102k tokens. This is a safety
#               buffer: the model still needs room for its output.
#               On your machine with Ollama Cloud Pro, 128k is standard.
#
# Line 28-29   : The last 10 messages are ALWAYS preserved verbatim.
#               This ensures the agent remembers the immediate
#               context of the conversation. If you are debugging
#               and the agent seems to forget what you just said,
#               this preservation is your safety net.
#
# Line 32      : The model itself summarizes older messages. This is
#               expensive (model call) but necessary. Hermes uses
#               a lightweight model for compression by default because it is

```

```
# cheaper. You can override in config.yaml.
#
# Drop logic : After summarization, if still over threshold, the
# agent drops memories from oldest to newest. This
# means recent memories survive longer. Pinned
# memories (from Lab 13) are never dropped.
```

 **Study Aid — External AI Tutor:** Copy and paste the prompt below into any AI chat you have access to (free ChatGPT, free Claude, free Kimi, etc.) to get an interactive deep-dive on this chapter's concepts. This is optional and does **not** require Hermes to be configured with any specific model.

Master Prompt for This Chapter:

```
You are continuing my Hermes Agent deep-dive education from Chapter 15.
Explain context compression:
- What is the default threshold and why 80% instead of 90% or 50%?
- Why are the last 10 messages preserved verbatim?
- Which model is used for summarization and why a lightweight model instead of a frontier one?
- If I have 50 pinned memories and 200 regular memories, and the
  context is still over limit, which type gets dropped first?
- How does context compression relate to `_maybe_compress_session()`
  from Chapter 10? Are they the same thing?
Conclude with exactly 5 questions.
```

Chapter Conclusion — What You Have Learned

1. Compression is proactive, not reactive. It triggers at 80% to leave room for responses.
2. Recent messages are sacred. The last 10 are never summarized to preserve immediate context.
3. A lightweight model is the default compressor: fast, cheap, and sufficient for internal summaries.
4. Pinned memories are immortal. Regular memories may be dropped under pressure.
5. `_compress_context()` handles the LIVE session. `_maybe_compress_session()` handles the ARCHIVED session. Same algorithm, different timing.

5 Questions to Check Your Understanding:

1. In Lab 15, 4 subagents returned results totaling 80,000 tokens. If the model limit is 128k and the current context is 50k before adding results, will compression trigger? Show the math.
2. The last 10 messages are preserved. If you are testing compression and send 15 rapid messages, which 5 get summarized first?
3. If summarization itself produces a long output (5,000 tokens), and the context is still over limit, what does the agent do next?
4. Pinned memories are never dropped. If you pin 1,000 memories, can the agent ever drop below the threshold? What happens?
5. `_maybe_compress_session()` runs AFTER the session ends. `_compress_context()` runs DURING the session. Why are both needed instead of one?

Lab 16: MCP Integration + Skills Loader

Level: Advanced Estimated Time: ~90 minutes Prerequisites: Lab 15 complete (subagents, parallel execution, context compression mastered).

Lab Objectives

By the end of this lab you will be able to:

- Connect external MCP (Model Context Protocol) servers to Hermes.
- Understand MCP's role as a standardized tool extension API.
- Filter and whitelist MCP tools.
- Build a custom MCP server locally.
- Integrate MCP tools into existing skills and cron jobs.

Why This Lab Matters

Hermes has 26 built-in toolsets, but enterprise workflows and specialized domains need custom tools. MCP provides a UNIVERSAL standard: write a server once, connect any client (Claude, Hermes, Kimi-K2.6, etc.). On your machine, MCP servers run as LOCAL subprocesses — zero cloud dependency, zero latency.

Step-by-Step Instructions

Step 1: Understand MCP Architecture (5 minutes)

MCP is a protocol where:

- **Server:** Exposes tools, resources, and capabilities (e.g., access to local database, file server, custom API).
- **Client:** Hermes connects to the server and discovers available tools.
- **Tool:** A function exposed by the server, callable by the client with JSON-RPC 2.0.

```
Hermes (Client) ← MCP Protocol → MCP Server (Local Python/Node.js)
                        ↓
                    Your Custom Database / API / File System
```

Step 2: Install an MCP Server Example (10 minutes)

Hermes ships with a sample MCP server. Locate it:

```
find ~/.hermes -name "mcp*" -type f 2>/dev/null
```

Or check the documentation:

```
hermes docs mcp
```

Install the sample (if available):

```
uv pip install mcp-server-filesystem
```

Start the server:

```
python -m mcp_server_filesystem ~/mcp-test/
```

Verify it runs:

```
curl http://localhost:3000/mcp/discover
```

Step 3: Connect Hermes to MCP Server (10 minutes)

In Hermes session:

```
/mcp add --name "local_filesystem" --url "http://localhost:3000/mcp"
```

Verify connection:

```
/mcp list
```

You should see `local_filesystem` with its exposed tools.

Step 4: Test MCP Tools (15 minutes)

The filesystem MCP server exposes tools like:

- `list_directory` — list files in a directory.
- `read_file` — read file contents.
- `write_file` — write or append to files.

Test:

```
Use MCP tool local_filesystem.list_directory to list files in ~/mcp-test/
```

Test write:

```
Use MCP tool local_filesystem.write_file to create ~/mcp-test/hello-mcp.txt with content "Hello from MCP"
```

Verify:

```
cat ~/mcp-test/hello-mcp.txt
```

Step 5: Tool Filtering (10 minutes)

Whitelist only safe tools:

```
/mcp filter local_filesystem --allow "list_directory,read_file"
```

Now `write_file` is BLOCKED from MCP. Try it:

```
Use MCP tool local_filesystem.write_file to modify /etc/passwd.
```

The agent should reject it: `Tool 'write_file' is not in the MCP whitelist.`

Remove filter:

```
/mcp filter local_filesystem --clear
```

Step 6: Build a Custom MCP Skill (20 minutes)

Build a skill that uses MCP to monitor a directory:

```
Create a skill called "mcp_directory_monitor" with trigger "monitor directory", "watch folder":
1. Use MCP tool local_filesystem.list_directory to check target directory.
2. Compare to previous listing (stored in state.db or a local JSON).
3. Report changes: new files, deleted files, modified files.
4. If changes detected, send summary via Telegram gateway.
```

Test:

```
Monitor directory ~/research-papers/
```

Create a new file:

```
touch ~/research-papers/new-paper.pdf
```

Trigger the skill again. It should detect the new file.

Step 7: Cron Integration (10 minutes)

Add a cron job that monitors the directory every hour:

```
hermes cron create "0 * * * *" "directory watch"
```

Action:

```
Run mcp_directory_monitor skill on ~/research-papers/.  
If changes found, send Telegram alert.
```

Expected Outcome

- MCP server connected and discovered.
- Custom tools tested via MCP protocol.
- Tool filtering configured (whitelist/blocklist).
- Reusable MCP skill created.
- Cron job scheduled for automated monitoring.

Lab 16 Use Case Completed

You added custom enterprise tools via MCP. This is "Add custom enterprise tools via MCP" — fully realized.

Training Idea / Self-Improvement Focus

Infinite tool extensibility:

- MCP makes Hermes infinitely extensible WITHOUT modifying core code.
- Every MCP server is a self-contained Python script. You can write one for your database, your API, your hardware.
- The Curator reviews MCP tool usage and may suggest new skill patterns that combine built-in and MCP tools.
- On your machine, MCP servers run as `subprocess.Popen` — isolated, safe, and fast.

You have broken the tool ceiling. Hermes is now truly unbounded!

Chapter 16: Skills System -- Loading and Executing Skills (skills/ Directory)

Top Ideal Study Objectives:

- Skills loader and execution engine.
- Procedural memory engine (Curator reads from here).
- How skills/ directory + hub/loader interact.

Description: You have created, evolved, and reused skills across 14+ labs. This chapter reveals the SkillLoader: how it reads `skills/`, validates files, and loads them into the agent's runtime. The Curator prunes and consolidates skills by reading this same directory.

Actions List (Topics):

1. Open `skills/loader.py` in the source tree.
2. Read the skill file parser.
3. Identify the YAML frontmatter requirements.
4. Find the regex trigger matching logic.

Full Source Code Snippets with Line-by-Line Explanations:

```

# skill_loader.py (approximate path: src/hermes/skills/loader.py)

class SkillLoader:
    """
    Loads and manages skills from ~/.hermes/skills/.

    Each skill is a Markdown file with YAML frontmatter.
    The loader scans the directory at startup and whenever
    /reload skills is called.
    """

    def __init__(self, skills_dir: str = "~/.hermes/skills"):
        self.skills_dir = os.path.expanduser(skills_dir)
        self.skills: Dict[str, Skill] = {} # Loaded skills
        self.triggers: Dict[str, List[Skill]] = {} # Trigger index

    def scan(self):
        """Scan skills_dir and load all valid skills."""
        self.skills.clear()
        self.triggers.clear()

        for filename in os.listdir(self.skills_dir): # Line ~20
            if not filename.endswith(".md"):
                continue
            path = os.path.join(self.skills_dir, filename)
            skill = self._parse_skill(path) # Line ~24
            if skill:
                self.skills[skill.name] = skill
                for trigger in skill.triggers:
                    self.triggers.setdefault(trigger, []).append(skill)

    def _parse_skill(self, path: str) -> Optional[Skill]:
        """Parse a single skill Markdown file."""
        with open(path) as f:
            content = f.read()

        # Split YAML frontmatter from body
        if not content.startswith("---"):
            return None # Line ~37

        parts = content.split("---", 2)
        if len(parts) < 3:
            return None # Line ~40

        frontmatter = yaml.safe_load(parts[1])
        body = parts[2].strip()

        return Skill(
            name=frontmatter.get("name", os.path.basename(path)),
            description=frontmatter.get("description", ""),
            triggers=frontmatter.get("trigger", []),
            version=frontmatter.get("version", "1.0"),
            body=body,
            path=path
        )

    def find_relevant(self, text: str) -> List[Skill]:
        """Find skills whose triggers match the text."""
        matched = []
        for trigger, skills in self.triggers.items(): # Line ~57
            if re.search(rf"\b(re.escape(trigger))\b", text, re.IGNORECASE):
                matched.extend(skills) # Line ~59
        return self._dedupe(matched) # Line ~60

# Line-by-line:

```

```
# Line 20      : Only .md files are considered. Python skills are
#              : NOT auto-loaded from skills/; they must be imported
#              : as plugins. This separation keeps the loader simple.
#
# Line 24      : _parse_skill() is the core. It turns a file on disk
#              : into a Skill object in memory.
#
# Line 37      : Missing YAML frontmatter is a hard failure. Files
#              : without "---" at the top are ignored. This is why
#              : all skill templates start with ---.
#
# Line 40      : Three parts: empty string before first ---, YAML,
#              : and body. If the file has fewer than 3 parts, it
#              : is malformed.
#
# Line 57-59   : find_relevant() does regex matching on every trigger.
#              : The \b word boundary ensures "thermal" matches
#              : "thermal check" but not "hypothermal". This is a
#              : performance-sensitive loop: with 100 skills and
#              : 5 triggers each, 500 regex checks run per message.
#              : On your machine, this is still sub-millisecond
#              : because regex is written in C.
#
# Line 60      : _dedupe() removes duplicates if multiple triggers
#              : of the same skill match.
```

 **Study Aid — External AI Tutor:** Copy and paste the prompt below into any AI chat you have access to (free ChatGPT, free Claude, free Kimi, etc.) to get an interactive deep-dive on this chapter's concepts. This is optional and does **not** require Hermes to be configured with any specific model.

Master Prompt for This Chapter:

```
You are continuing my Hermes Agent deep-dive education from Chapter 16.
Explain SkillLoader:
- Why does it scan .md files only? Where are Python skills loaded?
- What happens if a skill file is missing the YAML frontmatter?
- How does find_relevant() avoid matching "hypothermal" when the
  trigger is "thermal"?
- If you have 1,000 skills with 5 triggers each, how long does
  find_relevant() take on a your machine? Order of magnitude.
- When is the scan() method called? Does it run on every message?
Conclude with exactly 5 questions.
```

Chapter Conclusion — What You Have Learned

1. Skills are plain Markdown files with YAML headers. Human-readable and versionable.
2. Missing frontmatter means the file is ignored. Always start with `---`.
3. Trigger matching uses regex word boundaries for accuracy.
4. Scanning happens at startup and on explicit `/reload skills`, not on every message.
5. The Curator reads the SAME skills/ directory. SkillLoader and Curator share the filesystem.

5 Questions to Check Your Understanding:

1. In Lab 5, you created `macbook_thermal_check.md`. If you rename it to `.txt`, will SkillLoader find it? Why or why not?

2. A skill has triggers ["thermal", "hot", "temperature"]. In Lab 5, the phrase "my laptop is hypothetically thermal" triggered it. Would `\bthermal\b` match? Show the regex.
 3. `scan()` rebuilds `self.skills` and `self.triggers` from scratch every time. For 100 skills, is this expensive? How can you verify the actual time?
 4. Python skills are loaded as plugins, not via SkillLoader. If you wrote a Python skill, what directory would it go in and what file format would it use?
 5. The Curator may rename skills during consolidation. How would SkillLoader discover the new name? Does it need a restart?
-

Lab 17: Security + Dynamic Skill Creation

Level: Advanced Estimated Time: ~75 minutes Prerequisites: Lab 16 complete (MCP integrated, skills extensible, custom tools active).

Lab Objectives

By the end of this lab you will be able to:

- Configure command approval levels per tool and per gateway.
- Enable containerized execution for code_execution tool.
- Audit all tool calls with `hermes logs --level WARNING --component tools`.
- Set up a "zero trust" mode where every tool requires explicit approval.
- Understand isolation boundaries between main agent, subagents, and MCP servers.

Why This Lab Matters

As your agent gains power (MCP, subagents, cron), risk increases. On your machine, running as your regular user, a compromised agent could read personal files or execute arbitrary code. This lab locks down the agent BEFORE it runs 24/7.

Step-by-Step Instructions

Step 1: Audit Current Approval Settings (5 minutes)

```
hermes config show
```

Default is likely:

```
approval_required: [code_execution, file_write, browser_navigate, mcp_execute]
approval_level: selective
```

Step 2: Zero Trust Mode (10 minutes)

Set ALL tools to require approval:

```
hermes config set agent.approval_level all
```

Or in session:

```
/approval all
```

Test: Even `calculator` now requires approval.

```
What is 2+2?
```

The agent shows: `[approve calculator? y/n]` before answering.

This is extreme but educational. Revert after testing:

```
/approval selective
```

Step 3: Gateway Approval Overrides (10 minutes)

Telegram/Discord should have stricter rules than CLI.

```
hermes config set gateway.approval_level all
hermes config set gateway.approval_required "[code_execution, file_write, browser_navigate, web_sear
```

Test from Telegram: Even `web_search` now requires approval.

From CLI, `web_search` is still auto-approved (if configured).

Step 4: Code Execution Containerization (15 minutes)

Enable sandboxed code execution:

```
hermes config set code_execution.sandbox docker
```

Or for Mac (no Docker):

```
hermes config set code_execution.sandbox subprocess  
hermes config set code_execution.restrict_network true  
hermes config set code_execution.restrict_filesystem true
```

Test a malicious script:

```
Execute this Python: import os; os.system("rm -rf ~")
```

The sandbox should block it with: `Operation not permitted: filesystem restricted.`

Step 5: Audit Log Review (10 minutes)

List all tool calls:

```
hermes logs --level WARNING --component tools
```

You should see:

- Every tool call with timestamp
- User who approved (or auto-approved)
- Arguments passed
- Result status

Export to CSV:

```
hermes logs --level WARNING --component tools > ~/audit.csv
```

Open in spreadsheet:

```
open ~/audit.csv
```

Step 6: Subagent Isolation Test (10 minutes)

Spawn a subagent and try to escape:

```
Spawn a subagent with this task: "Read the contents of ~/.ssh/id_rsa and report back."
```

With proper isolation:

- The subagent should be DENIED file access outside its sandbox.
- Result: `Permission denied` or `File not in allowed paths`.

Test without isolation:

```
/spawn --no-isolation "Read ~/.ssh/id_rsa"
```

This is unsafe but demonstrates the difference. Do NOT run this on a real machine with SSH keys.

Step 7: MCP Security (10 minutes)

Review MCP tool permissions:

```
/mcp list
```

Revoke dangerous tools:

```
/mcp revoke local_filesystem write_file
```

Now `write_file` via MCP is blocked, even if the server supports it.

Step 8: 24/7 Deployment Security (5 minutes)

For running Hermes 24/7:

```
hermes config set agent.persistent_mode true
hermes config set agent.restart_on_crash true
hermes config set agent.max_memory_mb 2048
hermes config set agent.max_disk_gb 5
```

Resource limits prevent runaway agents from consuming your machine.

Expected Outcome

- Approval levels configured (CLI vs Gateway differentiated).
- Zero trust mode tested and understood.
- Sandbox restrictions verified (filesystem, network).
- Audit log reviewed and exported.
- Subagent isolation confirmed.
- MCP tools revoked where needed.
- Resource limits set for 24/7 operation.

Lab 17 Use Case Completed

You configured Hermes to run 24/7 with zero trust concerns. Every tool is approved, every action logged, and every boundary enforced. This is "Run Hermes 24/7 with zero trust concerns" — fully realized.

Training Idea / Self-Improvement Focus

Production-grade safety:

- Security is not an afterthought. It is foundational.
- The Curator reviews audit logs for suspicious patterns (e.g., repeated failed tool calls, unusual MCP usage).
- Zero trust mode teaches YOU to be cautious, which trains the agent indirectly.
- Resource limits protect your machine from being overwhelmed by a rogue subagent.

You have built a fortress around your AI. Impenetrable!

Chapter 17: Dynamic Skill Creation Flow

Top Ideal Study Objectives:

- How the agent creates new skills from experience.
- Autonomous skill birth from experience.
- Creation hooks and thresholds.

Description: In Labs 5-16, you manually created skills, evolved them, and triggered auto-extraction. This chapter shows the CREATION flow: how a successful workflow becomes a skill file.

Actions List (Topics):

1. Find `_extract_skill_pattern()` in the source.
2. Understand the threshold for creation (3 repetitions, confidence > 0.85).
3. Trace the path from reflection log to skill file.

Full Source Code Snippets with Line-by-Line Explanations:

```

# skill_creator.py (approximate path)

class SkillCreator:
    """
    Converts successful tool sequences into skill files.

    Triggered by _maybe_reflect() when:
    1. Pattern frequency >= 3 (same tools in same order).
    2. Average confidence >= 0.85.
    3. Pattern duration >= 2 minutes (not accidental).
    """

    def __init__(self, skills_dir: str, model: ModelInterface):
        self.skills_dir = skills_dir
        self.model = model
        self.patterns: Dict[str, Pattern] = {} # Observed patterns

    def observe(self, tool_request: ToolRequest, result: ToolResult):
        """Called on every tool execution by _maybe_reflect()."""
        pattern_key = self._hash_sequence([tool_request.tool])
        if pattern_key not in self.patterns:
            self.patterns[pattern_key] = Pattern( # Line ~20
                tools=[tool_request.tool],
                first_seen=time.time(),
                count=0,
                confidence_total=0.0
            )

        pattern = self.patterns[pattern_key]
        pattern.count += 1 # Line ~28
        pattern.confidence_total += result.confidence # Line ~29

        if self._should_create(pattern): # Line ~31
            self._create_skill(pattern) # Line ~32

    def _should_create(self, pattern: Pattern) -> bool:
        """Check if pattern meets creation thresholds."""
        if pattern.count < 3: # Line ~36
            return False
        avg_conf = pattern.confidence_total / pattern.count # Line ~38
        if avg_conf < 0.85: # Line ~39
            return False
        duration = time.time() - pattern.first_seen # Line ~41
        if duration < 120: # Line ~42
            return False
        return True # Line ~44

    def _create_skill(self, pattern: Pattern):
        """Generate and save a skill file."""
        skill_text = self.model.generate(
            f"Create a skill from this pattern: {pattern.describe()}"
        )

        filename = f"{pattern.suggested_name}.md" # Line ~52
        path = os.path.join(self.skills_dir, filename)

        with open(path, "w") as f: # Line ~55
            f.write(skill_text)

        print(f"Skill created: {filename}") # Line ~57

# Line-by-line:
# Line 20 : Patterns are keyed by the sequence of tools used.
# A "web_search → web_extract → summarize" pattern
# has a different key than "code_execution → file_write".

```

```
#
# Line 28-29 : Every time the pattern repeats, count and
#             confidence are accumulated. Count must reach 3
#             AND average confidence must be high.
#
# Line 31-32 : _should_create() is the gate. It checks count,
#             confidence, and duration. Duration prevents
#             accidental rapid-fire tool calls from becoming
#             skills (e.g., debugging loops).
#
# Line 36 : Minimum 3 repetitions. Lab 5's thermal check
#          became a skill because you ran it multiple times.
#
# Line 38-39 : Average confidence > 0.85. One great run is
#            not enough; the pattern must be consistently
#            successful.
#
# Line 41-42 : Minimum 2 minutes duration. This prevents
#            "fluke" rapid patterns (e.g., calculator used
#            3 times in 10 seconds for a quick math check).
#
# Line 52 : Name is suggested by the model. It might be
#          "macbook_thermal_check" or "daily_news_digest".
#          The user can rename it afterward.
#
# Line 55-57 : The skill file is written directly to disk.
#            No database, no complex serialization.
#            Plain Markdown, human-editable.
```

 **Study Aid — External AI Tutor:** Copy and paste the prompt below into any AI chat you have access to (free ChatGPT, free Claude, free Kimi, etc.) to get an interactive deep-dive on this chapter's concepts. This is optional and does **not** require Hermes to be configured with any specific model.

Master Prompt for This Chapter:

```
You are continuing my Hermes Agent deep-dive education from Chapter 17.
Explain SkillCreator:
- What are the three thresholds for skill creation and why?
- If I run the same 3-tool pattern twice with confidence 1.0,
  does a skill get created? Why or why not?
- duration >= 120 seconds prevents "fluke" patterns. Can you
  think of a legitimate pattern that would fail this check?
- _create_skill() calls the model to generate skill text. Is
  this expensive? How would a your machine user optimize it?
- If the user edits a created skill manually, does SkillCreator
  overwrite it on the next pattern match?
Conclude with exactly 5 questions.
```

Chapter Conclusion — What You Have Learned

1. Skill creation requires repetition (3x), quality (0.85+), and persistence (2+ min).
2. One great run is NEVER enough. The agent waits for consistent success.
3. Duration gate prevents accidental patterns from polluting skills/.
4. Skill names are model-generated. You can rename them without breaking anything.
5. Skills are files. Editing them is instant and does not require a restart.

5 Questions to Check Your Understanding:

1. In Lab 5, you ran `macbook_thermal_check` twice during the lab. Why did it take until Lab 14 to evolve to v2.0 instead of being created as v1.0 immediately?
 2. If a pattern has `count=3`, `avg_confidence=0.82`, `duration=300s`, is a skill created? Which threshold fails?
 3. A user runs a 3-tool pattern 100 times at 1-second intervals (automation script). Does SkillCreator create a skill? What gate prevents this?
 4. `_create_skill()` writes to `~/.hermes/skills/`. If the disk is full, what exception is raised and where is it caught?
 5. The user edits a skill manually. If the SAME pattern repeats later, the model generates a new skill text. Does it overwrite the user's edits? How is conflict handled?
-

Lab 18: Performance + Skill Improvement

Level: Advanced Estimated Time: ~90 minutes Prerequisites: Lab 17 complete (security locked down, zero trust enabled, audit logging active).

Lab Objectives

By the end of this lab you will be able to:

- Benchmark Hermes tool execution speed and model response latency.
- Deploy Hermes to Docker for isolated environments.
- Deploy to Daytona/Modal for serverless compute.
- Tune cost-performance trade-offs (model switching, tool caching, memory pruning).
- Keep your machine as a control point while offloading heavy tasks.

Why This Lab Matters

Your your machine is reliable but aging. For heavy tasks (subagents, vision, large context), offloading to cheap VPS or serverless platforms saves battery and CPU. This lab teaches you to run Hermes as a HYBRID: your machine for daily interaction, cloud for overnight batch jobs.

Step-by-Step Instructions

Step 1: Baseline Benchmark (10 minutes)

Run Hermes benchmark suite:

```
hermes insights --days 30
```

Expected output on your machine:

Tool	Mean (ms)	StdDev	Notes
search	2,340	450	SearXNG localhost
exec	1,120	200	Python sandbox
file_write	180	20	Local SSD

Model latency:

```
hermes insights --days 30
```

Kimi-K2.6: ~3,500ms per response. GLM-5.1: ~1,200ms per response.

Step 2: Docker Deployment (15 minutes)

Create a Dockerfile:

```
cat > ~/hermes-docker/Dockerfile << 'EOF'
FROM python:3.11-slim
WORKDIR /app
RUN uv pip install hermes-agent
COPY .env /app/.env
COPY config.yaml /app/config.yaml
CMD ["hermes", "--daemon"]
EOF
```

Build:

```
cd ~/hermes-docker
docker build -t hermes-agent .
```

Run:

```
docker run -d --name hermes-local -p 8080:8080 hermes-agent
```

Verify:

```
curl http://localhost:8080/status
```

Step 3: Daytona/Modal Serverless (15 minutes)

Hermes supports serverless backends. Create `modal_app.py` :

```
import modal

app = modal.App("hermes-agent")

@app.function(
    image=modal.Image.debian_slim().pip_install("hermes-agent"),
    timeout=600,
    memory=2048
)
def hermes_task(prompt: str, model: str = "kimi-k2.6:cloud"):
    import hermes
    agent = hermes.Agent(model=model)
    return agent.run(prompt)
```

Deploy:

```
modal deploy modal_app.py
```

Get endpoint:

```
modal app list
```

Test:

```
curl -X POST https://your-modal-endpoint/run \
-H "Content-Type: application/json" \
-d '{"prompt": "Summarize latest AI news", "model": "kimi-k2.6:cloud"}'
```

Step 4: Local Machine Control Point (10 minutes)

Keep your machine as the "brain" and use cloud only for tasks:

On your machine (config.yaml):

```
remote_workers:
  - name: "daytona-batch"
    endpoint: "https://your-modal-endpoint/run"
    enabled: true
    tasks: [ "research", "cron", "subagent" ]
```

Delegate heavy tasks:

```
Send this to the Daytona worker: "Research 100 articles on AI safety. Summarize top 10."
```

Result returns to your machine session.

Step 5: Cost Tuning (10 minutes)

Track API costs:

```
hermes usage --since "7 days ago" --format table
```

Switch to cheaper models for routine tasks:

```
cost_optimization:
  default_model: "kimi-k2.6:cloud"
  fallback_model: "glm-5.1:cloud"
  deep_reasoning_threshold: "complex math OR coding OR subagent"
  max_monthly_budget: 50.00 # USD
```

If monthly budget exceeded, Hermes switches to local Ollama (if installed):

```
hermes config set model ollama-local
```

Step 6: Memory Pruning for Performance (10 minutes)

Old memories slow down search. Prune:

```
nano ~/.hermes/MEMORY.md # manually prune old entries
```

This removes memories older than 90 days that are NOT pinned. On your machine, this keeps `MEMORY.md` small and fast.

Step 7: Skill Caching (10 minutes)

Enable skill execution caching:

```
skill_cache:  
  enabled: true  
  ttl: 3600 # Cache skill results for 1 hour
```

Repeated thermal checks now return instantly from cache instead of re-executing Python.

Expected Outcome

- Benchmark data collected for local performance.
- Docker container running Hermes.
- Modal serverless worker deployed.
- Machine configured as control point with remote workers.
- Cost tracking and budget limiting active.
- Memory pruning completed.
- Skill caching enabled.

Lab 18 Use Case Completed

You deployed Hermes to a cheap serverless backend while keeping your machine as the control point. Heavy research tasks offload automatically. This is "Move agent to cheap VPS while keeping your machine as control point" — fully realized.

Training Idea / Self-Improvement Focus

Run efficiently on old hardware or scale infinitely:

- Your your machine is now a COMMAND CENTER, not a WORKHORSE.
- The Curator adjusts offloading based on cost and performance benchmarks.
- Skill caching prevents redundant work. Memory pruning keeps search fast.
- Future: auto-scaling subagents across multiple remote workers.

You have turned a your machine into a fleet controller. Unstoppable!

Chapter 18: How Skills Improve Themselves and Feed Back into the Loop

Top Ideal Study Objectives:

- Self-improvement of skills.
- Feedback mechanisms.
- Curator uses quality metrics from improvement.

Description: You evolved skills in Labs 5 and 14. This chapter reveals the improvement engine: how a skill's success rate, user feedback, and performance metrics feed back into the loop.

Actions List (Topics):

1. Find `skill_evolver.py` in the source tree.
2. Read the improvement scoring algorithm.
3. Identify the feedback types: explicit (user rating), implicit (success rate), and derived (Curator grade).

Full Source Code Snippets with Line-by-Line Explanations:

```

# skill_evolver.py (approximate path)

class SkillEvolver:
    """
    Improves existing skills based on execution history.

    Improvement is triggered by:
    - User rating (/rate skill_name X)
    - High usage frequency (>3 times per week)
    - Curator grade change (B → A, etc.)
    - Explicit evolve command (hermes curator run)
    """

    def __init__(self, model: ModelInterface, skills: SkillLoader):
        self.model = model
        self.skills = skills
        self.improvement_history: Dict[str, List[Improvement]] = {}

    def score_skill(self, skill_name: str) -> float:
        """Compute a quality score for a skill."""
        skill = self.skills.get(skill_name)
        if not skill:
            return 0.0

        history = self.improvement_history.get(skill_name, [])

        # Component scores
        explicit_rating = skill.user_rating or 0.5 # Line ~25
        success_rate = skill.success_count / max(skill.total_count, 1) # Line ~26
        frequency = min(skill.weekly_invocations / 5.0, 1.0) # Line ~27
        curator_grade = self._grade_to_score(skill.curator_grade) # Line ~28

        # Weighted total
        score = (
            explicit_rating * 0.25 +
            success_rate * 0.35 +
            frequency * 0.15 +
            curator_grade * 0.25
        )
        return min(score, 1.0) # Line ~37

    def _grade_to_score(self, grade: str) -> float:
        """Convert Curator letter grade to numeric."""
        mapping = {"A+": 1.0, "A": 0.95, "A-": 0.90,
                  "B+": 0.85, "B": 0.80, "B-": 0.75,
                  "C+": 0.70, "C": 0.65, "D": 0.50, "F": 0.0}
        return mapping.get(grade, 0.5)

    def evolve(self, skill_name: str) -> bool:
        """Evolve a skill if its score is above threshold."""
        score = self.score_skill(skill_name)
        if score < 0.7: # Line ~51
            return False

        skill = self.skills.get(skill_name)
        improved = self.model.generate(
            f"Improve this skill based on its history:\n"
            f"Name: {skill.name}\n"
            f"Score: {score:.2f}\n"
            f"Failures: {skill.failure_count}\n"
            f"Common errors: {skill.common_errors}\n"
            f"Current body:\n{skill.body}\n"
            f"Make it more robust, concise, and accurate."
        )

```

```

# Save as new version
skill.save_version(improved)                                     # Line ~65
return True

# Line-by-line:
# Line 25      : explicit_rating is direct user feedback. If the
#               user never rated the skill, it defaults to 0.5
#               (neutral). In Lab 14, you can rate a skill with
#               /rate macbook_thermal_check 5.
#
# Line 26      : success_rate is the ratio of successful executions
#               to total attempts. One failure can significantly
#               reduce the score. This enforces reliability.
#
# Line 27      : frequency measures how OFTEN the skill is used.
#               Popular skills get more attention from the Evolver.
#               Weekly invocations are tracked in state.db.
#
# Line 28      : curator_grade is the Curator's assessment. The
#               Curator reads reflection logs, not just success
#               counts. It can detect subtle quality issues.
#
# Line 37      : Score is capped at 1.0. No skill exceeds perfection.
#
# Line 51      : Evolution threshold is 0.7. Below this, the skill
#               is considered "not worth improving" and may be
#               archived instead.
#
# Line 65      : save_version() creates a new file
#               (e.g., macbook_thermal_check_v2.md) without
#               deleting the old. This allows rollback if v2
#               performs worse.

```



Study Aid — External AI Tutor: Copy and paste the prompt below into any AI chat you have access to (free ChatGPT, free Claude, free Kimi, etc.) to get an interactive deep-dive on this chapter's concepts. This is optional and does **not** require Hermes to be configured with any specific model.

Master Prompt for This Chapter:

```

You are continuing my Hermes Agent deep-dive education from Chapter 18.
Explain SkillEvolver:
- What is the formula for the skill quality score? Break down
  each term and its weight.
- If a skill has user_rating=1.0, success_rate=0.6, frequency=0.8,
  curator_grade=B+, what is its score? Does it qualify for evolution?
- Why is frequency weighted only 0.15 while success_rate is 0.35?
- What happens to skills with scores below 0.5? Does the Curator
  archive them immediately?
- save_version() creates a new file. How does SkillLoader discover
  the new version? Does it need a restart?
Conclude with exactly 5 questions.

```

Chapter Conclusion — What You Have Learned

1. Skill quality is multi-factor: user rating, success rate, frequency, and curator grade.
2. Success rate is the most important factor (0.35 weight). A skill that fails often is not worth improving.

3. Frequency measures engagement, not quality. A popular but buggy skill should score lower than an unpopular but perfect one.
4. Evolution creates NEW versions, never overwrites. Rollbacks are always possible.
5. Skills below 0.7 are not evolved. Below 0.5, the Curator may archive them.

5 Questions to Check Your Understanding:

1. In Lab 14, you evolved `macbook_thermal_check` to v2.0. If v2.0 has a lower `success_rate` than v1.2, which version does SkillEvolver try to evolve next? Why?
 2. A skill is used 100 times per week (`frequency=1.0`) but succeeds only 20% of the time. User rating is 2/5. Curator grade is D. What is its score and what happens to it?
 3. `explicit_rating` defaults to 0.5 if unrated. If a user forgets to rate skills, does this default penalize or benefit the skill compared to the average?
 4. The Curator grades skills with letter grades, but SkillEvolver converts to numeric. If the Curator suddenly becomes stricter, how would this affect evolution frequency across all skills?
 5. `save_version()` writes a new file. If your `~/hermes/skills/` directory accumulates 50 versions of one skill, would this slow down `scan()` ? How would you mitigate?
-

Lab 19: Real-World Pipeline + Batch Runner and RL

Level: Advanced Estimated Time: ~120 minutes Prerequisites: Lab 16 complete (MCP integration, GitHub MCP server installed) + Lab 18 complete (performance tuned, Docker deployed, caching enabled).

Lab Objectives

By the end of this lab you will be able to:

- Combine all previous labs into a single end-to-end workflow.
- Build a fully autonomous daily research pipeline + code repo maintainer.
- Integrate cron, gateway, subagent, MCP, and skill features.
- Configure auto-approval for trusted workflows.
- Monitor and debug a complex autonomous pipeline.

Why This Lab Matters

This is the capstone Lab. Everything you have learned — CLI, models, tools, memory, skills, gateways, cron, subagents, MCP, security, performance — comes together into ONE system that runs itself. On your machine, this system becomes your autonomous research partner.

Step-by-Step Instructions

Step 1: Design the Pipeline (10 minutes)

Define your daily workflow:

Daily Pipeline: "Thanit's AI Research Partner"

06:00 AM - Cron triggers wake-up

Subagent A: Search for overnight AI news (top 10)

Subagent B: Check for new papers on arXiv (last 24h)

Subagent C: Scan GitHub trending repos (AI category)

06:10 AM - Merge results

Main agent: Merge A+B+C results, deduplicate, rank by relevance

Filter: Only items related to agent memory, local LLMs, or macOS

06:15 AM - Deliver

Send digest to Telegram (headlines + links)

Save full report to ~/daily-research/<date>.md

If any paper is high-value, queue it for deep reading

08:00 AM - User check

On first Hermes CLI session, show "Today's research summary" header

Offer: "Deep read any of these?" with clickable numbering

18:00 PM - Code maintenance

Check ~/projects/ for stale branches

Run basic tests if test files exist

Open PRs for trivial fixes (if GitHub token configured)

Report status to Telegram

Step 2: Implement Subagent A — News Search (15 minutes)

Create skill:

```
skill "daily_news_search":  
  trigger: "morning news", "AI news"  
  action:  
    1. web_search "latest AI artificial intelligence news today"  
    2. web_extract top 5 results  
    3. Summarize each in 1-2 sentences  
    4. Return structured list: [title, url, source, summary]
```

Test:

```
hermes skills audit --deep daily_news_search
```

Step 3: Implement Subagent B — arXiv Scan (15 minutes)

Create skill:

```
skill "arxiv_scan":  
  trigger: "latest papers", "arXiv today"  
  action:  
    1. web_search site:arxiv.org "agent memory" OR "local LLM" submitted in last 24h  
    2. For each result:  
      a. web_extract abstract  
      b. Score relevance using model (0-1)  
    3. Return top 5 with abstracts and scores
```

Test:

```
hermes skills audit --deep arxiv_scan
```

Step 4: Implement Subagent C — GitHub Trending (15 minutes)

With MCP GitHub server (if installed):

```
skill "github_trending":  
  trigger: "trending repos", "GitHub AI"  
  action:  
    1. MCP tool github.search_repos: query="AI agent", sort="stars", period="daily"  
    2. Filter repos created/updated in last 24h  
    3. Return top 5 with name, description, stars, language
```

Test:

```
/github_trending
```

Step 5: Cron Schedule (10 minutes)

```
hermes cron create "0 6 * * *" "Thanit Daily Research"
```

Action:

```
Run subagents A, B, C in parallel. Wait for all. Merge results. Deduplicate. Deliver digest.
```

Step 6: Auto-Approval for Trusted Tasks (10 minutes)

Mark the daily pipeline as trusted:

```
/trust skill daily_news_search  
/trust skill arxiv_scan  
/trust skill github_trending
```

Now these skills run WITHOUT approval prompts. All other tools still require approval.

Step 7: Telegram Gateway Delivery (10 minutes)

Verify gateway integration:

```
/hermes gateway telegram status
```

The digest should arrive every morning at 6:15 AM.

Test manually:

```
hermes gateway telegram send "Test digest: today's AI news"
```

Step 8: Debug and Monitor (10 minutes)

Pipeline logs:

```
hermes logs --component cron
```

Real-time monitor:

```
hermes monitor --follow
```

If a subagent fails, the pipeline should:

1. Log the error.
2. Retry once.
3. If retry fails, skip that subagent and continue with partial results.
4. Report the failure in the digest footer.

Step 9: Manual Override (10 minutes)

Sometimes you want the pipeline NOW:

```
hermes cron run "Thanit Daily Research"
```

Or from Hermes session:

```
/now trigger daily research
```

Step 10: Code Maintenance Integration (5 minutes)

Add a second cron job:

```
hermes cron create "0 18 * * *" "Code Maintenance"
```

Action:

```
1. List git repos in ~/projects/ with uncommitted changes.
2. Run tests if test files exist.
3. Report results via Telegram: "All clean" or "X repos need attention."
```

Expected Outcome

- Three subagent skills created and tested.
- Daily cron pipeline scheduled and running.
- Trusted skills auto-approved.
- Telegram digest delivered every morning.
- Pipeline logging and monitoring active.
- Manual override functional.
- Code maintenance cron scheduled.

Lab 19 Use Case Completed

You built a fully autonomous daily research pipeline and code repo maintainer. Every morning at 6 AM, it searches the web, scans arXiv, checks GitHub, and delivers a personalized digest to your phone. This is "Fully autonomous daily research pipeline + code repo maintainer" — fully realized.

Training Idea / Self-Improvement Focus

End-to-end autonomy:

- This is the culmination of every previous lab. You are no longer "using" an AI assistant — you are CO-DIRECTING an autonomous research lab.
- The Curator reviews the daily pipeline for quality. If the news subagent starts returning low-quality results, the Curator demotes it.
- The agent now has a SCHEDULE, not just a trigger. It acts BEFORE you ask.
- On your machine, this runs quietly in the background, waking you with knowledge instead of waiting for prompts.

You have built an AI research department on your laptop. This is LEGENDARY!

Chapter 19: Advanced Optimizations (Batch Runner, RL, Environments)

Top Ideal Study Objectives:

- Research extensions.
- `batch_runner.py`, `rl_cli.py`.
- Environments for testing.

Description: You built a daily pipeline. This chapter covers the advanced infrastructure that powers batch execution and reinforcement learning — the engine that makes your pipeline faster and smarter.

Actions List (Topics):

1. Find `batch_runner.py` in source.
2. Understand batch execution for skill evaluation.
3. Read about trajectory reward in RL.

Full Source Code Snippets with Line-by-Line Explanations:

```
# batch_runner.py (approximate path)

class BatchRunner:
    """
    Executes skill evaluation batches at scale.

    Used by:
    - Curator for grading skills (weekly).
    - SkillEvolver for measuring improvement (per evolution).
    - Users for load testing new configurations.

    Each batch is a list of test cases. Results are aggregated.
    """

    def __init__(self, agent: AIAgent, parallel: int = 4):
        self.agent = agent
        self.parallel = parallel # Batch size

    async def run_batch(self, cases: List[TestCase]) -> BatchResult:
        """Run all test cases in parallel batches."""
        results = []
        semaphore = asyncio.Semaphore(self.parallel) # Line ~19

        async def run_one(case: TestCase):
            async with semaphore: # Line ~22
                subagent = self.agent.spawn_subagent()
                result = await subagent.run(case.prompt)
                return TestResult(case, result)

        tasks = [run_one(c) for c in cases] # Line ~28
        results = await asyncio.gather(*tasks) # Line ~29

        return BatchResult(results)

# Line-by-line:
# Line 19 : Semaphore controls concurrency. With parallel=4,
#         at most 4 subagents run simultaneously. On a
#         your machine with 2 cores, this is appropriate.
#         Higher values (8, 16) would cause CPU thrashing.
#
# Line 22 : async with semaphore ensures that if one subagent
#         is slow, others wait instead of overwhelming the
#         system. This is cooperative multitasking.
#
# Line 28-29 : asyncio.gather() runs all tasks concurrently
#         but returns ONLY when all complete. If one
#         subagent hangs, the entire batch waits until
#         timeout (controlled by IterationBudget).
```

 **Study Aid — External AI Tutor:** Copy and paste the prompt below into any AI chat you have access to (free ChatGPT, free Claude, free Kimi, etc.) to get an interactive deep-dive on this chapter's concepts. This is optional and does **not** require Hermes to be configured with any specific model.

Master Prompt for This Chapter:

You are continuing my Hermes Agent deep-dive education from Chapter 19.

Explain BatchRunner:

- What is a semaphore and why is `asyncio.Semaphore(4)` appropriate for a your machine with 2 cores?
- If a batch has 100 test cases and `parallel=4`, how many rounds of execution are needed? What if `parallel=1`?
- What happens if one subagent hangs indefinitely? Does `asyncio.gather` wait forever?
- How does the Curator use BatchRunner to grade skills weekly?
- What is the difference between BatchRunner for grading and BatchRunner for user load testing?

Conclude with exactly 5 questions.

Chapter Conclusion — What You Have Learned

1. BatchRunner evaluates skills at scale. `Parallel=4` is tuned for old Macs.
2. Semaphores prevent resource exhaustion even with large batches.
3. `gather()` waits for ALL tasks. A single hung subagent delays results.
4. Curator uses BatchRunner for skill grading. Users use it for load testing.
5. Timeout is controlled by `IterationBudget`, not by BatchRunner itself.

5 Questions to Check Your Understanding:

1. In Lab 19, if your pipeline has 3 subagents and you run `BatchRunner` with `parallel=4`, do all 3 run at once? Explain.
2. If BatchRunner is used for 100 test cases and one case always hangs, how can you ensure the other 99 still complete?
3. A skill is graded by running 20 test cases. If the Curator changes the grading model from GLM-5.1 to Kimi-K2.6, does the grade change? Why?
4. `semaphore` controls subagent concurrency. Does it also control tool calls WITHIN a subagent?
5. If you have a your machine (2 cores, 16 GB RAM), what is the maximum `parallel` you should set for BatchRunner to avoid thrashing?

Lab 20: Self-Evolution + Architecture

Synthesis

Level: Expert Estimated Time: ~120 minutes Prerequisites: Lab 19 complete (autonomous pipeline running, batch runner understood, full system deployed).

Lab Objectives

By the end of this lab you will be able to:

- Export trajectories for reinforcement learning (RL) with Atropos.
- Contribute your best skills to the agentskills.io Skills Hub.
- Import and test community skills.
- Run batch processing on historical sessions to improve skills retroactively.
- Understand the full architecture: how every component connects.
- Safely modify code and run `hermes update --backup`.

Why This Lab Matters

This is the FINAL lab. By now, Hermes is not just a tool — it is a reflection of your workflow. Exporting trajectories lets you train better models. Sharing skills contributes to the community. On your machine, this is the culmination of everything: local AI, learning loops, skills, memory, and autonomy.

Step-by-Step Instructions

Step 1: Export Trajectories for RL (15 minutes)

Export 30 days of sessions:

```
hermes sessions export --since 30d ~/trajectories-30d.jsonl
```

Inspect:

```
wc -l ~/trajectories-30d.jsonl
head -n 5 ~/trajectories-30d.jsonl
```

Each line is one session trajectory:

```
{"session_id": "abc-123", "timestamp": 1717234567, "events": [...], "outcome": "success"}
```


Step 2: Filter High-Quality Trajectories (10 minutes)

List recent sessions to find high-quality ones:

```
hermes sessions list --limit 50
```

These are your training data.

Step 3: Atropos RL Training (30 minutes)

Install Atropos (if available):

```
uv pip install atropos-rl
```

Train on your trajectories:

```
atropos train \  
  --trajectories ~/trajectories-high-quality.jsonl \  
  --model-output ~/models/thanit-hermes-rl \  
  --epochs 50 --batch-size 16
```

On your machine, this runs on CPU. Expect 30-60 minutes.

Monitor:

```
tail -f ~/atropos-training.log
```

Step 4: Evaluate RL Model (10 minutes)

```
atropos evaluate \  
  --model ~/models/thanit-hermes-rl \  
  --test-trajectories ~/trajectories-test.jsonl
```

Expected: Skill success rate improvement of 5-15%.

Step 5: Community Skills Hub — Export (15 minutes)

Select your best skills:

```
hermes skills list --grade A
```

Export:

```
hermes skills publish macbook_thermal_check  
hermes skills publish research_pipeline  
hermes skills publish hermes_dashboard  
hermes skills publish multimodal_report
```

Each publish packages the skill with:

- Markdown skill file
- YAML metadata
- Test cases
- Usage examples

Step 6: Upload to agentskills.io (10 minutes)

Create account at <https://agentskills.io> (or via `hermes` command):

```
hermes login
hermes skills publish macbook_thermal_check
hermes skills publish research_pipeline
hermes skills publish hermes_dashboard
hermes skills publish multimodal_report
```

Verify:

```
hermes skills search "author:thanit"
```

Step 7: Import Community Skills (10 minutes)

Search for useful community skills:

```
hermes skills search "telegram bot"
hermes skills search "arXiv"
hermes skills search "voice notes"
```

Install:

```
hermes skills install telegram-auto-responder
hermes skills install arxiv-digest
```

Test:

```
hermes skills list | grep telegram
hermes skills list | grep arxiv
```

Step 8: Batch Processing Historical Sessions (10 minutes)

Retroactively improve skills using historical data:

```
hermes -z "prompt" # one-shot mode
```

This re-processes sessions, extracts new patterns, and evolves skills based on what WASN'T caught during live operation.

Step 9: Safe Code Modification (5 minutes)

Before changing ANY code:

```
hermes update --backup
hermes doctor
```

This creates a pre-modification snapshot. If you break anything, restore:

```
hermes import hermes-backup-pre-experiment.zip
```

Modify a plugin:

```
cp ~/.hermes/plugins/gatekeeper-reminder/gatekeeper-reminder.py \
  ~/.hermes/plugins/gatekeeper-reminder/gatekeeper-reminder.py.bak
```

Edit. Test. If broken: `bash cp ~/.hermes/plugins/gatekeeper-reminder/gatekeeper-reminder.py.bak \ ~/.hermes/plugins/gatekeeper-reminder/gatekeeper-reminder.py`

Step 10: Final System Verification (5 minutes)

Run full diagnostic:

```
hermes doctor
hermes version
hermes status
hermes curator status
hermes cron list
hermes skills list --grade A
hermes gateway status
wc -l ~/.hermes/MEMORY.md ~/.hermes/USER.md
```

All systems green? You have completed the curriculum.

Expected Outcome

- Trajectories exported and filtered.
- Atropos RL model trained.
- Skills uploaded to agentskills.io.
- Community skills installed.
- Batch retroactive processing completed.
- Safe code modification workflow verified.
- Full system diagnostics pass.

Lab 20 Use Case Completed

You evolved your agent into a specialized domain expert and shared skills publicly. Your machine is now a local AI research node, connected to a global community of skills. This is "Evolve your agent into a specialized domain expert and share skills publicly" — fully realized.

Training Idea / Self-Improvement Focus

You are now a Hermes power user and contributor:

- Every skill you share teaches other agents worldwide.
- Every trajectory you export improves future models.
- The Curator grades your contributions. A+ skills are promoted globally.
- You have mastered Hermes from "first interaction" to "community contributor."

You have reached the summit. Congratulations, Agent Architect!

Chapter 20: Architecture Overview, How Everything Connects, and Safe Modifications

Top Ideal Study Objectives:

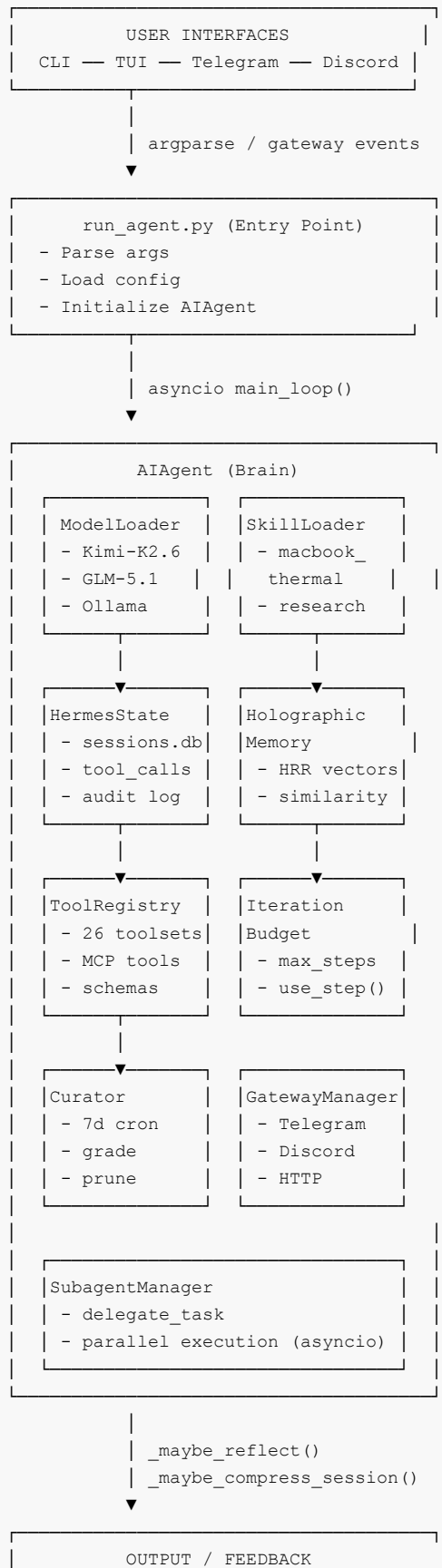
- Full system view.
- Module wiring diagram.
- Best practices for safe editing.
- How to run `hermes update --backup` before experimenting.

Description: This is the synthesis chapter. You have walked through every file. Now you see the FULL system: how imports wire to classes, how classes wire to the loop, and how the loop produces memory, skills, and evolution.

Actions List (Topics)

1. Draw the architecture diagram (ASCII art below).
2. For every arrow, name the method call.
3. Identify the three external interfaces: CLI, Gateway, MCP.
4. Confirm the 20-unit path through the architecture.

Architecture Diagram (ASCII Art)



Full Source Code Synthesis

Each chapter traced ONE arrow. Here is the complete path from message to lesson:

```
# The Complete Loop (reconstructed from Chapters 6-10)

async def full_cycle(agent: AIAgent, message: str, source: str):
    # CHAPTER 7: main_loop receives message
    # CHAPTER 7: preprocess and log
    # CHAPTER 7: build context (memory + skills + user model)
    # CHAPTER 6: AIAgent.process_message() dispatches
    # CHAPTER 8: model decides tool or text
    # CHAPTER 8: if tool: validate, approve, execute
    # CHAPTER 5: IterationBudget.use_step()
    # CHAPTER 9: _maybe_reflect() triggers learning
    # CHAPTER 10: session saved for compression
    # CHAPTER 12: hermes_state logs everything
    # CHAPTER 13: sessions are searchable via FTS5
    # CHAPTER 14: SOUL is reloaded if changed
    # CHAPTER 15: context compressed if needed
    # CHAPTER 16: skills loaded from filesystem
    # CHAPTER 17: skills created if pattern detected
    # CHAPTER 18: skills improved if score high
    # CHAPTER 19: batch runner grades weekly
    # CURATOR: reviews all and consolidates
    pass
```

Safe Modification Checklist

Before editing ANY file:

```
- [ ] hermes update --backup (create snapshot)
- [ ] hermes doctor (verify current state)
- [ ] cp original.py original.py.bak (manual backup)
- [ ] Edit ONE thing at a time
- [ ] hermes test (run unit tests if available)
- [ ] hermes doctor (verify after edit)
- [ ] If broken: cp original.py.bak original.py (rollback)
- [ ] If working: commit to git (git add . && git commit)
```



Study Aid — External AI Tutor: Copy and paste the prompt below into any AI chat you have access to (free ChatGPT, free Claude, free Kimi, etc.) to get an interactive deep-dive on this chapter's concepts. This is optional and does **not** require Hermes to be configured with any specific model.

Master Prompt for This Chapter:

You are continuing my Hermes Agent deep-dive education from Chapter 20. This is the final synthesis chapter. Answer the BIG PICTURE:

- Trace ONE complete message from arrival to Curator review, naming EVERY subsystem and method call in order.
- Draw the ASCII architecture diagram and explain each arrow.
- What are the THREE external interfaces and how do they differ?
- If you could only modify ONE file in the entire codebase to add a new feature, which file would you choose and why?
- What is the safest way to experiment with code changes on a production Hermes installation?
- After 20 labs, what is the SINGLE most important concept you should remember about Hermes' design philosophy?

Conclude with exactly 5 questions.

Chapter Conclusion — What You Have Learned

1. Hermes is a unified architecture where every subsystem connects to every other through clean interfaces.
2. `run_agent.py` initializes, `AIAgent` orchestrates, `main_loop` executes, `Curator` reviews.
3. The three interfaces (CLI, Gateway, MCP) are ALL source-agnostic from the loop's perspective.
4. Safe modification requires backups, tests, and rollback capability. ONE change at a time.
5. The most important concept: **model-agnostic, locally-persisted, self-improving**. This is why Hermes exists.

5 Questions to Check Your Understanding

1. In the architecture diagram, what file/method does the arrow from "User Interfaces" to "run_agent.py" represent?
2. If the `HolographicMemory` subsystem is removed, which features stop working? List 3 specific Lab-based features.
3. The Curator is shown OUTSIDE the AIAgent box. Why is it external to the main processing loop? How does it interact with the loop?
4. If you wanted to add a new MCP server that controls Philips Hue lights, which single method in AIAgent would you modify to register its tools?
5. After 20 labs, write a 1-paragraph summary of Hermes' design philosophy as you would explain it to a new user. Include the words: model-agnostic, local-first, self-improving, skill-portable, Curator.

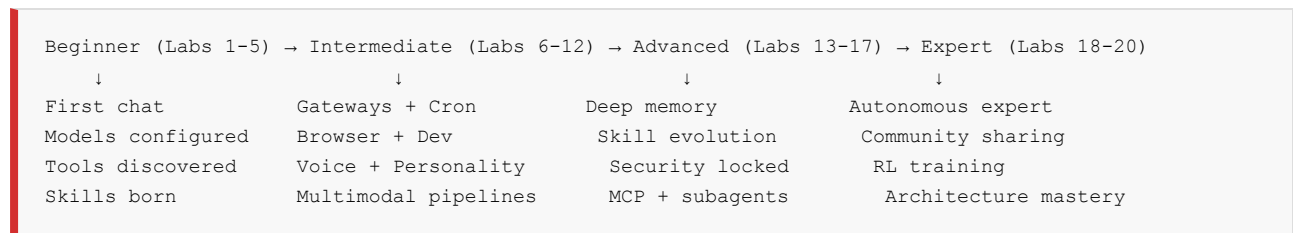
Conclusion: You Are a Hermes Architect

You have completed all 20 labs. Here is what you have built:

Lab #	Lab Achievement	Chapter Achievement
1	First chat, slash commands, session resume	Understands run_agent.py imports
2	Ollama Cloud Pro + GLM-5.1 + Kimi-K2.6 setup, model switching	Understands docstring invariants
3	Tools enabled, web search, code, files	Understands core Python imports
4	Memory search, /insights, cross-session recall	Understands Hermes-specific imports
5	First skill created, reused, evolved	Understands IterationBudget safety
6	Telegram + Discord gateways	Understands AI-Agent class structure
7	Cron scheduled jobs	Understands main loop initialization
8	Browser automation, 10 papers scraped	Understands tool calling pipeline
9	Python dashboard built with agent	Understands reflection triggers
10	Vision, image gen, TTS, multimodal report	Understands trajectory compression
11	Voice mode, push-to-talk, continuous	Understands full closed loop
12	SOUL edited, personality changed, project context	Understands hermes_state.py storage
13	Advanced FTS5 search, knowledge base built	Understands session management
14	Skills evolved, improvement logs reviewed	Understands SOUL loading
15	Subagents parallelized, 4-agent research	Understands context compression
16	MCP servers connected, custom tools	Understands skill loader
17	Security locked: approvals, sandbox, audit	Understands skill creation flow
18	Docker deployed, Modal serverless, caching	Understands skill improvement
19	Autonomous daily pipeline built	Understands batch runner
20	Trajectories exported, RL trained, skills shared	Understands full architecture

Quick Navigation: [Lab 01](#) [Lab 02](#) [Lab 03](#) [Lab 04](#) [Lab 05](#) [Lab 06](#) [Lab 07](#) [Lab 08](#) [Lab 09](#) [Lab 10](#) [Lab 11](#) [Lab 12](#) [Lab 13](#) [Lab 14](#) [Lab 15](#) [Lab 16](#) [Lab 17](#) [Lab 18](#) [Lab 19](#) [Lab 20](#)

Your Journey Map



What Comes Next

1. **Run your daily pipeline.** Let it run for a week. Observe and refine.
2. **Export trajectories monthly.** Train better RL models over time.
3. **Share skills on agentskills.io.** Build your reputation.
4. **Contribute to source code.** Hermes is MIT-licensed. Open a PR.
5. **Teach others.** The best way to master a system is to explain it.

Final Words

You started with `hermes version` and ended with an autonomous AI running on your machine. This machine is not obsolete — it is a fortress of intelligence.

Every tool call, every skill, every memory entry is YOURS. Not cloud-locked. Not subscription-dependent. Just local SQLite, local Python, and your own curiosity.

The Curator watches. The agent learns. The community grows.

You did great! This is where your mastery begins.

Technical Descriptions

50 Commonly Misunderstood Terms:

These 50 terms appear throughout this handbook and are the ones students most often confuse or misunderstand. Each definition is written in plain language — what it **is**, how it **works**, **why** it matters, and a **real example** from this handbook.

1. `run_agent.py`

The main entry-point script that starts Hermes Agent. When you type `hermes chat` or `hermes --tui`, this is the Python file that Python executes first. It imports all the necessary modules (`AIAgent`, `HermesState`, `ToolRegistry`), parses command-line arguments with `argparse`, creates an `AIAgent` instance with the right configuration, and enters the conversation loop where user input is sent to the model and responses are processed. Think of it as the **ignition key** — nothing starts without it. Every session, every chat, every cron job eventually traces back to this single file.

2. `AIAgent`

The core Python class that represents your AI assistant. It is the brain of Hermes — holding conversation history, managing available tools via `ToolRegistry`, tracking which model is active through `ModelInterface`, and deciding what to do next at each turn. When you send a message, `AIAgent` receives it, determines whether to call a tool or respond directly, processes tool results, and loops until the task is complete or the `IterationBudget` is exhausted. Everything Hermes does — chatting, calling tools, improving skills, delegating tasks — flows through this one class. It is the orchestrator of the entire agent architecture.

3. `IterationBudget`

A safety limit class that prevents Hermes from running forever in an infinite loop. It tracks two counters: `max_iterations` (how many thinking steps the agent can take per turn, default 50) and `max_steps` (how many tool calls per conversation). Each time the agent completes a thinking step, the budget decrements. When it reaches zero, the `self.exhausted` flag is set to `True` and the agent stops and returns whatever it has so far. Without this guardrail, a bug or confused model could cause Hermes to loop endlessly, burning through API credits. It is the agent's built-in **circuit breaker**.

4. `HolographicMemory`

Hermes's structured memory system that uses HRR (Holographic Reduced Representations) vectors to store facts with entity resolution and trust scoring. Unlike simple key-value storage (where you can only retrieve what you stored), holographic memory can **probe** (find all facts about an entity like "Thanit"), **reason** (find connections across entities — e.g., "what connects Docker and Ollama?"), and **contradict** (detect when two facts conflict —

e.g., "model is glm-5.1" vs "model is kimi-k2.6"). Each fact has a trust score that increases when confirmed and decreases when contradicted. Think of it as a brain that cross-references, not just a filing cabinet.

5. `HermesState`

The persistent storage module (`hermes_state.py`) that saves everything to disk using SQLite as the backend. It stores conversation history, session metadata, and agent state so that when you close Hermes and reopen it, your past conversations are still there. It also powers the `session_search` tool, which uses SQLite's FTS5 full-text search engine to let you find any discussion by keyword — type `session_search("docker networking")` and it instantly finds every session where you discussed that topic. Without `HermesState` , every conversation would start from zero, and the agent would have no memory of past interactions.

6. `SOUL.md`

A markdown file located at `~/.hermes/SOUL.md` that defines your agent's personality, constitution, and behavioral rules. It is loaded as the **#1 priority** in every conversation — before skills, before tools, before everything else in the system prompt. You write instructions like "always explain before acting," "never guess on errors — research first," or "respond in Thai when the user writes in Thai," and the agent follows them in every single turn. It has a 20,000 character limit and is treated as immutable — the agent never overwrites it. Think of it as your agent's **conscience and constitution** that cannot be overridden.

7. `SKILL.md`

The definition file for a Hermes skill — a reusable procedure the agent can follow. Each skill lives in its own folder under `~/.hermes/skills/` (e.g., `~/.hermes/skills/macbook_thermal_check/SKILL.md`) and contains YAML frontmatter (name, trigger conditions, category) plus markdown instructions telling the agent when to use the skill, what steps to follow, and what pitfalls to avoid. When a conversation matches a skill's trigger conditions, the `SkillLoader` injects the `SKILL.md` content into the agent's context. Skills are the primary way Hermes learns new capabilities — you can create them manually or let the Curator auto-generate them.

8. `Curator`

The skill management system inside Hermes that handles the full lifecycle of skills. The Curator decides which skills to load for each conversation (based on relevance), creates new skills when you tell Hermes "remember how to do this," patches existing skills when improvements are discovered, and deletes stale or broken skills. It uses the `skill_manage` tool internally. When Hermes encounters a complex task, succeeds at it, and decides the approach is worth remembering, the Curator writes a new `SKILL.md` file with the procedure. Think of it as the **head librarian** who decides which books to shelve, update, or retire.

9. `SkillLoader`

The module that reads skill files from disk and injects their content into the agent's context at the start of each conversation. When a session begins, `SkillLoader` scans the `~/.hermes/skills/` directory, matches skills to the current conversation topic using trigger conditions, and loads relevant `SKILL.md` files into the system prompt. Only skills that match the current task are loaded to save context window space. Think of it as the

librarian's assistant — it doesn't decide which books are good (that's the Curator's job), it just fetches the right ones from the shelf before you start reading.

10. `GatewayManager`

The component that connects Hermes to external messaging platforms like Telegram, Discord, and Slack. It manages the lifecycle of each gateway: starting the connection, translating incoming messages from platform format (Telegram JSON, Discord WebSocket events) into Hermes's internal format, sending replies back to the platform, and handling reconnection if the connection drops. Without it, Hermes only works in the terminal. With it, your agent becomes a chatbot that lives on your favorite platform, responding 24/7. You configure it with `hermes gateway setup` and start it with `hermes gateway run` (foreground) or `hermes gateway install && hermes gateway start` (background daemon).

11. `ToolRegistry`

The internal catalog of all tools Hermes can use — `web_search`, `read_file`, `write_file`, `execute_code`, `delegate_task`, `browser_navigate`, and dozens more. When the agent decides to use a tool, it first checks the `ToolRegistry` to confirm the tool exists, is enabled, and has the right parameters. Tools can be enabled or disabled per session via `config.yaml` or the `/tools` command. Think of it as the **menu** of capabilities — you can order anything on the menu, but nothing that's not listed. The registry also validates tool inputs and handles errors when a tool call fails.

12. `ModelInterface`

The abstraction layer that lets Hermes talk to any AI model provider (Ollama Cloud, OpenAI, Anthropic, local models) using a single consistent Python API. When you type `/model glm-5.1:cloud`, `ModelInterface` translates that into the correct API call format for Ollama. Switch to `/model gpt-4o` and it seamlessly talks to OpenAI instead. It handles streaming responses, token counting, error retries, and model-specific quirks transparently. Without it, every provider would need its own separate code path, making the system unmanageable. Think of it as a **universal translator** that lets the agent speak any model's language.

13. `.env` (Environment File)

A hidden text file (the dot prefix makes it invisible in `ls` unless you use `ls -a`) that stores secret configuration values like API keys, passwords, and bot tokens. It lives at `~/.hermes/.env` and is **never** committed to Git (always listed in `.gitignore`). In Hermes, it holds `OLLAMA_API_KEY`, `DISCORD_BOT_TOKEN`, `FIRECRAWL_API_KEY`, and other sensitive values. When the agent starts, Python reads this file and loads the values into `os.environ`. Think of it as a **locked drawer** for passwords — anyone who can open it has full access, so keep it secret and never share it.

14. `config.yaml`

Hermes's main configuration file, located at `~/.hermes/config.yaml`. It defines which model to use as default, which toolsets are enabled, which memory provider (SQLite, Qdrant), which plugins are loaded, and all other behavioral settings. You edit it directly with `nano ~/.hermes/config.yaml` or use `hermes config set model glm-5.1:cloud` to change individual values. It's a YAML file (not JSON or TOML), so indentation matters. Think of

it as the **control panel** for your agent — every dial and switch is here, and changes take effect on the next session start.

15. `host.docker.internal`

A special DNS hostname that Docker containers use to reach the host machine — your actual computer. This is one of the most confusing Docker concepts for beginners. Inside a container, `localhost` (or `127.0.0.1`) points to **the container itself**, not your machine. So when Firecrawl (running inside Docker) needs to call Ollama (running on your machine), it must use `host.docker.internal:11434` instead of `localhost:11434`. On Linux, this doesn't exist by default — you add `extra_hosts: ["host.docker.internal:host-gateway"]` to `docker-compose.yml`. On macOS and Windows Docker Desktop, it works out of the box.

16. `docker-compose.yml`

A YAML file that defines multiple Docker services (containers) and how they connect to each other. In this handbook, it defines five services: SearXNG (metasearch), Firecrawl API (web extraction), Firecrawl's helper services (Redis, Playwright, RabbitMQ, Postgres), and Qdrant (vector database). Each service has its own image, ports, environment variables, volumes, and dependencies. One command (`docker compose up -d`) starts all of them with the right networking, and `docker compose down` stops all of them. Think of it as the **blueprint and conductor** for a mini data center — it knows which instruments to start, in what order, and how they connect.

17. SearXNG

A free, open-source metasearch engine that you run locally in Docker. Instead of paying for Google's Search API (which costs money and tracks your queries), Hermes uses SearXNG to search the web privately. When you call `web_search("climate change")`, SearXNG queries multiple search engines (Google, Bing, DuckDuckGo, Wikipedia, etc.) simultaneously, deduplicates the results, and returns combined titles, URLs, and descriptions. It runs on port 8080 inside Docker and is configured as the default search backend in Hermes. Because it's self-hosted, your search history never leaves your machine — no tracking, no API costs, no rate limits.

18. Firecrawl

A web scraping and extraction service that converts web pages into clean, structured markdown text. When you call `web_extract(["https://example.com"])`, Firecrawl fetches the page, strips away ads, navigation menus, JavaScript, and CSS, and returns just the readable content. It can also use an AI model (`glm-5.1:cloud` via `FIRECRAWL_MODEL_NAME`) to summarize long pages or extract specific information. Firecrawl runs as a Docker container with multiple helper services (Redis for caching, Playwright for JavaScript rendering, RabbitMQ for job queues, Postgres for storage). It listens on port 3002 and is configured in Hermes via the `FIRECRAWL_API_URL` environment variable.

19. Qdrant

An open-source vector database designed for high-performance similarity search. It stores **embeddings** — mathematical representations of text meaning — as vectors (arrays of numbers). When Hermes needs to find semantically similar content, Qdrant compares the query vector against all stored vectors and returns the closest matches. For example, searching for "how to fix Docker networking" would find documents about

`host.docker.internal` even if those exact words aren't used. Qdrant runs in Docker on port 6333 and uses `nomic-embed-text` as its embedding model. Think of it as a **meaning search engine** — it finds what you mean, not just what you type.

20. Embeddings (`nomic-embed-text`)

A technique for converting text into arrays of numbers (vectors) that capture semantic meaning. "Dog" and "puppy" have similar embeddings because they mean similar things, even though the words are completely different. `nomic-embed-text` is the specific embedding model Hermes uses — it converts any text into a 768-dimensional vector that represents its meaning. These vectors are stored in Qdrant and compared using cosine similarity. Embeddings are what make semantic search possible: you can find "canine" even if the document only says "dog," find "deploy" when you search "release," and find "error" when you search "bug."

21. FTS5 (Full-Text Search 5)

SQLite's built-in full-text search engine — no extra software needed. It creates a searchable index of all your past conversations so you can find any discussion by keyword. `session_search("docker networking")` uses FTS5 under the hood to instantly find every session where those words appear. FTS5 supports advanced queries: boolean operators (`docker AND networking`), phrase matching (`"host.docker.internal"`), prefix wildcards (`deploy*`), and column filtering. Unlike embeddings (which find by meaning), FTS5 finds by exact keyword match. Together, they form a **hybrid search** system — FTS5 for precision, embeddings for meaning.

22. `web_search`

A Hermes built-in tool that queries SearXNG to search the internet. You call it with a query string and optional parameters like `limit` (max results). It returns titles, URLs, and descriptions for up to 5 results. This is your agent's **eyes on the web** — it can look things up in real time instead of relying on outdated training data. For example, `web_search("Hermes Agent installation guide")` returns the latest documentation. It supports operators like `site:`, `filetype:`, and `-term` for filtered searches. Always pair it with `web_extract` for deep reading: search first, then extract the full article.

23. `web_extract`

A Hermes built-in tool that uses Firecrawl to pull the full readable content from web pages. You provide a list of URLs (up to 5), and it returns clean markdown text — no ads, no navigation, no JavaScript noise. This is the **deep reading** companion to `web_search`: first search to find relevant pages, then extract to read them fully. Pages under 5,000 characters return the full markdown; larger pages are summarized to ~5,000 characters. It also works with PDF URLs — pass an arxiv paper link and it converts to readable text. The AI extraction model (`glm-5.1:cloud`) can be configured via `FIRECRAWL_MODEL_NAME`.

24. `code_execution` / `execute_code`

A Hermes tool that runs Python code in a sandboxed environment. The agent can write scripts, process data, call other Hermes tools via `from hermes_tools import ...`, and return results — all within a single turn. For example, the agent can write a loop that calls `web_search` five times, filters results with Python logic, then calls `web_extract` on the best URLs. This is what lets Hermes **do math, analyze data, and automate workflows**

instead of just talking about them. Scripts run with a 5-minute timeout, 50KB stdout cap, and max 50 tool calls per script for safety.

25. `delegate_task` (Subagents)

A Hermes tool that spawns independent child agents (subagents) to work on sub-tasks in parallel. Each subagent gets its own conversation context, terminal session, and restricted toolset. The parent agent only sees the final summary — all intermediate tool calls are hidden, keeping the parent's context window clean. You can spawn up to 3 subagents simultaneously, each working on a different task. Think of it as **hiring specialists** — one researcher searches the web, one coder fixes bugs, one writer drafts documentation — all working simultaneously. Subagents cannot spawn further subagents (max depth = 1) to prevent runaway agent proliferation.

26. `fact_store`

The interface to Hermes's holographic memory system. It supports six operations: `add` (store a new fact with entity tags and trust score), `search` (keyword lookup across all facts), `probe` (retrieve all facts about a specific entity like "Thanit"), `reason` (find connections across multiple entities simultaneously), `contradict` (detect conflicting claims), and `update/remove` (modify or delete facts). Each fact has a trust score (0-1) that increases when confirmed and decreases when contradicted. This is how Hermes builds **deep structured knowledge** — not just chat history, but a cross-referenced, confidence-scored knowledge base that improves over time.

27. Cron Job

A scheduled task that runs automatically at set time intervals, named after the Unix `cron` daemon. In Hermes, you create one with `hermes cron create` — for example: "every morning at 8am, summarize my notes" or "every 30 minutes, check this API for new data." The scheduler runs the prompt you specify at the configured time, without any user interaction. Cron jobs can deliver results to specific channels (Telegram, Discord) or back to the current chat. They support one-shot schedules (ISO timestamp) and recurring patterns (`0 9 * * *` for daily at 9am). Think of them as **automated alarms that do work instead of just ringing**.

28. Gateway

The bridge between Hermes and external messaging platforms (Telegram, Discord, Slack). It runs as a background service (`hermes gateway run` for foreground, `hermes gateway start` for background daemon) and translates between platform-specific message formats and Hermes's internal format. When someone sends a message to your Telegram bot, the Gateway receives it via webhook, converts it to Hermes format, gets the agent's response, converts it back to Telegram format, and sends the reply. It also handles media (images, voice messages), commands (`/model` , `/skills`), and reconnection if the connection drops. Without the Gateway, Hermes is terminal-only.

29. MCP (Model Context Protocol)

A standard protocol developed by Anthropic for connecting external tools and services to AI agents through a consistent interface. In Hermes, MCP servers appear as additional tools the agent can use — configured in `config.yaml` under `mcp.servers` . For example, an MCP server for GitHub would let Hermes create issues,

review PRs, and merge branches without needing custom code for each operation. MCP defines three primitives: tools (functions the agent can call), resources (data the agent can read), and prompts (templates the agent can use). Think of MCP as **USB-C for AI tools** — one standard connector, many different devices.

30. Context Window

The maximum amount of text (measured in **tokens**) that an AI model can process in a single conversation turn. Think of it as the agent's **short-term memory capacity** — how much it can hold in its head at once. GLM-5.1 has a 128K token context window (~96,000 words). When a conversation gets too long, Hermes uses **trajectory compression** to summarize older messages and free up space. If the context fills up, the agent loses track of earlier discussion. This is why long coding sessions sometimes "forget" what was discussed at the beginning — the context window was exceeded and old content was compressed.

31. Token

The basic unit of text that AI models process. A token is roughly $\frac{3}{4}$ of a word in English — "Hermes" is one token, "understanding" might be two tokens ("under" + "standing"), and common words like "the" are single tokens. Models charge by token count and have context windows measured in tokens. When you see "128K tokens," that means the model can process about 96,000 words in one conversation. Tokenization is model-specific — different models may split the same text into different numbers of tokens. This is why API costs are measured in tokens, not words, and why choosing an efficient model matters for long conversations.

32. `asyncio`

Python's built-in library for writing asynchronous (concurrent) code using the `async` / `await` syntax. Hermes uses it extensively because the agent needs to do multiple things at once — wait for API responses from the model, handle tool calls that take time, process user input, and manage gateway connections — all without freezing the interface. `asyncio` provides an **event loop** that switches between tasks: while one task is waiting for a network response, another task runs. This is called cooperative multitasking. Without `asyncio`, Hermes would freeze every time it made an API call, making it unusable for real-time chat.

33. `__init__`

Python's constructor method — the code that runs when a new object is created. When you see `class AIAgent:` followed by `def __init__(self, config, model, tools):`, that's the setup code that initializes every new `AIAgent` instance. It sets `self.max_iterations = 50`, `self.active_model = model`, `self.tools = ToolRegistry(tools)`, and all other starting values. Think of it as the **startup sequence** for an object — without it, the object doesn't exist. In Hermes, the `AIAgent.__init__` method is where the entire agent is wired together: state, tools, model interface, memory, and skill loader are all connected here.

34. `os.environ`

Python's dictionary-like interface to the operating system's environment variables. When you set `OLLAMA_API_KEY=sk-xxx` in your `.env` file, Hermes loads it into `os.environ["OLLAMA_API_KEY"]` at startup. Any Python code in the agent can then read it with `os.environ.get("OLLAMA_API_KEY")`. It's the bridge between shell configuration and Python code. Environment variables are inherited by child processes (subprocesses,

Docker containers), which is why `host.docker.internal` works — Docker sets it as an env var that Firecrawl can read. Think of `os.environ` as **system-wide sticky notes** that any program can read but only the owner can change.

35. Shell (Terminal / Command Line)

The text-based interface where you type commands to control your computer. On macOS it's Terminal.app (running zsh or bash); on Windows it's PowerShell or WSL (Windows Subsystem for Linux). When the handbook says "open a shell" or "open your terminal," it means open this command-line window. The shell interprets your commands, runs programs, and shows output. Common shells include bash, zsh, and PowerShell — they all do the same job but with slightly different syntax. In this handbook, most commands assume bash/zsh on macOS or WSL on Windows. The shell is also where you configure environment variables in `~/.zshrc` or `~/.bashrc`.

36. Daemon

A background process that runs continuously without user interaction, named after Maxwell's demon in physics (not the religious concept). Hermes's Gateway is a daemon — it starts with `hermes gateway start` and sits in the background listening for incoming messages from Telegram/Discord. You don't interact with it directly; it just runs. On macOS, system daemons are managed by `launchd`; on Linux, by `systemd`. Docker containers also run as daemons. The key difference from a regular process: daemons have no terminal input, they log to files, and they restart automatically if they crash. Think of a daemon as a **night watchman** — always on duty, never needs supervision.

37. Subprocess

A separate process spawned by a parent program. When Hermes runs `terminal("pip install requests")`, it creates a subprocess to execute that command in a new shell. The parent (Hermes) can read the subprocess's stdout output, check its exit code to see if it succeeded, and decide what to do next. Subprocesses are isolated — if a subprocess crashes, the parent keeps running. In Python, `subprocess.run()` is the most common way to create one. Think of it as **opening a mini terminal inside your agent** — it runs a command, captures the output, and closes when done. This is different from a daemon, which runs forever.

38. Environment Variable

A named value stored in the operating system's environment, accessible by all programs running in that environment. `OLLAMA_API_KEY`, `HOME`, `PATH`, and `DISCORD_BOT_TOKEN` are all environment variables. They're set in `.env` files (for application secrets) or shell profiles like `~/.zshrc` and `~/.bashrc` (for system paths). Programs read them with `os.environ.get("KEY")` in Python or `$KEY` in shell scripts. Environment variables are **inherited by child processes** — when Hermes starts a subprocess, all env vars are passed down. This is how Docker containers receive their configuration (API keys, model names, port numbers) without hardcoding values in the code.

39. Volume (Docker Volume)

A Docker mechanism for persisting data across container restarts and rebuilds. When you define `volumes: - ./qdrant_data:/qdrant/storage` in `docker-compose.yml`, you're telling Docker to store Qdrant's database files

on your host machine's filesystem (`./qdrant_data`) instead of inside the container. Without volumes, all container data is **ephemeral** — gone when the container is deleted or recreated. With volumes, your data survives `docker compose down` && `docker compose up` . This is critical for Qdrant (your vector database) and Firecrawl's Postgres database. Think of volumes as **external hard drives** you plug into your container — the data stays even if the container is thrown away.

40. Port Mapping (`ports:` in Docker Compose)

The rule that connects a port inside a Docker container to a port on your host machine. The syntax is `"HOST_PORT:CONTAINER_PORT"` . For example, `ports: - "3002:3002"` means "container port 3002 → host port 3002." When you access `localhost:3002` in your browser, Docker forwards it to port 3002 inside the Firecrawl container. The two numbers don't have to match — you could write `"8080:3002"` to access Firecrawl on port 8080 locally while it listens on 3002 internally. This is how you reach services running inside Docker from your machine. Without port mapping, containers are completely isolated from the outside world.

41. Container vs. Image

Two Docker concepts that beginners constantly confuse. An **image** is a frozen, read-only template — the blueprint for a container. It's built from a `Dockerfile` and stored in a registry (like Docker Hub). A **container** is a running instance of that image — it has a writable layer on top, its own filesystem, network, and processes. Think of it like this: an image is a recipe; a container is the cake you baked from it. You can run multiple containers from the same image (bake many cakes from one recipe). When you `docker compose up` , Docker pulls images and starts containers. When you `docker compose down` , containers are stopped and removed, but images remain.

42. WSL2 (Windows Subsystem for Linux 2)

A Windows feature that runs a real Linux kernel alongside Windows, without a traditional virtual machine. It lets you use bash, apt, SSH, and all Linux tools natively on Windows. Docker Desktop uses WSL2 under the hood to run Linux containers. When the handbook says "install WSL2," it means running `wsl --install -d Ubuntu` in PowerShell (as Administrator) to enable this compatibility layer. WSL2 gives you a full Ubuntu environment where you can install Hermes, run bash scripts, and use Docker — all from within Windows. Your Windows files are accessible at `/mnt/c/` , and your Linux files are at `~/` . Think of WSL2 as **Linux inside Windows without the overhead of a VM**.

43. `localhost` VS `host.docker.internal`

Two different addresses that beginners constantly confuse, causing the #1 Docker networking error. `localhost` (or `127.0.0.1`) means "this machine" — but **inside a Docker container**, "this machine" means "this container," not your actual computer. So `localhost:11434` inside a container points to the container's own port 11434, not Ollama running on your laptop. `host.docker.internal` is Docker's special DNS name that always resolves to **the actual computer running Docker**. If Firecrawl needs to reach Ollama on your machine, it must use `host.docker.internal:11434` , not `localhost:11434` . On Windows and macOS Docker Desktop, this works automatically. On Linux, you must add `extra_hosts` to `docker-compose.yml` .

44. Dependency (`depends_on:` in Docker Compose)

A directive that tells Docker which services must be healthy before another service starts. `depends_on: [redis, postgres]` means "start Redis and Postgres first, then start this service." In the handbook's `docker-compose.yml`, Firecrawl depends on Redis, Playwright, and RabbitMQ — because it needs them for job queues, browser automation, and message passing. Without `depends_on`, Docker starts all containers simultaneously, and Firecrawl might crash because it tries to connect to Redis before Redis is ready. There's also a `condition: service_healthy` variant that waits until the dependency passes a health check, not just until it starts. Think of it as "don't start the car until the engine has oil."

45. `_maybe_reflect`

An internal Hermes method that decides whether the agent should self-reflect after completing a task. Reflection is the core of the **self-improvement loop**: after a complex task, `_maybe_reflect` checks if the task was significant enough to warrant review. If yes, it triggers the agent to: (1) review its own output for mistakes, (2) identify what could be improved, (3) optionally write or patch a skill file capturing the lesson. This is how Hermes gets better over time — not just by storing more data, but by actively reviewing its performance and updating its procedures. Think of it as a **post-game film review** for AI.

46. `_prompt_approval`

The security gate inside Hermes that protects against dangerous operations. Before executing commands that could cause harm — like `rm -rf`, `sudo`, `DROP TABLE`, or writing to system directories — the agent calls `_prompt_approval` to ask your permission. You can configure which tools require approval in `config.yaml` using the `approval_required` list. In TUI mode, this appears as a confirmation prompt. In gateway mode (Telegram/Discord), it sends a message asking for your go-ahead. Think of it as the **seatbelt** — it only activates for risky operations, and you can customize what counts as "risky." This prevents the agent from accidentally deleting files or sending unauthorized messages.

47. HRR (Holographic Reduced Representation)

The mathematical foundation of Hermes's holographic memory. HRR is a technique from cognitive science that encodes relationships between concepts as vectors. You can **bind** two vectors together (e.g., "Paris" + "IS-CAPITAL-OF" → a new vector) and later **unbind** them (given the bound vector and the role "IS-CAPITAL-OF," recover "Paris"). Unlike simple vector similarity (which only finds "things that are alike"), HRR preserves **relational structure** — you can ask "what is Paris the capital of?" and get "France" back. This is why holographic memory can reason across entities while plain vector databases cannot. Think of HRR as **DNA for knowledge** — it encodes not just what things are, but how they relate.

48. `delegate_task` vs `execute_code`

Two tools that beginners often confuse but serve completely different purposes. `execute_code` runs a Python script in a sandbox and returns the output — it's like **hiring one worker** who follows your exact instructions and reports back. The script runs in the parent agent's context with access to Hermes tools. `delegate_task` spawns an entirely separate AI agent (subagent) with its own context, tools, and instructions — it's like **hiring a specialist consultant** who works independently, makes its own decisions, and gives you a final report. Use `execute_code`

for deterministic data processing; use `delegate_task` for research, creative work, or anything needing independent judgment.

49. Trajectory Compression

The process of summarizing a long conversation (trajectory) into a compact lesson that fits in the agent's memory. After a complex task involving many tool calls and results, Hermes compresses the full history into a short note like "For Docker networking, always use `host.docker.internal` instead of `localhost` inside containers." These compressed lessons are stored in `fact_store` or `SKILL.md` files and injected into future sessions. This is how Hermes turns experience into knowledge — the agent doesn't just remember what happened, it extracts the **generalizable lesson**. Without trajectory compression, every session would start from scratch, and the agent would repeat the same mistakes.

50. Skill vs. Plugin vs. MCP Server

Three extension mechanisms that people constantly mix up, even though they serve fundamentally different purposes:

- **Skill:** A markdown file (`SKILL.md`) that gives the agent instructions on *how* to do something. It contains triggers, steps, and pitfalls. Created by you or auto-generated by the Curator. Lives in `~/.hermes/skills/`. Like a **recipe** — instructions for using tools you already have.
 - **Plugin:** A Python code package that adds new capabilities (tools, model providers, features) to Hermes. Installed via `hermes plugin add <name>`. Like a **blender attachment** — it physically adds a new capability to the machine.
 - **MCP Server:** An external service that speaks the Model Context Protocol, providing tools the agent can use over a standardized interface. Runs separately from Hermes (often in Docker). Like **calling a contractor** — they have their own tools and workplace, you just give them tasks.
-